Under What Conditions Is Encrypted Key Exchange Actually Secure?

Jake Januzelli[1], Lawrence Roy*[2], and Jiayu Xu[3]

[1]Oregon State University, januzelj@oregonstate.edu
[2]Aarhus University, ldr709@gmail.com
[3]Oregon State University, xujiay@oregonstate.edu

Abstract

A Password-Authenticated Key Exchange (PAKE) protocol allows two parties to agree upon a cryptographic key, in the setting where the only secret shared in advance is a low-entropy password. The standard security notion for PAKE is in the Universal Composability (UC) framework. In recent years there have been a large number of works analyzing the UC-security of Encrypted Key Exchange (EKE), the very first PAKE protocol, and its One-encryption variant (OEKE), both of which compile an unauthenticated Key Agreement (KA) protocol into a PAKE.

In this work, we present a comprehensive and thorough study of the UC-security of both EKE and OEKE in the *most general* setting and using the *most efficient* building blocks:

1. We show that among the five existing results on the UC-security of (O)EKE using a general KA protocol, all are incorrect;
2. We show that for (O)EKE to be UC-secure, the underlying KA protocol needs to satisfy several additional security properties: though some of these are closely related to existing security properties, some are new, and all are missing from existing works on (O)EKE;
3. We give UC-security proofs for EKE and OEKE using Programmable-Once Public Function (POPF), which is the most efficient instantiation to date and is around 4 times faster than the standard instantiation using Ideal Cipher (IC).

Our results in particular allow for PAKE constructions from post-quantum KA protocols such as Kyber. We also present a security analysis of POPF using a new, weakened notion of *almost UC* realizing a functionality, that is still sufficient for proving composed protocols to be fully UC-secure.

*Author partially supported by the DOE CSGF (grant DE-SC0019323), the Danish Independent Research Council (grant DFF-0165-00107B "C3PO"), and the DARPA SIEVE program (contract HR001120C0085 "FROMAGER"). Any opinions, findings and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of DARPA. Distribution Statement "A" (Approved for Public Release, Distribution Unlimited).

Contents

1 Introduction

A *Password-Authenticated Key Exchange (PAKE)* protocol allows two parties to agree upon a cryptographic key in the setting where the only information shared in advance is a low-entropy password. Crucially, such protocols must be secure against man-in-the-middle adversaries that can arbitrarily modify the protocol messages sent between the two parties. PAKE protocols — and their extensions such as asymmetric PAKE and threshold PAKE — provide significant advantages over the traditional "password-over-TLS" approach to authentication, in that PAKE does not require transmission of public keys and thus does not rely on PKI. Recent years have witnessed an increasing amount of interest in PAKE from both academia and industry, and with the recent standardization process by the IETF [Cry20], practical implementations — in particular, integration of PAKE protocols and TLS — is under active consideration [DFG+23].

Security notions for PAKE. Since passwords have low entropy, an inevitable attack that must be taken into account by any security definition for PAKE is *online guessing*, where the adversary guesses a password pw^* and executes Alice's algorithm on pw^* while communicating with Bob; if pw^* is indeed Bob's password, then the adversary learns Bob's session key. The game-based security notion for PAKE [BPR00] requires that online guessing is the best possible attack; if the password is uniformly drawn from the dictionary Dict, then the adversary's advantage is at most negligibly greater than $q/|\mathsf{Dict}|$ for q online sessions. While this notion achieves a basic level of PAKE security, it comes with two significant drawbacks: First, it does not provide any security guarantee under parallel composition. This is especially devastating for PAKE, since (1) applications of PAKE generally involve a large number of users running the protocol in parallel, and (2) PAKE is usually composed with some symmetric-key primitives and rarely used in a standalone manner. Second, it fails to model password reuse across multiple accounts, which is (unfortunately) all too common in real life.

For these reasons, the game-based PAKE security definition has been superseded by the definition in the Universal Composability (UC) framework [CHK+05], where security remains under arbitrary composition of protocols. Password reuse is addressed by letting the environment choose the password, rather than assuming a specific distribution over the dictionary. Over the years the UC notion has become the de facto security standard for PAKE; for example, all candidates in the second round of the IETF standardization competition have a UC security analysis [AHH21, ABB+20, JKX18, HL19].

Encrypted Key Exchange (EKE). The very first PAKE protocol ever proposed is *Encrypted Key Exchange (EKE)* by Bellovin and Merritt in 1992 [BM92]. It compiles any unauthenticated Key Agreement (KA) protocol into a PAKE by encrypting all messages using a private-key encryption scheme with the password as the key (and the receiver can decrypt and recover the message in the underlying KA protocol if it holds the correct password). In the standard instantiation of EKE, the encryption scheme is Ideal Cipher (IC). When instantiated with a 2-round KA protocol, we immediately obtain a PAKE protocol that also has 2 rounds.[1]

Aside from its historical significance, EKE continues to play an important role in PAKE design. Protocols such as SPAKE1 and SPAKE2 [AP05] are essentially EKE instantiated with specific private-key encryption schemes and KA protocols, and others such as CPace [HL19] inherit some design ideas from EKE. It is fair to say that EKE and its variants form one of the two major

[1]The term "round" has various definitions in the literature. Here by 2-round we mean 2-flow, i.e., P sends a message to P', and P', upon receiving the message from P, sends a message back. If the two messages can be sent simultaneously, we call the protocol 1-simultaneous round.

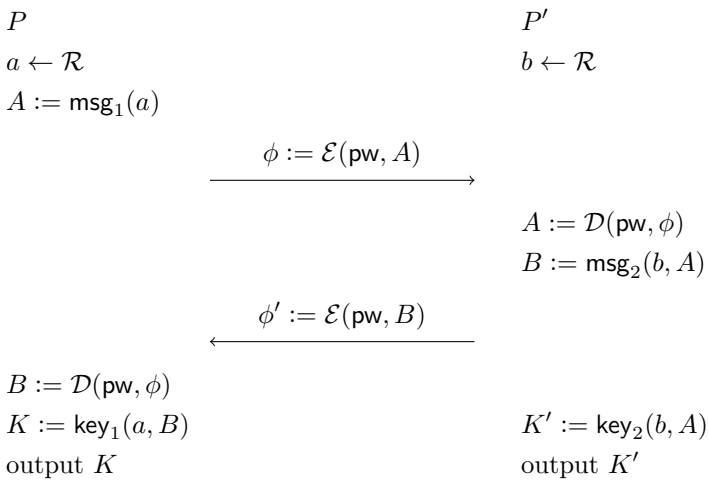

Figure 1: EKE with a 2-round KA protocol

paradigms for PAKE protocols (the other is Smooth Projective Hash Function (SPHF)-based PAKEs [GL03, GK10, KV11] which are generally less efficient).

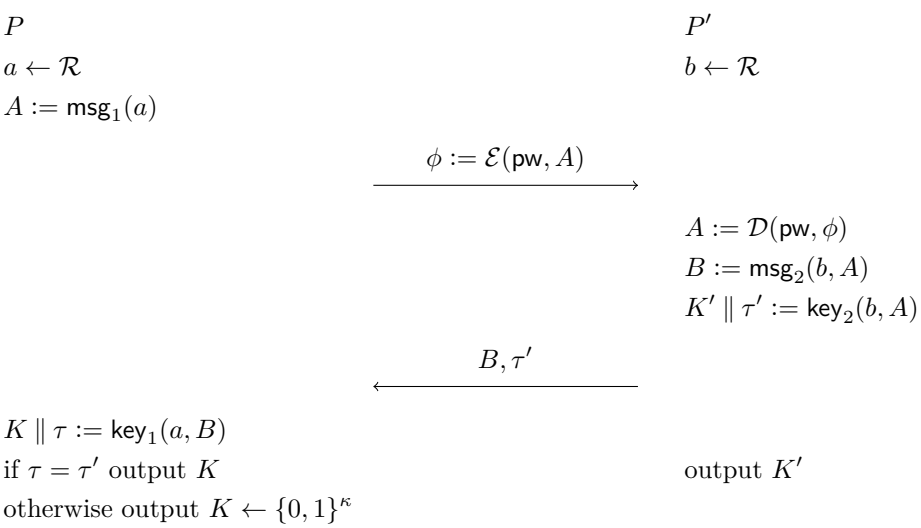

Figure 2: OEKE with a 2-round KA protocol. This version requires a KA protocol with long key. In OEKE-PRF the KA key K has length κ, PAKE session key is $\mathsf{PRF}_K(0)$, and $\tau = \mathsf{PRF}_K(1)$. In OEKE-RO $\mathsf{PRF}_K(x) = H(K, x)$

One-encryption EKE (OEKE). The following variant of EKE first appeared in [BCP03]: only the P-to-P' message is encrypted, and upon receiving the message from P, P' sends a *plaintext* KA message to P together with an authenticator that is the second part of K' (where K' is KA key of P'); the session key of P' is the first part of K'.[2] P computes its KA key K and checks if the authenticator is the second part of K, and if so, outputs the first part of K as its session key. This

[2]Since each party has an output key in the KA protocol and an output key in the PAKE protocol (the latter being its eventual output), it might be confusing what a "key" refers to. Throughout this work we use the terms "KA key" and "(PAKE) session key" to distinguish them.

protocol is called OEKE (O for "One-encryption").

OEKE requires that the underlying KA protocol have key length longer than PAKE session key length κ. If we only have a KA protocol whose key K is κ-bit long, one way to implement OEKE is to define the session key as $\mathsf{PRF}_K(0)$ and the authenticator as $\mathsf{PRF}_K(1)$ (where PRF is a PRF with κ-bit outputs) — that is, we compile the KA protocol into another KA protocol whose key is $\mathsf{PRF}_K(0) \,\|\, \mathsf{PRF}_K(1)$, and use the latter in OEKE. We call this protocol OEKE-PRF, and if the PRF is defined as $\mathsf{PRF}_K(x) = H(K, x)$ (where H is an RO), we call it OEKE-RO. All existing works on the UC-security of OEKE follow the OEKE-PRF paradigm [SGJ23, BCP+23].[3]

While OEKE is not as round-efficient as EKE if the underlying KA protocol is 1-simultaneous round (such as Diffie-Hellman), it avoids one instance of IC and is thus more efficient in computation cost than EKE.

1.1 Existing Security Analyses of (O)EKE

Despite its deceiving simplicity and its pivotal role in over 30 years of study of PAKE, a satisfactory formal security analysis of EKE has long been elusive. The original EKE paper in 1992 [BM92] does not provide a security proof (in fact, even a formal security definition did not exist until 2000 [BPR00]). [BPR00], in addition to its main contribution of proposing the game-based security definition for PAKE, proves that EKE satisfies this definition under the Computational Diffie-Hellman (CDH) assumption if the underlying KA protocol is hashed Diffie-Hellman and the underlying private-key encryption scheme is IC. However, this result suffers from three drawbacks:

- The result assumes a specific KA protocol (hashed Diffie-Hellman) is used in EKE, and it is not immediately clear which properties one should require of an arbitrary KA protocol for EKE to be secure. Pinpointing such properties is especially beneficial if we want to use a post-quantum KA protocol to achieve a PAKE under post-quantum assumptions — which was not a main concern when [BPR00] was published but has gained much importance and attention since then.
- The result assumes that the underlying private-key encryption scheme is IC. While it is known that an 8-round Feistel network is indifferentiable from an IC in the Random Oracle Model (ROM) [DS16], using an IC in EKE results in a PAKE protocol with significant cost. In particular, since the KA protocol is supposed to be hashed Diffie-Hellman, an IC *onto a group* is needed, which in turn requires 4 RO-hash onto a group operations which are inefficient.
- Finally, the result is in the game-based setting, which as we have mentioned has some critical drawbacks and has been superseded by the UC definition.

[BCP03] provides a security proof for OEKE, which suffers from the same drawbacks.

UC analyses of (O)EKE. It was not until 18 years later that EKE was formally proven secure in the UC framework [DHP+18]. (For OEKE, we had to wait 20 years [SGJ23, BCP+23].) Since then, there have been a number of UC analyses of (O)EKE, each with its own instantiation. Below we give a brief summary of existing works in this domain:

[3] Formally, [SGJ23] and [BCP+23] use a Key Encapsulation Mechanism (KEM) rather than a KA protocol as a building block, but a KEM is equivalent to a 2-round KA protocol: the first KA message is the KEM public key, the second KA message is the KEM ciphertext, and the KA key is the KEM key. (See, e.g., [SGJ23, Section 2.2].) [SGJ23] calls OEKE "EKE-KEM".

	proposed	game-based analysis	UC analysis
EKE	1992 [BM92]	2000 [BPR00]	2018 [DHP+18]
OEKE	2003 [BCP03]	2003 [BCP03]	2023 [SGJ23, BCP+23]

Table 1: Timeline of security analyses of (any instantiation of) EKE and OEKE

- [MRR20, Theorem 10] claims that EKE is UC-secure provided that the underlying KA protocol is secure and *pseudorandom* (the KA messages are indistinguishable from uniformly random) and the underlying encryption scheme is *Programmable-Once Public Function (POPF)*, which can be seen as a generalization of IC. [MRR20] introduces the necessary (game-based) properties of the POPF for EKE to be UC-secure. Using a POPF instead of an IC drastically reduces costs, as a POPF can be instantiated by a 2-round Feistel network (as opposed to 8-round in IC). Furthermore, since [MRR20] uses *any* secure and pseudorandom KA protocol, it allows for an instantiation of EKE under post-quantum assumptions. Unfortunately, as we will see below, the main result in [MRR20] is incorrect in three different ways.

- [SGJ23, Theorem 2] also claims UC-security of EKE assuming the underlying KA protocol is secure and pseudorandom, and the underlying encryption scheme is *Half Ideal Cipher (HIC)*. [SGJ23] proposes a UC definition for HIC and shows that it is realized by a 2-round Feistel network, except that the hash-and-XOR operation in the second round is replaced by an IC *over* 2κ*-bit strings*. Thus, while HIC (unlike POPF) requires an IC, it achieves UC-security and is easier to use in other contexts. However, we will show that this EKE result is incorrect in two different ways.
  In addition to the result above, [SGJ23, Theorem 3] claims that a certain variant of OEKE is UC-secure, again using HIC and a secure and pseudorandom KA protocol. Unfortunately, it is not entirely clear whether the protocol analyzed in [SGJ23] is OEKE-PRF or OEKE-RO, and there seems to be a mismatch between the theorem statement and its proof, as mentioned in [SGJ23, Footnote 14]:

  > *The proof below assumes a version of the protocol which uses* $\mathsf{prf}(K, \cdot)$ *to derive the authenticator* τ *and the session key [...] This version of the protocol requires an additional assumption on KEM. We will update the proof shortly to reflect the modified protocol and get rid of the additional assumption.*

  Using our terminology, the statement of [SGJ23, Theorem 3] claims the UC-security of OEKE-RO, while its proof is about the UC-security of OEKE-PRF; the latter requires an additional assumption on the underlying KA protocol (which is overlooked in the proof). However, it is never specified what this "additional assumption" is! As such, at least the security proof of [SGJ23, Theorem 3] seems incomplete.[4] There is also a separate issue that renders this result incorrect no matter whether OEKE-PRF or OEKE-RO is considered.

- [BCP+23, Theorem 1] proves the UC-security of EKE using IC and a secure and pseudorandom KA protocol, with the session key being an RO hash of the KA key together with the PAKE transcript. This result has a flavor similar to [MRR20, Theorem 10] and [SGJ23, Theorem 2], but is less efficient (it uses IC rather than POPF or HIC). Close scrutiny shows that this result is also incorrect: we will give two actual attacks that break the UC-security in different ways, and also show a flaw in a reduction in the hybrid proof that results in an incorrect bound on

[4] Looking ahead, this "additional assumption" is what we call *collision resistance* for the underlying KA protocol (see Sect. 3.1).

the tightness of the security.
[BCP+23, Theorem 2] proves the UC-security of OEKE-RO using IC and a secure and pseudorandom KA protocol, with the session key being an RO hash of the KA key together with the PAKE transcript. We will show that this security statement is also incorrect.

There are several additional analyses of EKE with *specific* KA protocols: [DHP+18, Theorem 6] shows that EKE using hashed Diffie-Hellman is UC-secure under CDH, and [LLHG23, Theorem 2] shows that EKE using hashed *twin* Diffie-Hellman (where one party sends two group elements g^{a_1} and g^{a_2} and the other sends a single group element g^b, and the KA key is $H(g^{a_1 b}, g^{a_2 b})$) is *tightly* UC-secure under CDH. While both security statements are correct, we will see that the proof of [DHP+18, Theorem 6] contains a flawed reduction in the hybrids very similar to the tightness issue in [BCP+23, Theorem 1]. Finally, the concurrent and independent work of [ABJS24] considers OEKE with *splittable* underlying KA protocols, meaning that the first KA message can be encoded as a bitstring and a group element.

See Table 2 for a summary of existing UC security analyses of (O)EKE.

We can see that despite a large amount of works analyzing the UC-security of (O)EKE in various flavors, all existing analyses using a general KA protocol are incorrect. This brings us to the question we ask in the title:

Under what conditions is EKE (and its variants) actually secure?

1.2 Our Contributions

In this work, we present a comprehensive study of the UC-security of EKE and its one-encryption variant. We consider the protocols instantiated with the *most efficient* encryption scheme, i.e., POPF; and the *most general* KA protocols, which allows us to obtain a wide range of PAKE protocols, including ones based on post-quantum assumptions. Concretely, our contributions are as follows:

Attacks on previous instantiations of (O)EKE. In Sect. 3 we motivate our KA security properties with several attacks:

- In Sect. 3.1 we show that OEKE (including its PRF and RO variants) is insecure if the underlying KA protocol does not satisfy *collision resistance* (Thm. 3.1). This implies that the proof of [SGJ23, Theorem 3] is incorrect;
- In Sect. 3.2 we show that EKE is insecure if the underlying KA protocol does not satisfy *pseudorandom non-malleability* (Thm. 3.6), which implies that [MRR20, Theorem 10] and [SGJ23, Theorem 2] are false;
- In Sect. 3.3 we show that EKE is insecure if the underlying KA protocol does not satisfy *strong pseudorandomness* (Thm. 2.5), which implies that [MRR20, Theorem 10] and [BCP+23, Theorem 1] are false.

Although some of these KA notions are related to existing security properties in the Key-Encapsulation Mechanism (KEM) literature, to the best of our knowledge we are the first to formalize strong pseudorandomness and the first to notice the necessity of these properties for the security of (O)EKE. In Sect. A we present several additional attacks and flaws in existing works; in particular, Sect. A.3 shows that pseudorandom non-malleability is also needed in OEKE, and Sect. A.4 presents a further attack that only applies to EKE using HIC or POPF (instead of IC).

result	protocol analyzed	KA protocol	encryption scheme	note
[DHP+18, Theorem 6]	EKE	hashed Diffie-Hellman (PAKE transcript included in hash)	IC	proof flawed (Sect. A.1)
[MRR20, Theorem 10]	EKE	general	POPF	result incorrect (Sects. 3.2 and 3.3, Sect. A.4)
[SGJ23, Theorem 2]	EKE	general	HIC	result incorrect (Sects. A.4 and 3.2)
[SGJ23, Theorem 3]	OEKE-?	general	HIC	result ambiguous, unclear if OEKE-PRF or OEKE-RO; security proof incorrect (Sect. 3.1); either way result also incorrect (Sect. A.3)
[LLHG23, Theorem 2]	EKE	hashed twin Diffie-Hellman (password included in hash)	IC	
[BCP+23, Theorem 1]	EKE	general (PAKE transcript included in hash)	IC	result incorrect (Sect. 3.3, Sects. A.1 and A.3)
[BCP+23, Theorem 2]	OEKE-RO	general (PAKE transcript included in hash)	IC	result incorrect (Sect. A.3)
our Thm. 5.4	EKE	general	POPF	
our Thm. 5.7	OEKE	general	POPF	

Table 2: UC security analyses of (O)EKE. Flawed analyses are marked in grey

POPF and "almost UC". The results in Sect. 3 are sufficient to recover correct security proofs of (O)EKE using IC as the encryption mechanism (those proofs are sketched in Sect. B). However, the most efficient instantiation of (O)EKE uses POPF as defined by [MRR20], so as an additional contribution we give our security proofs in the POPF setting. At a high level, a POPF is a function family $\{F_\phi\}$ where each F_ϕ is random everywhere, except that when generating ϕ one can program a single input/output pair (x, y) such that $F_\phi(x) = y$. To capture this notion of "uncontrollable outputs", it is required that for any weak pseudorandom function W_k, $W_k(F_\phi(x'))$ is pseudorandom for $x' \neq x$. A POPF can be viewed as a randomized version of IC, where programming $F_\phi(x)$ to be y is analogous to encrypting y under key x (resulting in ciphertext ϕ), and evaluating F_ϕ at x is analogous to decrypting the ciphertext ϕ with key x. However, in the case of POPF there can be many ϕ such that $F_\phi(x) = y$, whereas for IC there is only a single ϕ such that $\mathcal{E}(x, y) = \phi$. Although [MRR20] used game-based security properties for POPF in the analysis of EKE, they separate honest simulation and extraction [MRR20, Definitions 7.2-7.3], which is insufficient to analyze the case of a man-in-the-middle adversary where the simulator may have to do both simultaneously.

An obvious strategy to recover a proof of security is to define a UC functionality for POPF, and then use the UC POPF functionality in (O)EKE. Unfortunately, the POPF in [MRR20] does not seem to realize an appropriate UC functionality. The problem is the proof that $\{F_\phi\}$ has "uncontrollable outputs" reduces to the security of the weak PRF W_k, during which the non-tight reduction needs to make a guess over the adversary's random oracle queries. In other words, we reduce to the security of some schemes built *on top* of POPF, not to an *underlying assumption*. If we attempt to show the POPF UC-realizes a functionality, the simulator must make the RO behavior consistent with the POPF functionality, which requires it to guess the RO query, making the simulator fail. Crucially, while a loose *reduction* is sufficient to prove security, a UC *simulator* must correctly simulate the real world with overwhelming probability, as the environment could otherwise tell that it is in the ideal world with significant probability.

We solve this by showing that POPF "almost UC-realizes" a functionality $\mathcal{F}_{\text{POPF}}$, where instead of full UC-realization of a functionality, we only require that *a specific class* of higher-level protocols that use $\mathcal{F}_{\text{POPF}}$ remain secure if the real POPF is used instead.[5] We are able to achieve this weaker security notion, as now the guess of an RO query can appear in a *hybrid* in the proof of security of the higher-level protocol, which is not an issue. (A similar technique was used in [CDG+18].) At the same time, the almost UC notion is sufficient for the higher-level protocol; in particular, (O)EKE built on top of our POPF is (fully) UC-secure. We expect the notion of "almost UC-realization" to have applications beyond the present work. E.g, the security loss when replacing an ideal functionality with a protocol that UC-realizes it is additive by the UC composition theorem [Can01, Theorem 3]: almost-UC can be used to recover secure composition in cases where one expects a super-additive security loss.

Analysis of (O)EKE using POPF and suitable KA protocols. Equipped with our almost UC modeling of POPF and our additional KA properties, we now turn to our main constructive contribution: an analysis of both EKE and OEKE in the UC framework, using a general KA protocol. There are a large number of existing analyses of (O)EKE that vary according to:

1. The procedure by which PAKE session keys are derived.
2. The properties the KA satisfies, other than the standard notions of correctness and security.
3. The encryption mechanism used, whether that be IC, HIC, or POPF.

In this work, we clarify exactly which choices from these three dimensions result in a secure protocol, along the way obtaining the most efficient instantiations of (O)EKE to date:

1. *"Minimal" output function*: We show that for EKE using POPF, the session key needs to be a PRF of the second PAKE message under the KA key (we call this protocol EKE-PRF), and no other items need to be included; if we use IC instead, outputting the "raw" KA key suffices (provided that the KA protocol satisfies appropriate properties — see below). For OEKE, we show that the "raw" version is secure, and no items need to be hashed (again, provided that the KA protocol satisfies appropriate properties); this in particular covers both the PRF and RO variants.
2. *Exact KA properties*: We show that EKE-PRF is secure if the underlying KA protocol satisfies *strong pseudorandomness* and *pseudorandom non-malleability*, in addition to correctness and

[5] This idea of only requiring it to preserve the security of protocols built on top is similar to security-preserving samplers defined by [AWZ23]. The goal there is quite different however: the elimination of ROs from a 1-round protocol.

security. Additionally, we show that OEKE is secure if the underlying KA protocol satisfies pseudorandom non-malleability and *collision resistance* in addition to correctness and security, covering both the OEKE-PRF and OEKE-RO variants. In each case, we give concrete attacks showing that these additional KA properties are necessary for (O)EKE to be secure, and are not implied by existing standard properties.

3. *Most efficient instantiations*: Our main results about both EKE and OEKE use POPF, which are the most efficient instantiations to date. This provides significant efficiency advantage over the traditional IC implementation, as POPF only requires 2 rounds of Feistel network (as opposed to 8 in IC). To compare, the HIC construction of [SGJ23] requires a Feistel rounds to use an IC over 2κ-bit strings, while our construction only requires a random oracle. Though an IC over bitstrings is much less troublesome than an ideal cipher over curve points, it is still not trivial to instantiate — most block ciphers are designed for κ-bit block size, not 2κ. Suitable choices will either be less efficient or less conservative than instantiating a random oracle with a standard hash function.

Our goal is to not only present *what* is exactly needed, but also develop a thorough understanding of *why* exactly they are needed. To achieve this, we include a large number of comments and explanations along with our security proofs and attacks, as well as a comprehensive summary at the end of the paper (Sect. 6.2).

In addition to the theoretical advantages outlined above, our approach allows practitioners looking to implement (O)EKE to avoid analyzing the (O)EKE protocol as a whole, and instead merely verify whether their chosen KA protocol satisfies a few properties, which is much easier.

2 Preliminaries

For an (efficiently samplable) set S, we write $x \leftarrow S$ for sampling an element from S according to the uniform distribution. For an algorithm $\mathcal{A}$, we write $y \leftarrow \mathcal{A}(x; r)$ for running $\mathcal{A}$ on input x and randomness r, and obtaining $\mathcal{A}$'s output y; if $\mathcal{A}$ is deterministic, we instead write $y := \mathcal{A}(x)$. We use κ to denote the security parameter. For group operations, we use the multiplicative notation.

UC conventions. We assume w.l.o.g. that the session ID always includes the names of the two parties. Furthermore, for ideal objects such as RO and IC, we always assume that the session ID sid is part of the input (for IC, it is part of the key, i.e., a ciphertext is in the form of $\mathcal{E}(\mathsf{sid} \parallel k, m)$) and do not explicitly write them. This is for brevity only; we stress that in actual protocols the session ID should be included to achieve domain separation.

2.1 (Unauthenticated) Key Agreement Protocols

A 2-round *Key Agreement (KA)* protocol (henceforth KA protocol) consists of the following (deterministic) algorithms:

- $\mathsf{msg}_1(a) = A \in \mathcal{M}_1$: protocol-message function for party P
- $\mathsf{msg}_2(b, A) = B \in \mathcal{M}_2$: protocol-message function for party P'
- $\mathsf{key}_1(a, B) = K \in \mathcal{K}$: output function for party P
- $\mathsf{key}_2(b, A) = K' \in \mathcal{K}$: output function for party P'

In a standard execution of the protocol, party P samples randomness $a \leftarrow \mathcal{R}$ and sends $A := \mathsf{msg}_1(a)$ to party P'. P' then samples $b \leftarrow \mathcal{R}$, sends $B := \mathsf{msg}_2(b, A)$ to P, and outputs $K' := \mathsf{key}_2(b, A)$. Upon receiving B, P outputs $K := \mathsf{key}_1(a, B)$.

Definition 2.1. *A KA protocol is* ***correct*** *if*

$$\mathsf{key}_1(a, \mathsf{msg}_2(b, \mathsf{msg}_1(a))) = \mathsf{key}_2(b, \mathsf{msg}_1(a))$$

with overwhelming probability when $a, b \leftarrow \mathcal{R}$.

Security.

Definition 2.2. *A KA protocol is* ***secure*** *if the following two distributions are indistinguishable:*

$a \leftarrow \mathcal{R}$	$a \leftarrow \mathcal{R}$
$b \leftarrow \mathcal{R}$	$b \leftarrow \mathcal{R}$
$A := \mathsf{msg}_1(a)$	$A := \mathsf{msg}_1(a)$
$B := \mathsf{msg}_2(b, A)$	$B := \mathsf{msg}_2(b, A)$
$K := \mathsf{key}_1(a, B)$	$K \leftarrow \mathcal{K}$
output (A, B, K)	output (A, B, K)

Pseudorandomness. We also consider the notion of pseudorandomness in which the protocol messages are also indistinguishable from random.

Definition 2.3. *A KA protocol has* ***pseudorandom first message****, or* ***first pseudorandomness****, if the following two distributions are indistinguishable:*

$a \leftarrow \mathcal{R}$	
$A := \mathsf{msg}_1(a)$	$A \leftarrow \mathcal{M}_1$
output A	output A

Definition 2.4. *A KA protocol has* ***pseudorandom second message****, or* ***second pseudorandomness****, if the following two distributions are indistinguishable:*

$a \leftarrow \mathcal{R}$	$a \leftarrow \mathcal{R}$
$A := \mathsf{msg}_1(a)$	$A := \mathsf{msg}_1(a)$
$b \leftarrow \mathcal{R}$	
$B := \mathsf{msg}_2(b, A)$	$B \leftarrow \mathcal{M}_2$
output (A, B)	output (A, B)

The two pseudorandomness properties combined imply that the *joint distribution* of the two KA messages are indistinguishable from random.

Existing works on EKE use different terms for KA pseudorandomness and security, some of which are presented in the terms of KEM. See Table 3 for a comparison.

	first KA message / KEM public key pseudorandomness	second KA message / KEM ciphertext pseudorandomness	KA/KEM key pseudorandomness
[MRR20]	KA pseudorandomness		KA security
[SGJ23, Section 2.1]	KA random message		KA security
[SGJ23, Section 2.2]	KEM uniform public key	KEM anonymity[6]	KEM IND-security
[BCP+23]	KEM fuzziness	KEM anonymity	KEM indistingushability
this work	KA first pseudorandomness	KA second pseudorandomness	KA security

Table 3: Terminologies for KA pseudorandomness and security

Strong pseudorandomness. In the security analysis for EKE we need a stronger pseudorandomness property, which says that the second message is pseudorandom *even given the key derivation function used on it.*

Definition 2.5. *A KA protocol has* ***strong pseudorandom second message****, or* ***strong second pseudorandomness****, if the following two distributions are indistinguishable:*

$a \leftarrow \mathcal{R}$	$a \leftarrow \mathcal{R}$
$A := \mathsf{msg}_1(a)$	$A := \mathsf{msg}_1(a)$
$b \leftarrow \mathcal{R}$	
$B := \mathsf{msg}_2(b, A)$	$B \leftarrow \mathcal{M}_2$
$K := \mathsf{key}_1(a, B)$	$K := \mathsf{key}_1(a, B)$
output (A, B, K)	output (A, B, K)

Henceforth we may call the combination of first and second pseudorandomness *pseudorandomness*, and the combination of first and strong second pseudorandomness *strong pseudorandomness.*

To our knowledge, strong pseudorandomness does not directly correspond to any existing KA/KEM security notion, and as such has not appeared in any existing works on EKE that use general 2-round KA. In Sect. 3.3 we show strong pseudorandomness is necessary for the security of EKE (but not OEKE) and is not implied by the standard notions of security and pseudorandomness. However, a KEM that satisfies both SPR-CCA$_{\mathcal{S}}$ *and* SMT-CCA$_{\mathcal{S}}$ security from [Xag22] also satisfies strong pseudorandomness by a simple hybrid argument.

Example: Diffie-Hellman. The *plain (resp. hashed) Diffie-Hellman KA* is defined as follows: fix some cyclic group $(\mathbb{G}, p, g)$, and the relevant spaces are $\mathcal{R} = \mathbb{Z}_p$, $\mathcal{M}_1 = \mathcal{M}_2 = \mathbb{G}$, and $\mathcal{K} = \mathbb{G}$ (resp. $\mathcal{K} = \{0,1\}^\kappa$). The algorithms are

$$\begin{aligned} \mathsf{msg}_1(a) &= g^a \\ \mathsf{msg}_2(b, A) &= g^b \\ \mathsf{key}_1(a, B) &= B^a \text{ (resp. } H(B^a)) \\ \mathsf{key}_2(b, A) &= A^b \text{ (resp. } H(A^b)) \end{aligned}$$

[6] [SGJ23, Section 2.2] uses "KEM anonymity" to refer to a slightly weaker property, where two KEM ciphertexts on two randomly generated public keys are indistinguishable. ("KEM anonymity" in [BCP+23] requires the KEM ciphertext to be pseudorandom over the ciphertext space, i.e., it is equivalent to our KA second pseudorandomness.)

where $H : \mathbb{G} \to \{0,1\}^\kappa$ is a hash function (usually modeled as an RO). Note that the output of $\mathsf{msg}_2(b, A)$ does not depend on A, so this protocol can be executed in 1 simultaneous round; however, we present it as P' waits for message from P, in order to fit the general description of 2-round KA protocols.

It is well-known that plain Diffie-Hellman satisfies computational security under the DDH assumption, and its hashed variant satisfies computational security under the CDH assumption, the DDH assumption, or the Gap Diffie-Hellman (GDH) assumption, with a security loss of q (the number of the adversary's H queries) under CDH and no security loss under DDH or GDH. Both plain Diffie-Hellman and hashed Diffie-Hellman satisfy perfect correctness and perfect strong pseudorandomness.

Example: Kyber. *Kyber*[7] [SAB+22] is a post-quantum KEM currently under consideration for standardization by NIST. Without going into much detail, Kyber starts with a public-key encryption scheme based on module-LWE PKE, then applies the Fujisaki-Okamoto (FO) transform [FO99] to achieve KEM with CCA-security. The underlying public-key encryption has public keys and ciphertexts that can be thought of as vectors in $\mathbb{Z}_q^{256k}$, where $q = 3329$ and k is set based on the security level.[8] Correctness and CPA-security hold under the module-LWE assumption. Unfortunately, Kyber includes an optimization to compress the ciphertexts by rounding off some of the least significant bits, and this causes a small bias in the ciphertext distribution because q is not a power of two. Therefore, we must consider Kyber without this compression optimization. Without it, Kyber's public-key encryption scheme has pseudorandom public keys and ciphertexts [BCP+23].

We now describe the Kyber KEM in terms of the underlying Kyber PKE. This version is somewhat simplified compared to actual Kyber, as some of the hashes have been rearranged to make the presentation simpler by removing some of its optimizations. However, the modified hash calls are indifferentiable from the originals, so this should make no difference to the security arguments. Here are the algorithms, written as a KA instead of a KEM.

$$
\begin{aligned}
\mathcal{M}_1 &= \{0,1\}^{256} \times \mathbb{Z}_q^{256k} \\
\mathcal{M}_2 &= \mathbb{Z}_q^{256(k+1)} \\
\mathcal{K} &= \{0,1\}^{256} \\
\mathsf{msg}_1(a) &= \mathsf{Kyber.KeyGen}(a) \\
\mathsf{msg}_2(b, A) &= \mathsf{Kyber.Enc}(A, b; H(b, A)) \\
\mathsf{key}_2(b, A) &= H'(b, A, \mathsf{msg}_2(b, A)) \\
\mathsf{key}_1(a, B) &= \begin{cases} H'(\mathsf{Kyber.Dec}(a, B), A, B) & \text{if } B = \mathsf{msg}_2(\mathsf{Kyber.Dec}(a, B), A) \\ H'(a, B) & \text{otherwise} \end{cases}
\end{aligned}
$$

H' is a hash function onto $\{0,1\}^{256}$ (usually modeled as an RO), and $\mathsf{Kyber.Enc}(A, m; r)$ means to encrypt message m under public key A using randomness r.)

For an argument why Kyber satisfies various properties we need, see Sect. C.

[7] There are multiple versions of Kyber; our description follows the most recent draft submission (version 3). The draft standard [Nat23] (which renames Kyber to ML-KEM) does not hash A together with session key. We need this hash for collision resistance, so we would need to replace it by applying a hash to the KEM output.

[8] $k = 2$ for Kyber-512, $k = 3$ for Kyber-768, and $k = 4$ for Kyber-1024.

- On input $(\mathsf{NewSession}, \mathsf{sid}, P, P', \mathsf{pw}, \mathsf{role})$ from P, send $(\mathsf{NewSession}, \mathsf{sid}, P, P', \mathsf{role})$ to $\mathcal{S}$. Furthermore, if this is the first NewSession message for *sid*, or this is the second NewSession message for sid and there is a record $\langle P', P, \cdot \rangle$, then record $\langle P, P', \mathsf{pw} \rangle$ and mark it fresh.
- On $(\mathsf{TestPwd}, \mathsf{sid}, P, \mathsf{pw}^*)$ from $\mathcal{S}$, if there is a record $\langle P, P', \mathsf{pw} \rangle$ marked fresh, then do:
  - If $\mathsf{pw}^* = \mathsf{pw}$, then mark the record compromised and send "correct guess" to $\mathcal{S}$.
  - If $\mathsf{pw}^* \neq \mathsf{pw}$, then mark the record interrupted and send "wrong guess" to $\mathcal{S}$.
- On $(\mathsf{NewKey}, \mathsf{sid}, P, K^* \in \{0,1\}^\kappa)$ from $\mathcal{S}$, if there is a record $\langle P, P', \mathsf{pw} \rangle$, and this is the first NewKey message for *sid* and P, then output (sid, K) to P, where K is defined as follows:
  - If the record is compromised, then set $K := K^*$.
  - If the record is fresh, a key (sid, K') has been output to P', at which time there was a record $\langle P', P, \mathsf{pw} \rangle$ marked fresh, then set $K := K'$.
  - Otherwise sample $K \leftarrow \{0,1\}^\kappa$.

  Finally, mark the record completed.

Figure 3: UC PAKE functionality $\mathcal{F}_{\text{PAKE}}$

2.2 UC PAKE Functionality

We recall the standard UC PAKE functionality [CHK+05] in Figure 3. Below we briefly describe the idea behind the functionality; for a more detailed explanation, see [RX23, Section 2.2].

The functionality involves two parties, P with password pw and P' with password pw′. Each execution of the protocol incurs a session for P (i.e., a session initiated by P where P eventually outputs a key) and a session for P', both of which have a corresponding record marked with a state:

- When a session is established (by a NewSession command), it is marked fresh, and will remain fresh unless and until the (ideal) adversary attacks the session.
- The adversary may attack a fresh P session by sending a TestPwd command for P, on a *password guess* pw^*. This models an *online guessing attack* in the real world, where the adversary runs the algorithm of P' on a password guess pw^* and communicates with P. If $\mathsf{pw}^* = \mathsf{pw}$, this is a successful attack, and the session is marked compromised; otherwise the session is marked interrupted. (Symmetrically, the adversary may attack the P' session by sending a TestPwd command for P'.) Note that once a session becomes compromised or interrupted, it can never return to fresh; this in particular means that TestPwd can be run only once on any specific session.
- The adversary may end a session by sending a NewKey command, and the session will output a key depending on the states of this session and its counter session. After that, the session is marked completed (so that TestPwd cannot be sent after the session ends).

It is subtle and critical to our later sections how exactly a session key is determined, so let us explain the three cases under NewKey further:

- The "normal" case is that both the P session and the P' session are fresh, and $\mathsf{pw} = \mathsf{pw}'$. This models a correct protocol execution in the real world, where the adversary does not interfere and the two parties' passwords match. Say the P' session ends first; then P' outputs

a random session key K' (the third case under NewKey), and when the P session ends, P outputs session key $K = K'$ (the second case).

- If both the P session and the P' session are fresh, but $\mathsf{pw} \neq \mathsf{pw}'$, then this corresponds to an incorrect protocol execution where the adversary does not interfere but the two parties' passwords do not match. In this case P and P' output independent random keys (the third case).
- If the P session is compromised, it models the real-world scenario where the adversary has successfully performed an online guessing attack on P. In this case all security guarantees are lost, so we might as well let the adversary choose the session key for P (the first case). The same goes for the P' session (same below).
- If the P session is interrupted, it models the real-world scenario where the adversary has performed an unsuccessful online guessing attack on P. This means that the adversary should have no information about the session key of P; furthermore, the session key of P should also be independent of the session key of P'. So we let P output a random session key (the third case).
- Finally, if the P session is fresh but its counter session P' has been attacked (either compromised or interrupted), then again the adversary should have no information about the session key of P (because the P session is fresh), and the session key of P should also be independent of the session key of P' (because the P' session is attacked, so P' should output a session key that is either set by the adversary or independent of everything else). So we also let P output a random session key (the third case).

We stress that if a party's session key is random (the first three cases), *the adversary never gains any information about it.* For example, if the P session is fresh and the P' session is compromised, then the session key of P is still independent of the adversary's view (even though the adversary fully controls the session key of P'). The above holds even if the adversary learns the party's password after the session ends with a random session key; in this case the session is already marked completed, so the adversary cannot send TestPwd even if it gets to know the password later on.

3 Attacks on Previous Instantiations of (O)EKE

In this section, we show that a number of (O)EKE instantiations, including the ones claimed secure in [MRR20, Theorem 10], [SGJ23, Theorem 2], the proof of [SGJ23, Theorem 3], and [BCP+23, Theorem 1], are actually insecure. For each attack we also describe the security property necessary to prevent it. We assume that the encryption scheme used in (O)EKE is an IC; our attacks analogously apply to uses of POPF or HIC.

The attacks in Sects. 3.1 and 3.2 are presented with the underlying KA protocol being (some variants of) Diffie-Hellman. This is mainly for clarity of presentation, but we stress that (1) these attacks are sufficient to invalidate existing results on the security of (O)EKE, as (variants of) Diffie-Hellman satisfy all properties claimed to imply security yet the corresponding (O)EKE protocols are insecure, and (2) from these attacks we can see *in general* what type of additional properties the underlying KA protocol needs to satisfy. The attack in Sect. 3.3 assumes a general KA protocol.

3.1 Allowing Identity Element in Diffie-Hellman Makes OEKE Insecure

We begin with a simple attack on OEKE-PRF where the underlying KA protocol is plain Diffie-Hellman; that is, P sends $\mathcal{E}(\mathsf{pw}, g^a)$ to P', P' samples integer b and computes its KA key $K' = (g^a)^b = g^{ab}$ and PAKE session key $\mathsf{PRF}_{K'}(0)$, and finally sends g^b together with $\tau' = \mathsf{PRF}_{K'}(1)$ to P. P then computes $K = g^{ab}$ as $(g^b)^a$, and checks if $\tau' = \mathsf{PRF}_K(1)$ (if so, P outputs $\mathsf{PRF}_K(0)$ as its session key; otherwise P outputs a random session key).

Consider the following adversary: it disregards the P-to-P' message and sends $(e, \mathsf{PRF}_e(1))$ to P, where e is the identity group element. Then P computes $K = e^a = e$, so the check passes and P outputs session key $\mathsf{PRF}_K(0) = \mathsf{PRF}_e(0)$. That is, an adversary that does not know the password can predict the session key of P, which clearly breaks the security of PAKE. This attack can be easily generalized to any version of OEKE using plain Diffie-Hellman, and in particular OEKE-RO is also insecure; and it extends to OEKE using hashed Diffie-Hellman (the adversary sends $(e, \mathsf{PRF}_{H(e)}(1))$ to P). An obvious fix is to disallow e as a KA message, i.e., sample the exponent b from $\mathbb{Z}_p \setminus \{0\}$ rather than $\mathbb{Z}_p$. Note that the attack does not apply to EKE where the P'-to-P message g^b is encrypted under pw (so sending $\mathcal{E}(\mathsf{pw}, e)$ requires knowledge of pw).

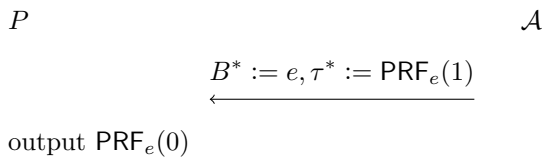

Figure 4: Attack on OEKE-PRF with plain Diffie-Hellman. $\mathcal{A}$ (that does not know pw) sends a single message to P and can predict the session key of P

The attack above shows that *any* KA protocol underlying OEKE must satisfy some form of *contributoryness* property, namely party 1 must "contribute" to the output key and party 2 cannot single-handedly bias the distribution of the key too much. We formalize the exact property necessary for the security of OEKE, which we call *collision resistance*. The proof of [SGJ23, Theorem 3] only requires the KA protocol to be secure and pseudorandom; in other words, it allows for using plain Diffie-Hellman — where e can be a message — in OEKE, so this proof is incorrect. (The theorem statement assumes a slightly different version of OEKE which prevents the attack in this section; see Thm. 3.3 below.)

Definition 3.1. *A KA protocol is* ***collision-resistant*** *if the key space is $\mathcal{K} = \{0,1\}^{3\kappa}$, and for any polynomially bounded q and any PPT adversary $\mathcal{A}$, the winning probability of $\mathcal{A}$ in the following game is negligible:*

$$\begin{array}{|l|}\hline \forall i \in [q]\colon a_i \leftarrow \mathcal{R} \\ \forall i \in [q]\colon A_i := \mathsf{msg}_1(a_i) \\ B^* \leftarrow \mathcal{A}(A_1, \ldots, A_q) \\ \forall i \in [q]\colon K_i \parallel \tau_i := \mathsf{key}_1(a_i, B^*) \\ \mathcal{A} \text{ wins if } \exists_{i \neq j}\, \tau_i = \tau_j \\ \hline \end{array}$$

Here, we split keys $\mathsf{key}_1(a_i, B^*) \in \{0,1\}^{3\kappa}$ into two chunks: $K \in \{0,1\}^\kappa$ and $\tau \in \{0,1\}^{2\kappa}$.[9]

[9]We require τ to be 2κ-bit long, as an adversary can win the experiment with probability roughly $q^2/2^{|\tau|}$. Technically this renders OEKE-PRF impossible, as K and τ now have different lengths. However, we can consider a version of OEKE-PRF where $K = \mathsf{PRF}_{\bar{K}}(0)$ and $\tau = \mathsf{PRF}_{\bar{K}}(1) \parallel \mathsf{PRF}_{\bar{K}}(2)$ (where $\bar{K}$ is the κ-bit key of the underlying KA protocol). Alternatively, we can stick to a κ-bit τ if we allow the adversary's advantage to be quadratic in κ.

Remark 3.2. *A version of collision-resistance for KEMs appears in [Xag22] as* SCFR-CCA *security. Our definition differs from theirs in three ways:*

1. *We do not give the adversary access to a decryption oracle;*
2. *We give the adversary polynomially many first messages instead of two;*
3. *We only require partial key collision to win the security game.*

Remark 3.3. *In OEKE-RO, if the authenticator τ is defined as $H(\mathsf{pw}', K', 1)$ rather than $H(K', 1)$, then collision resistance is unnecessary since the adversary needs to know the correct password in order to generate a valid authenticator, and the simulator can extract the password from the adversary's H queries. The OEKE-RO protocol analyzed in [BCP+23, Theorem 2] uses this approach, so it does not suffer from the problem in this section.*

Another way to circumvent the collision resistance requirement is to set τ to be $H(A, K', 1)$, so that only an adversary that knows the correct password can decrypt the P-to-P' message ϕ and obtain A. This is the approach taken by the statement of [SGJ23, Theorem 3]; however, its proof assumes a version where A is not hashed (see [SGJ23, Footnote 14]) and is thus incorrect.

3.2 EKE with Plain Diffie-Hellman Is Insecure

Our next attack considers the EKE instantiation where the underlying KA protocol is plain Diffie-Hellman; that is, P sends $\mathcal{E}(\mathsf{pw}, g^a)$ to P' and P' sends $\mathcal{E}(\mathsf{pw}, g^b)$ to P, and both parties decrypt each other's message and agree upon the session key g^{ab}. This protocol is insecure due to the following man-in-the-middle attack: an adversary can pass the P-to-P' message $\mathcal{E}(\mathsf{pw}, g^a)$ without modification, then guess pw and replace the P'-to-P message $\mathcal{E}(\mathsf{pw}, g^b)$ with $\mathcal{E}(\mathsf{pw}, g^{2b})$ (by decrypting the message and obtaining g^b, then encrypting $(g^b)^2$ under pw). Assuming the password guess is correct, P outputs $K = g^{2ab}$ and P' outputs $K' = g^{ab}$, so $K = (K')^2$. In other words, an adversary that does not attack the P' session but successfully attacks the P session, causes the session keys of P and P' to be correlated — which is not allowed in a UC-secure PAKE.

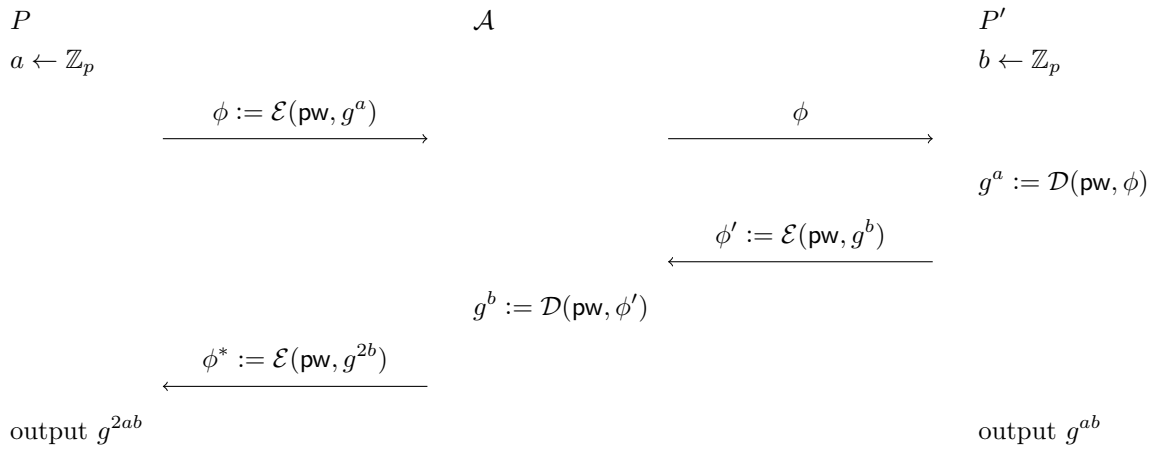

Figure 5: Attack on EKE with plain Diffie-Hellman. $\mathcal{A}$ only guesses pw in the second round

Indeed, the UC PAKE functionality $\mathcal{F}_{\text{PAKE}}$ guarantees that if the P' session is not attacked but its counter-session was (successfully or unsuccessfully) attacked, then the P' session is fresh and *the session key of P' is independent of everything else* (see the last bullet at the end of Sect. 2.2). This

in particular means that the protocol above violates UC-security for PAKE: when the adversary $\mathcal{A}$ passes $\mathcal{E}(\mathsf{pw}, g^a)$ to P', the UC simulator $\mathcal{S}$ does not know pw and has to let P' output a session key by sending a $(\mathsf{NewKey}, \mathsf{sid}, P', \star)$ message to $\mathcal{F}_{\mathrm{PAKE}}$, causing P' to output a random session key K' (independent of the view of $\mathcal{S}$). Later, when $\mathcal{A}$ sends $\mathcal{E}(\mathsf{pw}, g^{2b})$ to P, $\mathcal{S}$ can extract pw and compromise the P session by sending a correct $\mathsf{TestPwd}$ message, but K' is still independent of the view of $\mathcal{S}$, so $\mathcal{S}$ cannot make K' and the session key of P correlated.

It is not hard to turn the above observation into a formal proof of UC-insecurity:

Theorem 3.4. *EKE with the plain Diffie-Hellman KA does not UC-realize $\mathcal{F}_{\mathrm{PAKE}}$ in the $\mathcal{F}_{\mathrm{IC}}$-hybrid world.*

Proof. We assume the password dictionary Dict is a priori fixed and known to the simulator; this only makes the simulation potentially easier. There is no restriction on Dict except that $|\mathsf{Dict}| \geq 2$. Suppose there is a simulator $\mathcal{S}$ that generates an indistinguishable view for any PPT environment. Consider the environment $\mathcal{Z}$ in Figure 6 (for brevity, we omit sid in parties' messages and outputs below).

1. Sample $\mathsf{pw} \leftarrow \mathsf{Dict}$, and send $(\mathsf{NewSession}, \mathsf{sid}, P, P', \mathsf{pw})$ to P and $(\mathsf{NewSession}, \mathsf{sid}, P', P, \mathsf{pw})$ to P'. // let P and P' run the protocol on the same password pw
2. On ϕ from P, instruct the adversary to send ϕ to P'. Observe the output of P', K'. // pass the P-to-P' message without any modification
3. On ϕ' from P', query $B := \mathcal{D}(\mathsf{pw}, \phi')$ and then $\phi^* := \mathcal{E}(\mathsf{pw}, B^2)$, and instruct the adversary to send ϕ^* to P. Observe the output of P, K. // replace the P'-to-P message $\mathcal{E}(\mathsf{pw}, g^b)$ with $\mathcal{E}(\mathsf{pw}, g^{2b})$
4. Output 1 if $K = (K')^2$, and 0 otherwise. // guess "real world" iff $K = (K')^2$

Figure 6: Environment for EKE with plain Diffie-Hellman

In the real world, let a be the randomness of P, $A = g^a$, and b be the randomness of P'. Then $K' = A^b = g^{ab}$ and $K = (B^2)^a = g^{2ab}$, so $K = (K')^2$ and $\mathcal{Z}$ outputs 1 with probability 1.

In the ideal world, let $\mathsf{Compromise}$ be the event that the P' session is $\mathsf{compromised}$ before it outputs K'. This happens if and only if $\mathcal{S}$ sends $(\mathsf{TestPwd}, \mathsf{sid}, P', \mathsf{pw})$ (i.e., making a correct password guess for P') before P' outputs. The crucial observation is that *at the end of step 2,* the view of $\mathcal{S}$ only consists of $(\mathsf{NewSession}, \mathsf{sid}, P, P')$ and $(\mathsf{NewSession}, \mathsf{sid}, P', P)$[10], so pw *is independent of the view of* $\mathcal{S}$. Thus,

$$\Pr[\mathsf{Compromise}] \leq \frac{1}{|\mathsf{Dict}|}.$$

Now assume that $\mathsf{Compromise}$ does not happen; in other words, when $\mathcal{S}$ lets P' output K' via a NewKey command to $\mathcal{F}_{\mathrm{PAKE}}$, the P' session is fresh or $\mathsf{interrupted}$. Either way, $\mathcal{F}_{\mathrm{PAKE}}$ samples $K' \leftarrow \{0,1\}^\kappa$. In the rest of the experiment, the view of $\mathcal{S}$ consists of pw from IC queries, which is independent of K', so K' *is independent of the view of* $\mathcal{S}$ *throughout the experiment.* Next, consider the state of the P session before it outputs K:

- If it is fresh, and the P' session was also fresh before P' outputs K', then $\mathcal{F}_{\mathrm{PAKE}}$ enters the

[10] Formally it also consists of ϕ which is copied from the message simulated by $\mathcal{S}$ itself. Below we omit such messages for readability.

second case under NewKey, so $K = K'$. Thus, $K = (K')^2$ iff $K' = e$ (the identity group element), which happens with probability $1/p$;

- If it is interrupted, or it is fresh and the P' session was interrupted before P' outputs K', then $\mathcal{F}_{\text{PAKE}}$ enters the third case under NewKey, so $K \leftarrow \mathbb{G}$ (independent of K') and the probability that $K = (K')^2$ is $1/p$;
- If it is compromised (note that at this time $\mathcal{S}$ knows pw from IC queries, so it is able to compromise the P session), then $\mathcal{F}_{\text{PAKE}}$ enters the first case under NewKey, so K is set by $\mathcal{S}$. However, as we have just argued, K' is a random element of $\mathbb{G}$ in the view of $\mathcal{S}$, so the probability that $K = (K')^2$ is $1/p$.[11]

We conclude that as long as Compromise does not happen, the probability that $K = (K')^2$ — in other words, the probability that $\mathcal{Z}$ outputs 1 — is $1/p$. Overall, the probability that $\mathcal{Z}$ outputs 1 in the ideal world is at most

$$\Pr[\mathsf{Compromise}] + \frac{1}{p} \leq \frac{1}{|\mathsf{Dict}|} + \frac{1}{p},$$

so the distinguishing advantage of $\mathcal{Z}$ is at least

$$1 - \frac{1}{|\mathsf{Dict}|} - \frac{1}{p},$$

which is non-negligible since $|\mathsf{Dict}| \geq 2$. Thus, such a "successful" simulator $\mathcal{S}$ does not exist, which concludes the proof. □

Remark 3.5. *We note that the proof of Thm. 3.4 does not rely on the simulator $\mathcal{S}$ being PPT; in other words, the failure of the simulator is "statistical" and even a computationally unbounded simulator still cannot generate an indistinguishable view for our environment $\mathcal{Z}$.*

The above shows that [MRR20, Theorem 10] and [SGJ23, Theorem 2] are false, since both theorems only require security and pseudorandomness of the underlying KA protocol, which are satisfied by plain Diffie-Hellman.

Necessity of pseudorandom non-malleability. The issue above points to a property of the underlying KA protocol that is not commonly seen in the literature but is required for the security of EKE: Consider a "semi-man-in-the-middle" adversary that sees both protocol messages, but can only modify the second, i.e., the P'-to-P message. Then as long as the adversary modifies the P'-to-P message, the output key of P' is pseudorandom even if the adversary additionally sees the output of P. We call this property *non-malleability*. In fact, we require a stronger security property we call *pseudorandom non-malleability*, which says that the P'-to-P message and the key of P' are both pseudorandom (see Sect. A.3 for why we require pseudorandom non-malleability instead of just non-malleability).

Definition 3.6. *A KA protocol is **pseudorandom non-malleable** if the following two distributions*

[11] This is assuming the group order p is odd; if it is even then the probability is up to $2/p$, and the subsequent probability analysis needs to change accordingly.

are indistinguishable:

$a \leftarrow \mathcal{R}$	$a \leftarrow \mathcal{R}$
$b \leftarrow \mathcal{R}$	
$A := \mathsf{msg}_1(a)$	$A := \mathsf{msg}_1(a)$
$B := \mathsf{msg}_2(b, A)$	$B \leftarrow \mathcal{M}_2$
$K' := \mathsf{key}_2(b, A)$	$K' \leftarrow \mathcal{K}$
$B^* \leftarrow \mathcal{A}(A, B, K')$	$B^* \leftarrow \mathcal{A}(A, B, K')$
abort if $B^* = B$	abort if $B^* = B$
$K := \mathsf{key}_1(a, B^*)$	$K := \mathsf{key}_1(a, B^*)$
output K to $\mathcal{A}$	output K to $\mathcal{A}$

Remark 3.7. *Non-malleability alone is implied by the CCA-security of a KEM, but pseudorandom non-malleability corresponds roughly to the IND$-CCA security of a KEM. A version of pseudorandom non-malleability for KEMs appears in [Xag22] as* SPR-CCA$_{\mathcal{S}}$ *security. Our definition is the same as* SPR-CCA$_{\mathcal{S}}$ *when the simulator* $\mathcal{S}$ *samples a uniform random ciphertext, except that we only allow the adversary to make a single decryption oracle query.*

Remark 3.8. *Recall that (standard, not strong) second pseudorandomness says that* $B := \mathsf{msg}_2(b, A)$ *and* $B \leftarrow \mathcal{M}_2$ *are indistinguishable, even given* $A := \mathsf{msg}_1(a)$*. This is of course implied by pseudorandom non-malleability. We present second pseudorandomness as a separate property for clarity, and also because it is a standard property that was mentioned in several prior works (while pseudorandom non-malleability was not).*

History of the attack. We stress that the flaw in Sect. 3.2 was not discovered by us. First, [SGJ23] seems to have pointed out that [MRR20, Theorem 10] is false ("[W]e think that it is unlikely that EKE can provably realize UC PAKE based on the POPF properties alone"); however, it still misses the attack in this section, and [SGJ23, Theorem 2] is also false. Second, in the talk for the [SGJ23] paper [Jar23], the presenter Stanislaw Jarecki pointed out their own mistake and mentioned the attack on EKE with plain Diffie-Hellman, which we repeat here. According to [Jar23], the attack was found in a follow-up study (which was later published as [BGHJ24]). As such, the credit belongs to [Jar23, BGHJ24]. However, the ePrint version of [SGJ23] has not been updated accordingly after the talk, and to the best of our knowledge, we are the first to present this attack in written form. Furthermore, the formal proof of UC-insecurity (Thm. 3.4) is our work.

3.3 EKE Is Insecure If the Underlying KA Is Not Strongly Pseudorandom

The two attacks in previous sections assume the (O)EKE protocol uses (some variants of) the Diffie-Hellman KA. In this section we present a general attack showing that for EKE to be secure the KA protocol needs to be *strongly* pseudorandom. This is not an issue for Diffie-Hellman which has perfect pseudorandomness (i.e., the protocol messages are uniform), but in general strong pseudorandomness is not implied by security and pseudorandomness — as we will show next — and thus must be presented as a property of its own. Please see Thms. 2.2 to 2.5 for the definitions of these properties.

Indistinguishability between six distributions (but not the seventh). In a KA protocol, each of the first message A, the second message B, and the key K may be "real" or "random", resulting in 8 potential joint distributions of (A, B, K). (See Table 4 for definitions of "real" and "random".) The case where A, B are random but K is real (henceforth (random A, random B, real

K), or simply (random, random, real)) is not well-defined, since for K to be real, either a or b needs to be defined (which implies that either A or B needs to be real). Are the remaining 7 distributions indistinguishable from each other?

	real	random
A	$a \leftarrow \mathcal{R}$ $A := \mathsf{msg}_1(a)$	$A \leftarrow \mathcal{M}_1$
B	$b \leftarrow \mathcal{R}$ $B := \mathsf{msg}_2(b, A)$	$B \leftarrow \mathcal{M}_2$
K	$K := \mathsf{key}_1(a, B)$ or $K := \mathsf{key}_2(b, A)$	$K \leftarrow \mathcal{K}$

Table 4: Definitions of "real" and "random" for A, B, K

Recall that first pseudorandomness says that real A is indistinguishable from random A, and second pseudorandomness says that (real A, real B) is indistinguishable from (real A, random B). Note that the definition of real B does not use a (it only uses A) and thus can be simulated by a reduction to first pseudorandomness; this means that pseudorandomness (first and second combined) implies that the 4 joint distributions of A and B — where both A and B might be real or random — are indistinguishable from each other. Since random K can be simulated without any knowledge about A or B, pseudorandomness implies that the following 4 distributions are indistinguishable from each other:

- (real, real, random),
- (real, random, random),
- (random, real, random),
- (random, random, random).

Security says that (real, real, random) is indistinguishable from (real, real, real), so now we have 5 indistinguishable distributions under pseudorandomness plus security. Furthermore, (random, real, real) is indistinguishable from (real, real, real); this is because a reduction to first pseudorandomness can sample b on its own and simulate both real B and real K without knowing a. (This immediately implies Thm. 5.2, which says that (random, real, real) is indistinguishable from (random, random, random).) In summary, 6 out of the 7 distributions are indistinguishable from each other.

What about the last distribution, (real, random, real)? Suppose we attempt to construct a reduction that shows the indistinguishability from (real, real, real). The reduction, on (real A, real B) or (real A, random B), needs to simulate real K — which it cannot do because it knows *neither* a (which is used in real A) *nor* b (which is used in real B). What we need here is that (real A, real B) and (real A, random B) are indistinguishable *even given real* K, which is exactly *strong* second pseudorandomness.

In summary, security plus strong pseudorandomness imply the indistinguishability between all 7 distributions of (A, B, K) (which in particular implies Thm. 5.5 which says that (real, random, real) is indistinguishable from (real, random, random)); whereas security plus pseudorandomness only imply the indistinguishability between 6 distributions of (A, B, K), with (real, random, real) excluded.

Remark 3.9. *If the KA protocol is 1-simultaneous round, i.e., $B = \mathsf{msg}_2(b)$ does not depend on A, then strong pseudorandomness is implied by pseudorandomness: the reduction, on real B or random B, can sample a on its own and simulate both real A and real K without knowing b. [SGJ23, Theorem 2] only considers 1-simultaneous round KAs, so the distinction between pseudorandomness and strong pseudorandomness does not exist there; whereas [MRR20, Theorem 10] considers general 2-round KAs and overlooks this subtlety.*

Counterexample. We now present a concrete counterexample to show that security and pseudorandomness combined indeed do not imply the indistinguishability between (real, random, real) and the other 6 distributions. Consider a variant of hashed Diffie-Hellman, where

$$\begin{aligned}\mathsf{msg}_1(a) &= g^a\\ \mathsf{msg}_2(b, A) &= (g^b, H_0(A^b))\\ \mathsf{key}_2(b, A) &= H_1(A^b)\end{aligned}$$

(where $H_0, H_1 \colon \mathbb{G} \to \{0,1\}^\kappa$ are ROs). In this variant, P (that holds a) can use the H_0 hash to check whether it received a valid response to A; we use this to break strong pseudorandomness. Let

$$\mathsf{key}_1(a, (B_0, B_1)) = \begin{cases} H_1(B_0^a) & \text{if } B_1 = H_0(B_0^a)\\ H_2(g^a) & \text{otherwise}\end{cases}$$

Correctness is easily verified. Security and pseudorandomness still hold, as g^a and g^b are uniform elements of $\mathbb{G}$, and $H_0(A^b)$ and $H_1(A^b)$ are indistinguishable from random strings assuming CDH. However, (real, random, real) and (random, random, random) are easily distinguishable: in the former distribution, $K = H_2(A)$ with overwhelming probability because B_1 is uniform and so has negligible chance of satisfying $B_1 = H_0(B_0^a)$, while in the latter $K \neq H_2(A)$ with overwhelming probability, as they are independently random κ-bit strings.

Necessity of strong pseudorandomness in EKE. Consider the following simple attack on EKE (using IC): the adversary disregards P', and on ϕ from P sends random ϕ^* to P. After P outputs K, the adversary queries $A := \mathcal{D}(\mathsf{pw}, \phi)$ and $B := \mathcal{D}(\mathsf{pw}, \phi^*)$ (where pw is the password of P).

In the real world, A is the KA message generated by the honest P, so A is real; B is the IC decryption of random ϕ^*, so B is random; and K is computed by the honest P on a and B, so K is real. In other words, the environment's view is (real, random, real). Now consider the ideal world: before P outputs K the simulator only sees a random ϕ^* from the adversary and has no knowledge about pw, so it cannot send a correct $\mathsf{TestPwd}$ command and thus $\mathcal{F}_{\mathrm{PAKE}}$ will set the session key of P to be random. (The simulator sees pw *after the session of P completes*, at which time it cannot do anything to change the session key of P.) That is, in the ideal world the environment's view is ($\star$, $\star$, random), where $\star$ could be real or random, depending on the simulator's strategy. However, we have just seen that without strong pseudorandomness (real, random, real) is distinguishable from *all other 6 distributions*, so no matter what the two $\star$ are, the ideal world and the real world are distinguishable (the environment simply runs the distinguisher between (real, random, real) and the appropriate ($\star$, $\star$, random) for KA).

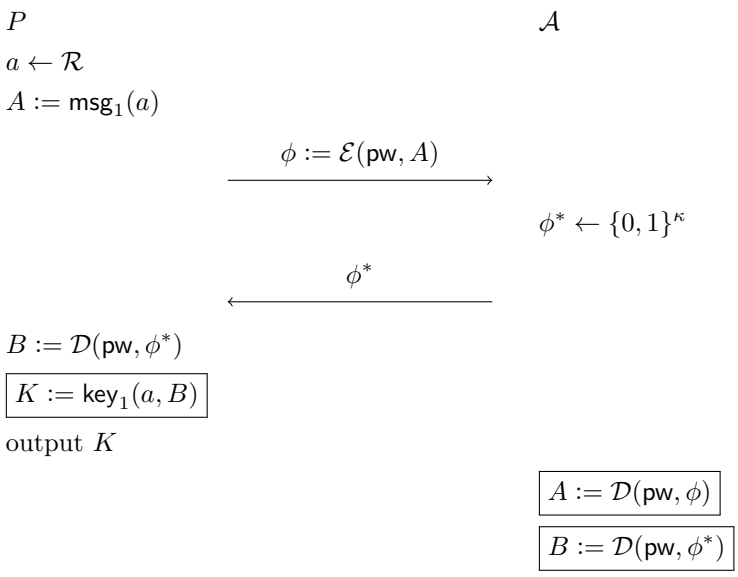

Figure 7: Attack on EKE with a KA protocol that does not satisfy strong pseudorandomness. In the real world $\mathcal{Z}$ sees (real A, random B, real K) (boxed texts). In the ideal world the simulator knows pw only after the session of P completes, so K is random. Without strong pseudorandomness, (real A, random B, real K) is distinguishable from ($\star$, $\star$, random K)

The above shows that [MRR20, Theorem 10] is false in a way different from Sect. 3.2, since it claims the security of EKE without requiring strong pseudorandomness for KA. Furthermore, the issue persists even if K is hashed, since security plus pseudorandomness do not even imply the unpredictability of real K given real A and random B (as shown in the counterexample above). This suggests that [BCP+23, Theorem 1] is also false, since it does not require strong pseudorandomness. Also note that the attack above does not apply to OEKE, where the adversary needs to come up with a valid authenticator in order for P to output a real session key.

3.4 Summary

We summarize the requirements on the underlying KA protocol for (O)EKE in Table 5. (For the attack on EKE using HIC/POPF, see Sect. A.4.)

	security and pseudorandomness	strong pseudorandomness	pseudorandom non-malleability	collision-resistance
EKE-PRF (or EKE if IC is used)	✓	✓ (overlooked in [MRR20, BCP+23])	✓ (overlooked in [MRR20, SGJ23] and security analyses in [DHP+18, BCP+23])	
OEKE	✓		✓ (overlooked in [SGJ23, BCP+23])	✓ (overlooked in security analysis of [SGJ23])
EKE using HIC/POPF does not realize $\mathcal{F}_{\text{PAKE}}$ no matter what KA protocol is used, which is overlooked in [MRR20, SGJ23]				

Table 5: Requirements on the underlying KA protocol in (O)EKE

How "real" are these attacks? It is fair to ask whether the attacks in this section correspond to "real-world" attacks, or if they merely break the UC PAKE security notion.

- The attack in Sect. 3.1 allows the adversary to predict a party's session key without knowing its password. Obviously, this completely breaks the security of PAKE.
- The attack in Sect. 3.3 breaks the *forward secrecy* of PAKE: the adversary sends a random message during the protocol session, but if it learns the password at any point after the session completes, the adversary can distinguish the party's session key from random (or even predict it). Forward secrecy is a standard requirement of modern key exchange, so this also constitutes a practical attack.
- The attack in Sect. 3.2 can be viewed as an attack on a generalized "per session" version of forward secrecy. Here, the adversary only learns the password during the second (P'-to-P) session, after the first (P-to-P') session has already completed and P' has output its session key. In this case the session key of P' should be secure (because the adversary did not learn the password until after the session ended), yet the adversary can cause the two parties' session keys to be correlated.

As a general point, attacks that break UC-security but don't appear to correspond to a "real-world" attack on the protocol directly can often be used to create a "real-world" attack on a higher-level *composed* protocol. As one example, in [ABB+20] the authors define a weakening of the UC-PAKE functionality called *lazy-extraction PAKE* (lePAKE) and show that several PAKE protocols that do not realize the full UC-PAKE functionality still realize this weaker functionality. Although it may not be clear initially how the definition of lePAKE leads to a "real-world" attack on the protocol itself, as observed in [Sho20] it is immediate that composing an lePAKE with a secure channels protocol is insecure.

On "folklore" results. We remark that the UC-security of EKE had long been a "folklore" result in the community, before a formal proof was presented. However, it seems unclear which exact version of EKE was understood to be UC-secure, and as [MRR20, Theorem 10] and [SGJ23, Theorem 2] suggest, some might have held the false belief that EKE with plain Diffie-Hellman is UC-secure (while others seem to have the correct understanding that the Diffie-Hellman output has to be hashed). This reveals the problem with such "folklore" results: without a formal analysis, people

cannot even agree upon what the result exactly is!

Our observation echoes Oded Goldreich's comment in 2020 [Ode20]:

> *In contrast to its sociological meaning, in [theory of computation] this term (i.e., folklore) typically means some vaguely specified fact that is known to some experts. I wish to highlight two key ingredients regarding this notion: First, that the known fact is not clearly defined (i.e., its definition is lacking when compared to the standard norms of the discipline). Second, that this knowledge is "shared" by few people, who typically publicize their claim of knowledge only after others who were excluded from the folklore actually discover it, distill it, study it, and publish it. [...] [O]ne should hope for the elimination of all folklore: Any fact of value should be specified, distilled, worked-out, and published.*

4 Almost Universally Composable POPF

As mentioned in Sect. 1.2, the POPF definition in [MRR20] is unusual in requiring a higher-order security definition, saying that any weak PRF must remain secure when using the POPF as input. We now generalize this idea significantly to UC protocols, by defining a notion of *almost UC* realization, which means that a protocol must be composable with some class of protocols built on top.

Definition 4.1. *Let $\mathcal{F}$ be an ideal functionality and $\mathcal{P}$ be a set of tuples of functionalities, protocols, and simulators. A protocol π $\mathcal{P}$-*almost UC realizes *an ideal functionality $\mathcal{F}$ if, for every protocol ρ that UC realizes an ideal functionality $\mathcal{F}'$ in the $\mathcal{F}$-hybrid model using a simulator*[12] *$\mathcal{S}$ such that $(\mathcal{F}', \rho, \mathcal{S}) \in \mathcal{P}$, the composed protocol $\rho^{\mathcal{F}\to\pi}$ (i.e., ρ with $\mathcal{F}$ instantiated by π) UC realizes $\mathcal{F}'$.*

If $\mathcal{P}$ contains every possible $(\mathcal{F}', \rho, \mathcal{S})$ then Thm. 4.1 describes the standard notion of UC-realization, by the UC composition theorem.

4.1 The Functionality $\mathcal{F}_{\text{POPF}}$

A POPF can be thought of as a family of random functions $\{F_\phi\}$, with two interfaces: Program, where a party picks a function F_ϕ with $F_\phi(x^*) := y^*$ for (x^*, y^*) of its choice, and Eval, where a party evaluates a function F_ϕ on a specific input x. Crucially, every honest function F_ϕ can be programmed at only one point (x^*, y^*), and for any $x \neq x^*$, F_ϕ is random. Furthermore, POPFs must satisfy the *uncontrollable output* property: for an adversarially generated ϕ^*, the output of $F_{\phi^*}(x)$ is pseudorandom — in particular, it is a suitable input of some higher-level schemes which use random inputs (e.g., a weak PRF, as in the definition in [MRR20]) — except on a single x "extracted" during the evaluation of $F_{\phi^*}(x)$.

Our POPF ideal functionality, $\mathcal{F}_{\text{POPF}}$, is shown in Figure 8. We now define POPFs using it and our almost UC definition above.

Definition 4.2. *A protocol π is a POPF if it $\mathcal{P}$-almost UC realizes $\mathcal{F}_{\text{POPF}}$. Here, $\mathcal{P}$ is the set of all $(\mathcal{F}', \rho, \mathcal{S})$ such that $\mathcal{S}$ emulates* TestOutput *in the same way as $\mathcal{F}_{\text{POPF}}$. That is, $\mathcal{S}$ must (lazily) sample a random function $R \leftarrow \mathcal{Y}^{\{0,1\}^\kappa \times \mathcal{X}}$, use it to emulate $(\mathsf{TestOutput}, \mathsf{sid}, \alpha, x)$ queries by returning $R(\alpha, x)$, and only make read-only oracle queries to R (without programming R in any way).*

[12] Technically, UC realization requires one simulator for every adversary. We only care about the simulator for the dummy adversary, which simply obeys whatever commands the environment gives to it.

Global variables (per sid):

- Transcript $T = \{\}$ of POPF evaluations.
- Set $\Phi = \{\}$ of honest POPF indices.
- Malicious POPF index $\phi^* = \bot$.
- Random function $R \leftarrow \mathcal{Y}^{\{0,1\}^* \times \mathcal{X}}$ of uncontrollable outputs (defined via lazy sampling).
- String $\alpha^* = \bot$.

On $(\mathsf{Program}, \mathsf{sid}, x^*, y^*)$ from party P (where $x \in \mathcal{X}$ and $y \in \mathcal{Y}$):

1. There are two cases, depending on whether there is an entry $(\cdot, x^*, y^*) \in T$.
   - A. If there is no such entry, or if P is malicious, send $(\mathsf{Program}, \mathsf{sid})$ to $\mathcal{A}^*$. Wait until $\mathcal{A}^*$ responds with $(\mathsf{Program}, \mathsf{sid}, \phi)$ such that there is no entry $(\phi, \cdot, \cdot) \in T$. Then add ϕ to Φ.
   - B. Otherwise, there is such an entry, and P is honest. Send $(\mathsf{Program}, \mathsf{sid}, \{\phi \mid (\phi, x^*, y^*) \in T\})$ to $\mathcal{A}^*$. Wait until $\mathcal{A}^*$ responds with $(\mathsf{Program}, \mathsf{sid}, \phi)$ such that $(\phi, \cdot, \cdot) \notin T$ or $(\phi, x^*, y^*) \in T$.
2. If $(\phi, x^*, y^*) \notin T$, add (ϕ, x^*, y^*) to T.
3. Send $(\mathsf{Program}, \mathsf{sid}, \phi)$ to P.

On $(\mathsf{Eval}, \mathsf{sid}, \phi, x)$ from party P (where $x \in \mathcal{X}$):

4. If $\phi \notin \Phi$, $\phi^* = \bot$, and P is honest, then:
   - (1) Send $(\mathsf{Extract}, \mathsf{sid}, \phi)$ to $\mathcal{A}^*$ and wait for response $(\mathsf{Extract}, \mathsf{sid}, x^*, \alpha)$.
   - (2) Send $(\mathsf{Eval}, \mathsf{sid}, \phi, x^*)$ to $\mathcal{A}^*$ and wait for response $(\mathsf{Eval}, \mathsf{sid}, y^*)$.
   - (3) Set $\phi^* := \phi$, $\alpha^* := \alpha$, and add (ϕ, x^*, y^*) to T.
5. Check if there is an entry $(\phi, x, y) \in T$. If not, generate y according to three cases:
   - A. If $\phi \in \Phi$, sample $y \leftarrow \mathcal{Y}$.
   - B. If $\phi = \phi^*$, let $y := R(\alpha^*, x)$.
   - C. Otherwise, send $(\mathsf{Eval}, \mathsf{sid}, \phi, x)$ to $\mathcal{A}^*$ and wait for response $(\mathsf{Eval}, \mathsf{sid}, y)$.

   Finally, add (ϕ, x, y) to T.
6. Send $(\mathsf{Eval}, \mathsf{sid}, y)$ to P.

On $(\mathsf{TestOutput}, \mathsf{sid}, \alpha, x)$ from $\mathcal{A}^*$ (where $\alpha \in \{0,1\}^*$ and $x \in \mathcal{X}$):

7. Send $(\mathsf{TestOutput}, \mathsf{sid}, R(\alpha, x))$ to $\mathcal{A}^*$.

Figure 8: Ideal functionality $\mathcal{F}_{\text{POPF}}$ (with domain $\mathcal{X}$ and range $\mathcal{Y}$).

We require Thm. 4.2 to ensure the simulator for the higher-level protocol does not program R, since the simulator for the POPF must program R in order for R to match evaluations of the POPF.

$\mathcal{F}_{\text{POPF}}$ maintains a set T of defined function values, where $(\phi, x, y) \in T$ means that $F_\phi(x) = y$. When party P (either honest or malicious) wants to program on a pair (x^*, y^*), $\mathcal{F}_{\text{POPF}}$ lets the adversary specify a function index ϕ. In particular, if there are no functions F_ϕ in the family such that $F_\phi(x^*)$ is set to be y^*, the adversary must choose a *new* index ϕ; this ensures that F_ϕ is never programmed on two distinct points. $\mathcal{F}_{\text{POPF}}$ also adds ϕ to the set of honest POPF indices Φ, which indicates that F_ϕ is "programmable-once" and has been programmed on one input/output pair. On the other hand, if there are such functions F_ϕ such that $F_\phi(x^*) = y^*$ (and P is honest), $\mathcal{F}_{\text{POPF}}$ lets the adversary know all such indices ϕ.[13] Then the adversary has two options: either pick one of these existing indices (i.e., the programming process is identical to a previous one), or choose a *new* index ϕ, just as in the previous case.

When a party P wants to evaluate $F_\phi(x)$, whose result has not been defined through T, $\mathcal{F}_{\text{POPF}}$ has several cases based on how ϕ was generated.

- If ϕ is an honest index, this means that F_ϕ is "programmable-once" and has already been programmed on another input/output pair, so it chooses $F_\phi(x)$ at random.
- If ϕ is the "designated malicious index" ϕ^*, we want to use $F_\phi(x)$ as the input of some higher-level scheme. The adversary should be able to learn $F_\phi(x)$, but not control it when x differs from its chosen target x^*. Therefore, we set $F_\phi(x)$ to be the output of a random function R, queried on α and x. (α allows for the partial control of $F_\phi(x)$ given by rejection sampling on ϕ.)
- Otherwise, i.e., if ϕ is a malicious index other than ϕ^*, $\mathcal{F}_{\text{POPF}}$ allows the adversary to program F_ϕ on all inputs; in particular, $\mathcal{F}_{\text{POPF}}$ asks the adversary for $F_\phi(x)$.

The "designated malicious index" ϕ^* is chosen as the first malicious POPF index ϕ^* given to Eval by an honest party. Having only a single designated target is a necessary limitation of our POPF construction — requiring that multiple POPFs have jointly uncontrollable outputs is much more stringent than just requiring a single POPF to be uncontrollable. In all existing protocols that use POPFs, they only need to be individually uncontrollable and not jointly uncontrollable, so this limitation is not too onerous.

4.2 POPF Construction

Our POPF construction is identical to the 2-round Feistel POPF in [MRR20]. It uses an abelian group $\mathbb{G}$[14] and two ROs $H : \{0,1\}^* \times \{0,1\}^{3\kappa} \to \mathbb{G}$ and $H' : \{0,1\}^* \times \mathbb{G} \to \{0,1\}^{3\kappa}$. The function family is indexed by $\phi = (s, t) \in \{0,1\}^{3\kappa} \times \mathbb{G}$ and defined as

$$F_\phi(x) = H(x, s \oplus H'(x, t)) \cdot t.$$

To program on an input/output pair (x^*, y^*), one can compute $\phi = (s, t)$ via the following

[13] A malicious party could exploit this to find whether a given point (x^*, y^*) has been evaluated, so we only allow honest parties to trigger this case.

[14] In this section the group $\mathbb{G}$ does not need to be cyclic, which is different from the Diffie-Hellman group in Sects. 2 and 3. In particular, the EKE and OEKE protocols in Sect. 5 can work in a non-cyclic Abelian group when instantiated with the POPF in this section. To make this distinction clear, in this section we simply use $|\mathbb{G}|$ (rather than p) for the group order.

process: sample $r \leftarrow \{0,1\}^{3\kappa}$, and solve the equations

$$\begin{cases} H(x^*, r) \cdot t = y^*, \\ s \oplus H'(x^*, t) = r \end{cases}$$

for first t, then s. Note that for a pair of (x^*, y^*), there are exponentially many ϕ such that $F_\phi(x^*) = y^*$. This is a crucial difference from the ideal cipher, where the encryption is deterministic.

The formal description of our construction is shown in Figure 9. Both of the two ROs are implemented in the same ideal functionality $\mathcal{F}_{\text{RO}}$.

Parameters:

- Random oracle functionality $\mathcal{F}_{\text{RO}}$ (see Figure 10).

On $(\mathsf{Program}, \mathsf{sid}, x^*, y^*)$ (where $x^* \in \mathcal{X}$ and $y^* \in \mathbb{G}$):

1. Choose $r \leftarrow \{0,1\}^{3\kappa}$.
2. Query $h := H(x^*, r)$, and compute $t := y^*/h$.
3. Query $h' := H'(x^*, t)$, and compute $s := r \oplus h'$.
4. Output $(\mathsf{Program}, \mathsf{sid}, (s, t))$.

On $(\mathsf{Eval}, \mathsf{sid}, (s, t), x)$ (where $x \in \mathcal{X}$, $s \in \{0,1\}^{3\kappa}$, and $t \in \mathbb{G}$):

5. Query $h' := H'(x, t)$, and compute $r := s \oplus h'$.
6. Query $h := H(x, r)$, and compute $y := h \cdot t$.
7. Output $(\mathsf{Eval}, \mathsf{sid}, y)$.

Figure 9: POPF construction π_{POPF} (with domain $\mathcal{X}$ and range $\mathbb{G}$). The ROs H, H' are queried using $\mathcal{F}_{\text{RO}}$

4.3 Security Analysis

Theorem 4.3. *The protocol π_{POPF} is a POPF. That is, for any protocol ρ that UC realizes an ideal functionality $\mathcal{F}'$ in the $\mathcal{F}_{\text{POPF}}$-hybrid model, where the simulator emulates* TestOutput *and R in the same way as $\mathcal{F}_{\text{POPF}}$, the composed protocol $\rho^{\mathcal{F} \to \pi}$ UC realizes $\mathcal{F}'$.*

4.3.1 The Pseudo-simulator

As the first step of our security analysis, we describe in Figure 11 what we call the "pseudo-simulator" for π_{POPF}, $\mathcal{S}_{\text{pseudo}}$. $\mathcal{S}_{\text{pseudo}}$ can be viewed as an attempt to prove that π_{POPF} realizes $\mathcal{F}_{\text{POPF}}$; however, to successfully program a maliciously generated POPF's output to match the random function R, $\mathcal{S}_{\text{pseudo}}$ has to make a random guess over all of the adversary's H' queries, and its simulation is successful only if the guess is correct. As such, $\mathcal{S}_{\text{pseudo}}$ is not a valid simulator showing that π_{POPF} realizes $\mathcal{F}_{\text{POPF}}$, but it is a critical step towards building the actual simulator presented next.

Below we explain how $\mathcal{S}_{\text{pseudo}}$ works. It can be divided into two separate goals: simulating honestly generated POPFs without knowing where they were programmed, and forcing maliciously generated POPFs to have output matching the random function R.

Honest POPFs. When $\mathcal{S}_{\text{pseudo}}$ is asked to program on an input/output pair (which $\mathcal{S}_{\text{pseudo}}$ does not know), it simply chooses a random $\phi = (s, t)$. It then needs to answer the adversary's RO queries appropriately, so that evaluating the POPF via the RO gives the same result as calling Eval on the ideal functionality. On an $H(x, r)$ query, $\mathcal{S}_{\text{pseudo}}$ checks if $r = s \oplus H'(x, t)$ for some *honest*

Parameters:

- Abelian group $\mathbb{G}$ with order $2^{2\kappa} \leq |\mathbb{G}| < 2^{2\kappa+1}$.

Global variables:

- Initialize a *list* $T_{\text{RO}} := []$ of RO evaluations.

On $H(x, r)$ (for session sid) from party P:

1. Ignore this query if $r \notin \{0,1\}^{3\kappa}$.
2. If there is not a matching query "$h = H(x, r)$" $\in T_{\text{RO}}$, sample $h \leftarrow \mathbb{G}$ and append "$h = H(x, r)$" to T_{RO}.
3. Return h to P.

On $H'(x, t)$ (for session sid) from party P:

1. Ignore this query if $t \notin \mathbb{G}$.
2. If there is not a matching query "$h' = H'(x, t)$" $\in T_{\text{RO}}$, sample $h' \leftarrow \{0,1\}^{3\kappa}$ and append "$h' = H'(x, t)$" to T_{RO}.
3. Return h' to P.

Figure 10: Ideal functionality $\mathcal{F}_{\text{RO}}$.

$\phi = (s, t)$, i.e., the adversary is trying to compute $y = F_\phi(x) = H(x, r) \cdot t$. If so, then $\mathcal{S}_{\text{pseudo}}$ sends an Eval command to $\mathcal{F}_{\text{POPF}}$ to obtain y, and then programs $H(x, r) := y/t$.

Malicious POPFs. On the other hand, the $H(x, r)$ query might be the adversary computing $F_{\phi^*}(x)$ for the "designated malicious index" $\phi^* = (s^*, t^*)$. Note that ϕ^* might not have been chosen yet — $\mathcal{S}_{\text{pseudo}}$ only learns ϕ^* when Extract is called. It only knows that if this H query is evaluating $F_{\phi^*}(x)$ then $H'(x^*, t^*)$ must have been queried to compute s^*, and $H'(x, t^*)$ must have been queried to find r. Therefore, $\mathcal{S}_{\text{pseudo}}$ *has to choose a guess* t_g *for what* t^* *will be, among all of the adversary's* H' *queries.* Then $\mathcal{S}_{\text{pseudo}}$ guesses s^* by solving for $s_g = H'(x, t_g) \oplus r$, obtains y by explicitly querying the random function R[15], and programs $H(x, r) := y/t_g$. In all other cases, $\mathcal{S}_{\text{pseudo}}$ answers the RO queries by lazy sampling.

Later, Extract will be called, and $\mathcal{S}_{\text{pseudo}}$ will have to find the unique point x^* programmed by the POPF. Recall that to find (s^*, t^*), the adversary needs to compute first $t^* = y^*/H(x^*, r)$, then $s^* = r \oplus H'(x^*, t^*)$; the query $H'(x^*, t^*)$ is called the *anchor query.* That is, $\mathcal{S}_{\text{pseudo}}$ identifies the adversary's anchor query as the query (x^*, t^*) to H' (resulting in h^*) such that $H(x^*, s^* \oplus h^*)$ was queried before the H' query. However, note that there is an exception: given any $\phi = (s, t)$, the adversary can choose another index $\phi' = (s_\phi, t_\phi)$ and input x^* such that

$$s_\phi \oplus H'(x^*, t_\phi) = s \oplus H'(x^*, t),$$

causing

$$F_{\phi'}(x^*) = F_\phi(x^*) \cdot (t_\phi/t)$$

without making any H query. In this case, the anchor query is defined as the query $H'(x^*, t)$. Either way, $\mathcal{S}_{\text{pseudo}}$ outputs x^* in the anchor query, which implicitly sets $F_\phi(x^*)$ because the ideal functionality sends an Eval command to the simulator when $F_\phi(x^*)$ is evaluated. Finally, to answer an Eval command, $\mathcal{S}_{\text{pseudo}}$ simply returns the honestly computed function value (note that this might trigger a fresh H query, and how to answer it is described at the beginning of this paragraph).

[15] If $\mathcal{S}_{\text{pseudo}}$ were to send an Eval command to $\mathcal{F}_{\text{POPF}}$ here, $\mathcal{F}_{\text{POPF}}$ might enter step 5C in Fig. 8 (note that it would not enter step 4 since it is $\mathcal{S}_{\text{pseudo}}$ that sends the command, and would not enter step 5A since ϕ^* is a malicious index), in which case $\mathcal{F}_{\text{POPF}}$ would send an Eval command back to $\mathcal{S}_{\text{pseudo}}$ and cause a loop.

The guessing step while answering H' queries is why the pseudo-simulator is not (quite) a simulator, as the probability of a correct guess is $1/(q'_h + 1)$ (including the additional guess that Extract will never be called or that $H'(x, t^*)$ wasn't queried.). Since we cannot prove that π_{POPF} realizes $\mathcal{F}_{\text{POPF}}$, we turn to the "almost UC" notion of Thm. 4.1.

4.3.2 The Simulator

Let $\mathcal{F}'$ be an ideal functionality and $\mathcal{S}$ be the simulator for the overlying protocol ρ UC realizing $\mathcal{F}'$. In Figure 12, the POPF simulator $\mathcal{S}_{\text{POPF}}$ is constructed from $\mathcal{S}$ and the pseudo-simualtor.

4.3.3 Security Proof

Proof overview. To prove that π_{POPF} is a POPF (Thm. 4.2), we must show that its composition with ρ is simulated by $\mathcal{S}_{\text{POPF}}$. If π_{POPF} realized $\mathcal{F}_{\text{POPF}}$, we could simply apply the UC theorem. Unfortunately, it does not, because of the aforementioned aborts in the pseudo-simulator $\mathcal{S}_{\text{pseudo}}$.

Our proof has a similar structure to the proof of the UC theorem, but adjusted to take into consideration the aborts inside $\mathcal{S}_{\text{pseudo}}$. Recall that the UC theorem states that, if an "inner protocol" π_{inner} realizes functionality $\mathcal{F}_{\text{inner}}$ (let $\mathcal{S}_{\text{inner}}$ be the simulator), and an "outer protocol" ρ realizes functionality $\mathcal{F}'$ in the $\mathcal{F}_{\text{inner}}$-hybrid world (let $\mathcal{S}$ be the simulator for ρ), then the combined protocol $\rho^{\mathcal{F}_{\text{inner}} \to \pi_{\text{inner}}}$ also realizes $\mathcal{F}'$. At a high level, the hybrid proof goes through the following steps:

1. Real world: $\rho^{\mathcal{F}_{\text{inner}} \to \pi_{\text{inner}}} \Leftrightarrow \mathcal{A}$. The composed protocol is interacting with an adversary $\mathcal{A}$.
2. Intermediate world: $\rho \Leftrightarrow \mathcal{S}_{\text{inner}} \Leftrightarrow \mathcal{A}$. Now ρ uses $\mathcal{F}_{\text{inner}}$ directly, and the adversary $\mathcal{A}$ has been wrapped by interacting with $\mathcal{S}_{\text{inner}}$. This is indistinguishable from the real world, since π_{inner} UC realizes $\mathcal{F}_{\text{inner}}$.
3. Ideal world: $\mathcal{F}' \Leftrightarrow \mathcal{S} \Leftrightarrow \mathcal{S}_{\text{inner}} \Leftrightarrow \mathcal{A}$. Now there are no protocols, and the adversary is doubly wrapped by two simulators (or equivalently, the two simulators have been combined together). This is indistinguishable from the intermediate world, since ρ realizes $\mathcal{F}'$.

In our setting, the "inner protocol" is π_{POPF}, whose simulator $\mathcal{S}_{\text{pseudo}}$ has high probability of abort. Thus, we add aborts to our real and ideal worlds, and show that the "real world with abort" and the "ideal world with abort" are indistinguishable. Below we use the notation $p\mathcal{W}$ to represent running process $\mathcal{W}$ with probability p, and aborting instead with probability $1 - p$.

1. Real world with abort: $\frac{1}{q'_h+1}(\rho^{\mathcal{F}_{\text{POPF}} \to \pi_{\text{POPF}}} \Leftrightarrow \mathcal{A})$.
2. Pseudo-intermediate world: $\rho \Leftrightarrow \mathcal{S}_{\text{pseudo}} \Leftrightarrow \mathcal{A}$. Assuming that $\mathcal{S}_{\text{pseudo}}$ does not abort, π_{POPF} is indistinguishable from $\mathcal{F}_{\text{POPF}} \Leftrightarrow \mathcal{S}_{\text{pseudo}}$, so this is indistinguishable from the real world. The probability $\frac{1}{q'_h+1}$ was chosen to make the abort occur with equal probability between the two worlds, so that they are indistinguishable.
3. Pseudo-ideal world: $\mathcal{F}' \Leftrightarrow \mathcal{S} \Leftrightarrow \mathcal{S}_{\text{pseudo}}$. This is indistinguishable from the pseudo-intermediate world, since ρ realizes $\mathcal{F}'$.
4. Ideal world with abort: $\frac{1}{q'_h+1}(\mathcal{F}' \Leftrightarrow \mathcal{S}_{\text{POPF}})$. This step has no analogy with the UC theorem. The idea is to combine $\mathcal{S}_{\text{pseudo}}$ with $\mathcal{S}$, then make some tweaks so that the abort will no longer be necessary. While we cannot prove that π_{POPF} realizes $\mathcal{F}_{\text{POPF}}$ due to the abort in $\mathcal{S}_{\text{pseudo}}$, we can remove the abort *in the context of combined protocols*, because $\mathcal{S}_{\text{POPF}}$ can now modify $\mathcal{S}$'s definition of the random function R (as we specifically required that $\mathcal{S}$ not program R).

Initialize a *list* $T_{\mathrm{RO}} := []$ of RO evaluations and a set $\Phi := \{\}$ of honest POPF indices. Sample $g \leftarrow [0, q'_h]$ and $\mathrm{A} \leftarrow (\{0,1\}^{3\kappa})^{\{0,1\}^{3\kappa} \times \mathbb{G}}$. Set $t_g := \perp$.

On $(\mathsf{Program}, \mathsf{sid})$ or $(\mathsf{Program}, \mathsf{sid}, \Sigma)$ from $\mathcal{F}_{\mathrm{POPF}}$:

1. Choose $s \leftarrow \{0,1\}^{3\kappa}$ and $t \leftarrow \mathbb{G}$.
2. Add (s,t) to Φ and send $(\mathsf{Program}, \mathsf{sid}, (s,t))$ to $\mathcal{F}_{\mathrm{POPF}}$.

On $(\mathsf{Extract}, \mathsf{sid}, \phi^* = (s^*, t^*))$ from $\mathcal{F}_{\mathrm{POPF}}$:

3. Abort if the guess t_g is wrong. There are two cases:
   - A. If $H'(x, t^*)$ has been queried for some x, abort if $t_g \neq t^*$.
   - B. Otherwise, abort if $g \neq 0$.
4. Search for the anchor query "$h' = H'(x^*, t^*)$" $\in T_{\mathrm{RO}}$, which is the query satisfying one of these conditions:
   - A. There is an earlier query "$h^* = H(x^*, r^*)$" $\in T_{\mathrm{RO}}$ such that $r^* = s^* \oplus h'$.
   - B. There is an earlier query "$h'_\Phi = H'(x^*, t_\Phi)$" $\in T_{\mathrm{RO}}$, for some $(s_\Phi, t_\Phi) \in \Phi$, such that $s^* \oplus h' = s_\Phi \oplus h'_\Phi$.
5. If there is no anchor query, choose an arbitrary $x^* \in \mathcal{X}$.
6. Send $(\mathsf{Extract}, \mathsf{sid}, x^*, \mathrm{A}(s,t))$ to $\mathcal{F}_{\mathrm{POPF}}$.

On $(\mathsf{Eval}, \mathsf{sid}, \phi = (s,t), x)$ from $\mathcal{F}_{\mathrm{POPF}}$:

7. Evaluate $y := H(x, s \oplus H'(x,t)) \cdot t$, using the $\mathcal{F}_{\mathrm{RO}}$ interface defined below.
8. Send $(\mathsf{Eval}, \mathsf{sid}, y)$ to $\mathcal{F}_{\mathrm{POPF}}$.

On $H(x,r)$ from P aimed at $\mathcal{F}_{\mathrm{RO}}$:

9. Find a query "$h = H(x,r)$" $\in T_{\mathrm{RO}}$, or determine h using to the following three cases if none exists:
   - A. If there exist $(s,t) \in \Phi$ and "$h' = H'(x,t)$" $\in T_{\mathrm{RO}}$ such that $r = s \oplus h'$, then send $(\mathsf{Eval}, \mathsf{sid}, (s,t), x)$ to $\mathcal{F}_{\mathrm{POPF}}$. On response $(\mathsf{Eval}, \mathsf{sid}, y)$ from $\mathcal{F}_{\mathrm{POPF}}$, compute $h := y/t$.
   - B. Otherwise if $t_g \neq \perp$ and there is a query "$h' = H'(x, t_g)$" $\in T_{\mathrm{RO}}$, then set $s_g := r \oplus h'$. Send $(\mathsf{TestOutput}, \mathsf{sid}, \mathrm{A}(s_g, t_g), x)$ to $\mathcal{F}_{\mathrm{POPF}}$, and on response $(\mathsf{TestOutput}, \mathsf{sid}, y)$, compute $h := y/t_g$.
   - C. Otherwise sample $h \in \mathbb{G}$.

   In all cases, append "$h = H(x,r)$" to T_{RO}.
10. Return h to P.

On $H'(x,t)$ from P aimed at $\mathcal{F}_{\mathrm{RO}}$:

11. If $g \neq 0$ and t is the g-th unique value that appears in such a query, set $t_g := t$.
12. Find a query "$h' = H'(x,t)$" $\in T_{\mathrm{RO}}$, or if none exists, sample $h' \leftarrow \{0,1\}^{3\kappa}$ and append "$h' = H'(x,t)$" to T_{RO}.
13. Return h' to P.

At the end of the experiment:

14. If $\mathsf{Extract}$ has not been called, abort unless $g = 0$.

Figure 11: POPF pseudo-simulator $\mathcal{S}_{\mathrm{pseudo}}$.

Parameters:

- Simulator $\mathcal{S}$ for overlying protocol ρ UC realizing $\mathcal{F}'$.

Global variables:

- Lists T_{RO}, T_R of RO evaluations.
- The extraction target $\phi^* = (s^*, t^*)$.
- A randomly sampled string $\alpha^* \leftarrow \{0,1\}^{3\kappa}$.

Run the simulator $\mathcal{S}$, delivering its messages with $\mathcal{F}'$ as normal. Communication of $\mathcal{S}$ with $\mathcal{A}$ and the corrupted parties is filtered when $\mathcal{S}$ is acting as $\mathcal{F}_{\text{POPF}}$. Such messages are handled as follows.

On queries $(\mathsf{Program}, \mathsf{sid})$, $(\mathsf{Program}, \mathsf{sid}, A)$, $(\mathsf{Extract}, \mathsf{sid}, \phi)$, $(\mathsf{Eval}, \mathsf{sid}, \phi, x)$, $H(x, r)$, or $H'(x, t)$:

1. Handle the query as in $\mathcal{S}_{\text{pseudo}}$, except:
   A. In Extract, skip the abort (step 3), and return α^* instead of $\text{A}(s^*, t^*)$.
   B. In $H(x, r)$, remove case 9B to avoid querying R (through TestOutput).
2. At the end of the experiment, skip the abort (step 14).

Additionally, modify $\mathcal{S}$ to replace the random function R with the following.
On lookup $R(\alpha, x)$:

3. If there is a tuple $(\alpha, x, r) \in T_R$, return r.
4. Pick r according to two cases:
   A. If $\alpha = \alpha^*$ and $x \neq x^*$, evaluate $r := H(x, s^* \oplus H'(x, t^*)) \cdot t^*$.
   B. Otherwise, sample $r \leftarrow \mathbb{G}$.
5. Add (α, x, r) to T_R.
6. Return r.

Figure 12: POPF simulator $\mathcal{S}_{\text{POPF}}$.

The proof concludes by noting that, if the environment has advantage $\mathsf{Adv}_{\text{POPF-abort}}$ of distinguishing the real world with abort from the ideal world with abort, then its actual advantage of distinguishing the real world from the ideal world (without abort) is upper-bounded as $\mathsf{Adv}_{\text{POPF}} \leq (q'_h + 1)\mathsf{Adv}_{\text{POPF-abort}}$.

Bad events. Even if $\mathcal{S}_{\text{pseudo}}$ does not abort, its simulation is still not perfect, due to the possibility of some bad events. It is more convenient to exclude the bad events from the beginning of the proof (i.e., in the real world). Formally, this can viewed as modifying the real world so that if any of these bad events occur then it will reset, and start running the ideal world from the start instead. Note, however, that these events will be defined using Φ and $\phi^* = (s^*, t^*)$, which are defined in the ideal functionality, not the real world. But they *can* all be observed by the honest parties (and so by the environment) in the real world: Φ is the set of all outputs generated by the ideal functionality's Program interface, which is only run by honest parties as it is never called by the simulator, and ϕ^* is the first index not in Φ that is passed to Eval by an honest party. In the real world, we define them to match these observables.

Below we list these bad events:

- Bad_1: When $(\mathsf{Program}, \ldots)$ queries $h = H(x^*, r)$ and $h' = H'(x^*, t)$, the inputs overlaps with previous H and H' queries. That is, h and h' are not freshly random.
- Bad_2: When $(\mathsf{Program}, \ldots)$ computes (s, t), there is some $(s', t) \in \Phi$; or if (s^*, t^*) is defined, $t = t^*$.
- Bad_3: When $(\mathsf{Program}, \ldots)$ is queried, returning (s, t), look through the queries of the form

"$H(x,r)$" in $\mathcal{F}_{\mathrm{RO}}$'s set T_{RO} from before this Program query. That is, exclude the H query made during Program. The bad event occurs if there is a corresponding entry "$s \oplus r = H'(x,t)$" $\in T_{\mathrm{RO}}$, or if this entry is later added to T_{RO}.

- Bad_4: The honest POPF used by a given RO query is ambiguous. That is, for some "$H(x,r)$" $\in T_{\mathrm{RO}}$, there are distinct $(s,t),(s',t') \in \Phi$ such that both "$s \oplus r = H'(x,t)$" and "$s' \oplus r = H'(x,t')$" are in T_{RO}.
- Bad_5: The anchor query is not unique. That is, after all queries have been made, there are two distinct queries "$H'(x_0,t)$", "$H'(x_1,t)$" $\in T_{\mathrm{RO}}$ that both satisfy the conditions of being an anchor query.

Let Bad be the disjunction of these bad events. Assuming that Bad cannot occur can change the advantage by at most $\Pr[\mathsf{Bad}]$. Therefore, the environment's advantage will be bounded as $\mathsf{Adv}_{\mathrm{POPF}} \leq (q'_h + 1)\mathsf{Adv}_{\mathrm{POPF\text{-}abort}} + \Pr[\mathsf{Bad}]$.

Detailed proof. Let $\mathcal{Z}$ be the environment, and $\mathcal{A}$ be the adversary.

Lemma 4.4. *Assume $\mathcal{Z}$ issues q_p Program commands, q_h commands to $\mathcal{F}_{\mathrm{RO}}$ where the last bit of* sid *is* 0 *(i.e., H queries), and q'_h commands to $\mathcal{F}_{\mathrm{RO}}$ where the last bit of* sid *is* 1 *(i.e., H' queries). Then in the real world,*

$$\Pr[\mathsf{Bad}] \leq \frac{(2q'_h + 3q_p + 1)q_p}{2|\mathbb{G}|} + \frac{q_p(q_p+3)(q_h+q_p) + (q_h + q'_h + q_p)^2(q'_h + q_p)}{2^{3\kappa+1}}.$$

Proof. We bound the probability of each bad event, assuming that the previous bad events do not occur. All of our bad events are contained in unions of simpler bad events, such as sampling $t \leftarrow \mathbb{G}$ and finding that it equals a predetermined value $g \in \mathbb{G}$, which have obviously negligible probabilities. The sizes of these unions are polynomials in the number of executions of Program, H, and H'. Note, however, that the total number of executions of H (resp. H') is really $q_h + q_p$ (resp. $q'_h + q_p$), not q_h (resp. q'_h), because every call to Program issues one query to each of H and H',

- Bad_1: When the $h := H(x^*, r)$ query is executed by Program, the value r is freshly sampled from $\{0,1\}^{3\kappa}$. There are $q_h + q_p$ previous executions of H, so r overlaps a past query with probability at most $q_h/2^{3\kappa}$. Assuming that r is distinct, the result h is freshly random from $\mathbb{G}$. Then $H'(x^*, t)$ will also be distinct from all past queries, except with probability at most $q'_h/|\mathbb{G}|$, because $t = y^*/h$. That is, t will be uniformly random in $\mathbb{G}$, independent from all past queries to H', of which there are at most $q'_h + q_p$. Therefore, a union bound shows that

$$\Pr[\mathsf{Bad}_1] \leq \frac{(q_h + q_p)q_p}{2^{3\kappa}} + \frac{(q'_h + q_p)q_p}{|\mathbb{G}|}.$$

- Bad_2: We first note that a pair (s,t) is added to Φ only when a Program command is issued by $\mathcal{Z}$, so $|\Phi| \leq q_p$. The value t is generated by π_{POPF} using an RO query as $t = y^*/H(x^*, r)$, and $H(x^*, r)$ is freshly random assuming that Bad_1 does not occur. Therefore, $t \in \mathbb{G}$ is uniformly random and independent of Φ, and the probability that t is already in Φ, i.e., there is a collision in the t values in Φ, is at most $\binom{q_p}{2}/|\mathbb{G}|$. Also, the probability that Program generates a pair (s,t) such that $t = t^*$ is at most $q_p/|\mathbb{G}|$. Therefore,

$$\Pr[\mathsf{Bad}_2 \wedge \neg\mathsf{Bad}_1] \leq \frac{q_p^2 + q_p}{2|\mathbb{G}|}.$$

- Bad_3: For Bad_3 to occur, it must hold that $s \oplus r = H'(x,t)$ (or equivalently, $s = r \oplus H'(x,t)$) for some s,t generated by $\mathsf{Program}$ and some x,r with $H(x,r)$ *previously* queried by either $\mathcal{Z}$ or $\mathsf{Program}$. There are at most q_p (s,t) pairs and at most $q_h + q_p$ H queries; these two combined uniquely determine the values of x and t, which in turn determine the $H'(x,t)$ query. For each pair $(s,t) \in \Phi$ and each $H(x,r)$ query, assuming that Bad_1 does not occur, s is a uniformly random string in $\{0,1\}^{3\kappa}$ independent of r and $H'(x,t)$[16], so the probability that $s = r \oplus H'(x,t)$ is $1/2^{3\kappa}$. A union bound gives

$$\Pr[\mathsf{Bad}_3 \wedge \neg\mathsf{Bad}_1] \leq \frac{q_p(q_h+q_p)}{2^{3\kappa}}.$$

- Bad_4: For Bad_4 to occur, it must hold that $s \oplus r = H'(x,t)$ and $s' \oplus r = H'(x,t')$ for some distinct pairs $(s,t),(s',t')$ generated by $\mathsf{Program}$ and some x,r with $H(x,r)$ queried by $\mathcal{Z}$ or $\mathsf{Program}$. This implies that $s \oplus s' = H'(x,t) \oplus H'(x,t')$. Similarly to the analysis of Bad_3, the query $H(x,r)$ and the POPF indexes (s,t) and (s',t') uniquely determine the values of x,t,t', which in turn determine *both* the $H'(x,t)$ query and the $H'(x,t')$ query. Assuming that Bad_1 and Bad_2 do not occur, we have that $t \neq t'$, so $s, s', H'(x,t), H'(x,t')$ are mutually independent strings in $\{0,1\}^{3\kappa}$. Therefore,

$$\Pr[\mathsf{Bad}_4 \wedge \neg\mathsf{Bad}_1 \wedge \neg\mathsf{Bad}_2] \leq \frac{q_p(q_p-1)(q_h+q_p)}{2^{3\kappa+1}}.$$

  (The analysis of Bad_4 is not covered by the analysis of Bad_3, since here r might depend on s.)

- Bad_5: Recall that for an anchor query (x,t), either of the followings must hold:
  - $r = s \oplus H'(x,t)$, where $H(x,r)$ was previously queried by $\mathcal{Z}$; or
  - $s \oplus H'(x,t) = s_\Phi \oplus H'(x,t_\Phi)$ for some (s_Φ, t_Φ) sampled by an honest party,

  where s,t are specified by the $\mathsf{Extract}$ command. Therefore, if there are two distinct anchor queries, then one of the followings must happen:
  - $r_0 \oplus H'(x_0,t) = s = r_1 \oplus H'(x_1,t)$ for some previous queries $H(x_0,r_0)$ and $H(x_1,r_1)$;
  - $r \oplus H'(x_0,t) = s = s_\Phi \oplus H'(x_1,t_\Phi) \oplus H'(x_1,t)$ for some previous $H(x_0,r)$ query and some (s_Φ,t_Φ) sampled by an honest party; or
  - $s_{\Phi,0} \oplus H'(x_0,t_{\Phi,0}) \oplus H'(x_0,t) = s = s_{\Phi,1} \oplus H'(x_1,t_{\Phi,1}) \oplus H'(x_1,t)$ for some $(s_{\Phi,0},t_{\Phi,0})$ and $(s_{\Phi,1},t_{\Phi,1})$ sampled by an honest party.

  Let us count the ways in which these sub-events can occur. All variables in the first sub-event are determined by the two H queries and an H' query; assuming Bad_2 does not occur, all variables in the second sub-event are determined by the queries $H(x_0,r)$, $H'(x_0,t)$, and $H'(x_1,t_\Phi)$ (note that s_Φ is uniquely determined by t_Φ); assuming Bad_2 does not occur, all variables in the third sub-event are determined by the $H'(x_0,t)$, $H'(x_0,t_{\Phi,0})$, and $H'(x_1,t_{\Phi,1})$ queries. In all three sub-events, one of the H' queries will come last, and so be uniformly random in $\{0,1\}^{3\kappa}$, independent of the H queries and the other variables, and so will trigger the bad event with probability $2^{-3\kappa}$. Adding up, we get

$$\Pr[\mathsf{Bad}_5 \wedge \neg\mathsf{Bad}_2] \leq \frac{q_h^2(q_h'+q_p) + 2q_h(q_h'+q_p)^2 + (q_h'+q_p)^3}{2^{3\kappa+1}} = \frac{(q_h+q_h'+q_p)^2(q_h'+q_p)}{2^{3\kappa+1}}.$$

[16] We can ignore the $H'(x^*,t)$ query made in $\mathsf{Program}$, because the corresponding $H(x^*,r)$ query has been excluded.

Summing up the five bad events above yields the lemma. □

Lemma 4.5. *Assume that* Bad *does not occur. Then $\mathcal{Z}$'s distinguishing advantage between the real world with abort and the pseudo-intermediate world is at most $q_h^2/2^{3\kappa+1}$.*

Proof. Consider the following hybrid argument:

Hybrid 0: This is the real world with abort. The environment $\mathcal{Z}$'s view is shown in Figure 13. Note that the experiment aborts with probability $q'_H/(q'_H+1)$ (see steps 5 and 8), but otherwise behaves exactly as the same as the real world.

Sample $g \leftarrow [0, q'_H]$.

On $(\mathsf{Program}, \mathsf{sid}, x^*, y^*)$ (where $x^* \in \mathcal{X}$ and $y^* \in \mathbb{G}$):

1. Choose $r \leftarrow \{0,1\}^{3\kappa}$.
2. Compute $t := y^*/H(x^*, r)$.
3. Compute $s := r \oplus H'(x^*, t)$.
4. Output $(\mathsf{Program}, \mathsf{sid}, (s, t))$.

On $(\mathsf{Eval}, \mathsf{sid}, (s, t), x)$ (where $x \in \mathcal{X}$, $s \in \{0,1\}^{3\kappa}$, and $t \in \mathbb{G}$):

5. If this is the first Eval message for sid, abort if either $g \neq 0$ and t is not the g-th unique value that appears in a query $H'(\cdot, t)$, or if $g = 0$ and $H'(\cdot, t)$ has been queried before.
6. Compute $y := H(x, s \oplus H'(x, t)) \cdot t$.
7. Output $(\mathsf{Eval}, \mathsf{sid}, y)$.

At the end of the experiment:

8. If Eval has not been called, abort unless $g = 0$.

Figure 13: $\mathcal{Z}$'s view in the real world with abort.

Hybrid 1: At the end of $(\mathsf{Program}, \mathsf{sid}, x^*, y^*)$, define $F_{(s,t)}(x^*) := y^*$; at the beginning of $(\mathsf{Eval}, \mathsf{sid}, (s, t), x)$, if $F_{(s,t)}(x)$ is already defined, then return $(\mathsf{Eval}, \mathsf{sid}, y)$ where $y := F_{(s,t)}(x)$; at the end of $(\mathsf{Eval}, \mathsf{sid}, (s, t), x)$, define $F_{(s,t)}(x) := y$.

Suppose that in hybrid 1, $F_{(s,t)}(x)$ is already defined when $\mathcal{Z}$ sends a command $(\mathsf{Eval}, \mathsf{sid}, (s, t), x)$. This means that there is a previous command $(\mathsf{Program}, \mathsf{sid}, x, y)$ with $t = y/H(x, r)$ and $s = r \oplus H'(x, t)$ for some $r \in \{0,1\}^{3\kappa}$. But then $y = F_{(s,t)}(x) = H(x, r) \cdot t = H(x, s \oplus H'(x, t)) \cdot t$, exactly as how y is computed in hybrid 0. The only other difference between hybrids 0 and 1 is the bookkeeping of the F function. Therefore, hybrids 0 and 1 are identical.

The following hybrids 2 and 3 consider the indices of honest POPFs $(s, t) \in \Phi$.

Hybrid 2: On $(\mathsf{Program}, \mathsf{sid}, x^*, y^*)$, choose $(s, t) \leftarrow \{0,1\}^{3\kappa} \times \mathbb{G}$ and output $(\mathsf{Program}, \mathsf{sid}, (s, t))$. Furthermore, compute $r := s \oplus H'(x^*, t)$; if $H(x^*, r)$ is queried, return y^*/t.

Assuming Bad_4 does not occur, when $\mathcal{Z}$ queries $H(x^*, r)$, there do not exist two distinct pairs $(s, t), (s', t')$ such that $r = s \oplus H'(x^*, t) = s' \oplus H'(x^*, t')$, so the query can be answered unambiguously *if we only consider those queries defined by* Program *commands.* Furthermore, assuming Bad_3 does not occur, it cannot happen that an H-query defined by a Program command has already been queried during a previous Eval command. It follows that H queries can be answered unambiguously, and thus hybrid 2 is well-defined.

We can see that in both hybrids 1 and 2, the variables $s, t, r, h = H(x^*, r)$ satisfy the equations

$$s \oplus r = H'(x^*, t), \quad y^* = h \cdot t;$$

if we choose $(r, h) \leftarrow \{0,1\}^{3\kappa} \times \mathbb{G}$ then we get hybrid 1, whereas if we choose $(s, t) \leftarrow \{0,1\}^{3\kappa} \times \mathbb{G}$ then we get hybrid 2. So hybrids 1 and 2 are identical.

Hybrid 3: On $(\mathsf{Eval}, \mathsf{sid}, (s,t), x)$, if $F_{(s,t)}(x)$ is undefined and $(s,t) \in \Phi$, sample $y \leftarrow \mathbb{G}$, set $F_{(s,t)}(x) := y$, and return $(\mathsf{Eval}, \mathsf{sid}, y)$. Furthermore, compute $r := s \oplus H'(x,t)$; when $H(x,r)$ is queried, return y/t.

Similar to the analysis in hybrid 2, assuming neither Bad_3 nor Bad_4 occurs, hybrid 3 is well-defined. By an analysis similar to that in hybrid 2, hybrids 2 and 3 are identical.

The following hybrids 4–7 consider the "designated malicious index" (s^*, t^*).

Hybrid 4: In step 5 of Figure 13 (note that the previous hybrids do not change this step), if the experiment does not abort, set $(s^*, t^*) := (s, t)$ and find the anchor query "$H'(x^*, t^*)$". If there is no anchor query, choose an arbitrary $x^* \in \mathcal{X}$.

Assuming Bad_5 does not occur, the anchor query is uniquely defined, hence hybrid 4 is well-defined. The only difference between hybrids 3 and 4 is bookkeeping, so they are identical.

Hybrid 5: Sample $\mathrm{A} \leftarrow \left(\{0,1\}^{3\kappa}\right)^{\{0,1\}^{3\kappa} \times \mathbb{G}}$ at the beginning of the experiment. In the case that there is an "$h' = H'(x, t_g)$" query and then an "$H(x,r)$" query for some x, r (where x may or may not be equal to x^*), set $s_g := r \oplus h'$, $y := R(\mathrm{A}(s_g, t_g), x)$, and answer with $h := y/t_g$.

Let $\mathsf{Collide}$ be the event that two queries $\mathrm{A}(s_g, t_g)$, $\mathrm{A}(s'_g, t'_g)$ generate the same output; obviously, $\Pr[\mathsf{Collide}] \leq q_h^2/2^{3\kappa+1}$. We now argue that in hybrid 5, if $\mathsf{Collide}$ does not occur, then the y values are independently random in $\mathcal{Z}$'s view for different $H(x,r)$ queries. Suppose that there are two different queries $H(x_0, r_0), H(x_1, r_1)$. We have that (1) if $x_0 \neq x_1$, then the corresponding inputs to R are different; (2) if $x_0 = x_1 = x$ but $r_0 \neq r_1$, then the corresponding s values are $r_0 \oplus H'(x, t_g)$ and $r_1 \oplus H'(x, t_g)$ which are different, so the corresponding inputs to R are different because $\mathsf{Collide}$ does not occur. We conclude that all the outputs of R — i.e., the y values — are independently random in $\mathcal{Z}$'s view.

We can see that in both hybrids 4 and 5, the variables y, h satisfy the equation

$$y = h \cdot t_g;$$

if we choose $h \leftarrow \mathbb{G}$ then we get hybrid 4, whereas if we choose $y \leftarrow \mathbb{G}$ then we get hybrid 5 (except when $\mathsf{Collide}$ occurs). Therefore, $\mathcal{Z}$'s distinguishing advantage between hybrids 4 and 5 is at most $\Pr[\mathsf{Collide}] \leq q_h^2/2^{3\kappa+1}$.

Hybrid 6: In the condition in hybrid 4 — that is, when step 5 of Figure 13 runs, assuming it does not abort — if $F_{(s^*,t^*)}(x^*)$ is not already defined and $x \neq x^*$, then compute $y^* := H(x^*, s^* \oplus H'(x^*, t^*)) \cdot t^*$ and set $F_{(s^*,t^*)}(x^*) := y^*$ (in addition to what hybrid 4 already does). Note that the condition in hybrid 5 may be triggered while computing the H output.

In hybrid 5, $F_{(s^*,t^*)}(x^*)$ is defined as y^* when $\mathcal{Z}$ sends $(\mathsf{Eval}, \mathsf{sid}, (s^*, t^*), x^*)$, whereas in hybrid 6, $F_{(s^*,t^*)}(x^*)$ is defined as y^* when $\mathcal{Z}$ sends $(\mathsf{Eval}, \mathsf{sid}, (s^*, t^*), x)$ for the first time for any x. This change does not affect $\mathcal{Z}$'s view.

Hybrid 7: On $(\mathsf{Eval}, \mathsf{sid}, (s^*, t^*), x)$, if $x \neq x^*$ and $F_{(s^*,t^*)}(x)$ is undefined, set $y := R(\alpha^*, (s^*, t^*), x)$ and $F_{(s^*,t^*)}(x) := y$, and return $(\mathsf{Eval}, \mathsf{sid}, y)$.

The difference between hybrids 6 and 7 is that an $(\mathsf{Eval}, \mathsf{sid}, (s^*, t^*), x)$ command is answered with $H(x, s^* \oplus H'(x, t^*)) \cdot t^*$ in hybrid 6 and $R(\alpha^*, (s^*, t^*), x)$ in hybrid 7. We consider two cases:

- If $\mathcal{Z}$ has queried $H(x, s^* \oplus H'(x, t^*))$:
  - If $\mathcal{Z}$ queried $H(x, s^* \oplus H'(x, t^*))$ and then $H'(x, t^*)$, this means that $H'(x, t^*)$ is an anchor query and thus $x = x^*$, so the change in hybrid 7 does not affect this case.

– If $\mathcal{Z}$ queried $H'(x,t^*)$ and then $H(x, s^* \oplus H'(x,t^*))$, according to the description of hybrid 5, we have that $R(\alpha^*,(s^*,t^*),x) = H(x, s^* \oplus H'(x,t^*)) \cdot t^*$ since $t^* = t^g$ (because we haven't aborted), so $\mathcal{Z}$'s views in hybrids 6 and 7 are identical.

- If $\mathcal{Z}$ has not queried $H(x, s^* \oplus H'(x,t^*))$:
  - $H(x, s^* \oplus H'(x,t^*))$ appears in the experiment only when $\mathcal{Z}$ sends $(\mathsf{Eval}, \mathsf{sid}, (s_\Phi, t_\Phi), x)$ for some other (s_Φ, t_Φ) such that $s^* \oplus H'(x,t^*) = s_\Phi \oplus H'(x,t_\Phi)$. But then $H'(x,t^*)$ is an anchor query and thus $x = x^*$, so the change in hybrid 7 does not affect this case.
  - If $H(x, s^* \oplus H'(x,t^*))$ does not appear in the experiment, then both $R(\alpha^*,(s^*,t^*),x)$ and $H(x, s^* \oplus H'(x,t^*))$ are random elements of $\mathbb{G}$, so $\mathcal{Z}$'s views in hybrids 6 and 7 are identical.

We conclude that $\mathcal{Z}$'s views in hybrids 6 and 7 are identical.

We now claim that hybrid 7 is identical to the pseudo-intermediate world. In both worlds, a $(\mathsf{Program}, \mathsf{sid}, x^*, y^*)$ command is answered with $(s,t) \leftarrow \{0,1\}^{3\kappa} \times \mathbb{G}$, and an H' query is answered with a random string in $\{0,1\}^{3\kappa}$. For an $(\mathsf{Eval}, \mathsf{sid}, (s,t), x)$ command, in both worlds,

- If $F_{(s,t)}(x)$ is already defined, then the answer is $F_{(s,t)}(x)$. This can be seen from hybrid 1 and step 5 (without entering any of the sub-conditions) of Figure 8.
- If $F_{(s,t)}(x)$ is undefined and $(s,t) \in \Phi$, then the answer is a uniformly random element of $\mathbb{G}$. This can be seen from hybrid 3 and step 5A of Figure 8.
- If $F_{(s,t)}(x)$ is undefined and $(s,t) = (s^*,t^*)$, then if $x = x^*$, the answer is $H(x^*, s^* \oplus H'(x^*,t^*)) \cdot t^*$ as can be seen from hybrid 6 and step 4 of Figure 8; if $x \neq x^*$, the answer is $R(\alpha^*,(s^*,t^*),x)$ as can be seen from hybrid 7 and step 5B of Figure 8.
- If $F_{(s,t)}(x)$ is undefined, $(s,t) \in \Phi$ and $(s,t) \neq (s^*,t^*)$, then the answer is $H(x, s \oplus H'(x,t)) \cdot t$. This is unchanged throughout the hybrids and in the pseudo-intermediate world can be seen from step 5C of Figure 8.

Finally, for an $H(x,r)$ query, in both worlds,

- If $r = s \oplus H'(x,t)$ for some $(s,t) \in \Phi$, then the answer is $y/t = F_{(s,t)}(x)/t$. This can be seen from hybrid 3 and step 9A of Figure 11.
- If there is an "$h' = H'(x,t_g)$" query and then an "$H(x,r)$" query for some x, r, then the answer is $y/t_g = R(\alpha^*,x)/t_g$. This can be seen from hybrid 5 and step 9B of Figure 11.
- Otherwise the answer is a random element in $\mathbb{G}$. This is unchanged throughout the hybrids and in the pseudo-intermediate world can be seen from step 9C of Figure 11.

We conclude that hybrid 7 and the pseudo-intermediate world are identical. The only hybrid that generates a non-identical view is hybrid 5, so $\mathcal{Z}$'s distinguishing advantage between the real world with abort and the pseudo-intermediate world is at most $q_h^2/2^{3\kappa+1}$. This completes the proof. □

Lemma 4.6. *Assume that* Bad *does not occur, and at most q_R queries to R are made by $\mathcal{S}$. Then $\mathcal{Z}$'s distinguishing advantage between the pseudo-ideal world from the ideal world with abort is at most $3q_R^2/2^{3\kappa+1}$.*

Proof. Consider the following hybrid argument:

Hybrid 0: This is the pseudo-ideal world. Step 9B of the pseudo-simulator $\mathcal{S}_{\text{pseudo}}$ currently programs H so that $H(x, H'(x, t_g) \oplus s_g) \cdot t_g = R(\mathrm{A}(s_g, t_g), x)$ holds for all s_g, whenever the H' query is made before the H query. In particular, this holds for $(s_g, t_g) = (s^*, t^*)$ and $x \neq x^*$, by the uniqueness of the anchor query (or else Bad_5 would occur). When such an H query is made for any s_g, $\mathcal{S}_{\text{pseudo}}$ computes $\alpha = \mathrm{A}(s_g, t_g)$ and $R(\alpha, x)$, and uses them to define the output of H. $R(\alpha, (s_g, t_g), x)$ is either a freshly random value here, or if it was previously queried (e.g., by $\mathcal{S}$) then it was sampled randomly when it was first queried.

Hybrid 1: Change to an equivalent distribution by swapping which of H and R is used to define the other. Instead of sampling R and programming H to match as with case 9B, let H be sampled uniformly and program R to match. However, there is no need to program $R(\alpha, x)$ other than when $\alpha = \alpha^*$ (i.e., $\alpha = \mathrm{A}(s^*, t^*)$), since no other evaluation of A is ever revealed by $\mathcal{S}_{\text{pseudo}}$. Therefore, we only program $R(\alpha^*, x)$ for $x \neq x^*$.

More precisely, let H be defined as in the ideal world, by removing step 9B from $\mathcal{S}_{\text{pseudo}}$. Then whenever $R(\alpha^*, x)$ is evaluated (after s^*, t^* and α^* have been defined), if $x \neq x^*$, compute $r = H(x, H'(x, t^*) \oplus s^*) \cdot t^*$ and program $R(\alpha^*, x) := r$. Steps 1B and 3–6 of Figure 12 are pseudocode for these two changes.

This change can only be noticed if either the R evaluations are not all independently random, or if $\mathcal{Z}$ manages to make an $R(\alpha, x)$ query on some $\alpha = \mathrm{A}(s_g, t_g)$ not yet revealed by the simulator. That is, either there was a collision in A, or when $\mathcal{Z}$'s $R(\alpha, x)$ query was made either $\alpha \neq \alpha^*$ or α^* was not yet been returned by $\mathsf{Extract}$. There are at most q_R queries to A and to R, since A is only queried by H, and before the hybrid queries to H result in queries to R.Therefore, these events are upper bounded by $q_R^2/2^{3\kappa+1}$ and $q_R^2/2^{3\kappa}$, respectively, so $\mathcal{Z}$'s distinguishing advantage between hybrids 1 and 2 is at most $3q_R^2/2^{3\kappa+1}$.

Hybrid 2: Remove the RO A, and instead just sample $\alpha^* \leftarrow \{0,1\}^{3\kappa}$ and replace the query to $\mathrm{A}(s^*, t^*)$ with α^*. The is equivalent because A is only called on input (s^*, t^*).

Hybrid 3: Notice that g and t_g are no longer used, except for the abort. This abort always has probability $q'_H/(q'_H + 1)$, so we can remove g and replace it with a simple abort with this probability.

Hybrid 4: Combine the higher-level protocol's simulator $\mathcal{S}$ with the modified pseudo-simulator $\mathcal{S}_{\text{pseudo}}$. Call their composition $\mathcal{S}_{\text{POPF}}$. We are now at the pseudo-ideal world.

The only hybrid that generates a non-identical view is hybrid 5, so $\mathcal{Z}$'s distinguishing advantage between the real world with abort and the pseudo-intermediate world is at most $3(q_h + q_R)^2/2^{3\kappa+1}$. This completes the proof. □

We can now put these lemmas together to get a proof of Thm. 4.3.

Proof. First, we bound $\mathsf{Adv}_{\text{POPF-abort}}$ using the hybrid argument outlined above. These hybrids are first those given in Thm. 4.5, then a single hybrid change for the security of ρ, and finally those in Thm. 4.6. Adding the advantages together, we get

$$\begin{aligned}\mathsf{Adv}_{\text{POPF-abort}} &\leq \frac{q_h^2}{2^{3\kappa+1}} + \mathsf{Adv}_\rho + \frac{3q_R^2}{2^{3\kappa+1}} \\ &= \mathsf{Adv}_\rho + \frac{q_h^2 + 3q_R^2}{2^{3\kappa+1}}.\end{aligned}$$

Finally, put this together with Thm. 4.4 to get:

$$\begin{aligned}
\mathsf{Adv}_{\mathrm{POPF}} &\leq (q'_h+1)\mathsf{Adv}_{\mathrm{POPF\text{-}abort}} + \Pr[\mathsf{Bad}] \\
&\leq (q'_h+1)\mathsf{Adv}_{\rho} + \frac{(q'_h+1)(q_h^2+3q_R^2)}{2^{3\kappa+1}} + \frac{(2q'_h+3q_p+1)q_p}{2|\mathbb{G}|} \\
&+ \frac{q_p(q_p+3)(q_h+q_p)+(q_h+q'_h+q_p)^2(q'_h+q_p)}{2^{3\kappa+1}} \\
&\leq (q'_h+1)\mathsf{Adv}_{\rho} + \frac{(2q'_h+3q_p+1)q_p}{2|\mathbb{G}|} + \frac{(q_h+q'_h+q_R+q_p+1)^3}{2^{3\kappa}}.
\end{aligned}$$

□

5 PAKE Protocols Based on POPF

In this section we present our main results, namely the UC-security analysis of EKE and OEKE using POPF. Concretely,

- In Sect. 5.2 we prove that EKE-PRF using POPF is UC-secure assuming the underlying KA protocol satisfies security, strong pseudorandomness, and pseudorandom non-malleability. Additional results about the "raw" version of EKE, including (1) EKE using IC is UC-secure, and (2) EKE using POPF realizes the weaker "PAKE with same password test" functionality $\mathcal{F}_{\text{PAKE-sp}}$ (both results are under the same assumptions on KA), are argued in Sect. B.
- In Sect. 5.3 we prove that OEKE using POPF is UC-secure assuming the underlying KA protocol satisfies security, pseudorandomness, pseudorandom non-malleability, and collision resistance. This covers the OEKE-PRF and OEKE-RO variants in existing works.

Our starting observation is that the first round of EKE and OEKE are exactly identical. To make our security proofs more concise and modular, we abstract the first round into an ideal functionality $\mathcal{F}_{\text{EKE-1r}}$, and prove that the first round of (O)EKE realizes $\mathcal{F}_{\text{EKE-1r}}$; after that, we prove separately that the EKE and OEKE are secure in the $\mathcal{F}_{\text{EKE-1r}}$-hybrid world.

5.1 The First-Round Functionality and Protocol

The functionality. See Figure 14 for the UC functionality $\mathcal{F}_{\text{EKE-1r}}$ representing the first-round of (O)EKE, parameterized by a specific underlying KA protocol KA. (Although we name the functionality $\mathcal{F}_{\text{EKE-1r}}$, we stress that the first round of OEKE is represented by the same functionality.)

Recall that in the first round of both EKE and OEKE, party P (with password pw) computes its KA message A, programs ϕ such that $A = F_\phi(\mathsf{pw})$, and sends ϕ to P'. Then P' (with password pw') evaluates $A' = F_\phi(\mathsf{pw}')$, samples randomness $b \leftarrow \mathcal{R}$, sends $B := \mathsf{msg}_2(b, A')$ to P, and outputs $K' := \mathsf{key}_2(b, A')$. (Note that the randomness a that corresponds to A is not used in the first round.) The man-in-the-middle adversary has the following capacity:

- The adversary can evaluate $F_\phi(x)$ for any (fresh) x of its choice. If $x = \mathsf{pw}$ (which is the point at which F_ϕ is programmed) the result is A, otherwise the result is a random value.
- The adversary may or may not modify the P-to-P' message ϕ:
  - Suppose the adversary does not modify the message. Then if $\mathsf{pw} = \mathsf{pw}'$ (i.e., the passwords of P and P' match), P' will send $B = \mathsf{msg}_2(b, A)$ to P and output $K' = \mathsf{key}_2(b, A)$; if $\mathsf{pw} \neq \mathsf{pw}'$, P' will send $B = \mathsf{msg}_2(b, A')$ and output $K' = \mathsf{key}_2(b, A')$ where $A' = F_\phi(\mathsf{pw}')$ is a random value, so B and K' are pseudorandom.

Parameters: KA protocol $\mathsf{KA} = (\mathsf{msg}_1, \mathsf{msg}_2, \mathsf{key}_1, \mathsf{key}_2)$.

- On input $(\mathsf{Program}, \mathsf{sid}, P, P', \mathsf{pw}, A)$ from P, send $(\mathsf{Program}, \mathsf{sid}, P, P')$ to $\mathcal{S}$. If this is the first Program message for sid, then record $\langle \mathsf{Program}, P, P', \mathsf{pw}, A \rangle$.
- On input $(\mathsf{SampleResp}, \mathsf{sid}, P', P, \mathsf{pw}')$ from P', send $(\mathsf{SampleResp}, \mathsf{sid}, P', P)$ to $\mathcal{S}$. If this is the first SampleResp message for sid, then sample $b \leftarrow \mathcal{R}$ and record $\langle \mathsf{SampleResp}, P', P, \mathsf{pw}', b \rangle$.
- On $(\mathsf{Eval}, \mathsf{sid}, P, P', x)$ from $\mathcal{S}$, send A to $\mathcal{S}$, where A is defined as follows:
  - If there is a record $\langle \mathsf{Program}, P, P', \mathsf{pw}, A' \rangle$ with $\mathsf{pw} = x$, set $A := A'$.
  - Otherwise if there is a record $\langle \mathsf{Eval}, \mathsf{pw}, A'' \rangle$ with $\mathsf{pw} = x$, set $A := A''$.
  - Otherwise sample $a \leftarrow \mathcal{R}$ and set $A := \mathsf{msg}_1(a)$. Record $\langle \mathsf{Eval}, x, A \rangle$.
- On $(\mathsf{Deliver}, \mathsf{sid}, P, P', \mathsf{pw}^*, A^*)$ from $\mathcal{S}$, if there is a record $\langle \mathsf{SampleResp}, P', P, \mathsf{pw}', b \rangle$, and this is the first Deliver message for sid, then output (sid, B, K') to P', where B and K' are defined as follows:
  1. If $\mathsf{pw}^* = \perp$ and there is a record $\langle \mathsf{Program}, P, P', \mathsf{pw}, A \rangle$, overwrite $\mathsf{pw}^* := \mathsf{pw}$ and $A^* := A$.
  2. Next, if $\mathsf{pw}^* = \mathsf{pw}'$, then set $B := \mathsf{msg}_2(b, A^*)$ and $K' := \mathsf{key}_2(b, A^*)$. Else, sample $B \leftarrow \mathcal{M}_2$ and $K' \leftarrow \mathcal{K}$.

Figure 14: Ideal functionality $\mathcal{F}_{\text{EKE-1r}}$ representing the first round of (O)EKE.

  - Suppose the adversary modifies the message ϕ to some other ϕ^*, which incorporates a password guess pw^*. Let $A^* = F_{\phi^*}(\mathsf{pw}^*)$; if $\mathsf{pw}^* = \mathsf{pw}'$ (i.e., the password guess is correct), P' will send $B = \mathsf{msg}_2(b, A^*)$ to P and output $K' = \mathsf{key}_2(b, A^*)$, and the adversary may compute K' as $\mathsf{key}_1(a^*, B)$ (where a^* is the randomness corresponding to A^*). If $\mathsf{pw}^* \neq \mathsf{pw}'$, P' will send $B = \mathsf{msg}_2(b, A')$ and output $K' = \mathsf{key}_2(b, A')$ where $A' = F_{\phi^*}(\mathsf{pw}')$ is a random value, so B and K' are pseudorandom.

In the real world, the evaluation of $F_\phi(x)$ is modeled by the Eval command that defines the answer A just as in the ideal world, except that if $\mathsf{pw} \neq x$ the answer is the "real" $\mathsf{msg}_1(a)$ instead of random. For the password test modeled by the Deliver command, the ideal adversary $\mathcal{S}$ may specify a pw^* and an A^*, and there are three cases:

- $\mathsf{pw}^* = \perp$ models the real-world scenario where the adversary does not modify the P-to-P' message. Then if $\mathsf{pw} = \mathsf{pw}'$, the functionality sets $B = \mathsf{msg}_2(b, A)$ and $K' = \mathsf{key}_2(b, A)$. (Concretely, the functionality enters step 1 and redefines $\mathsf{pw}^* := \mathsf{pw}$ and $A^* := A$, and then enters step 2, checks that $\mathsf{pw}^* = \mathsf{pw}'$, and sets $B = \mathsf{msg}_2(b, A^*)$ and $K' = \mathsf{key}_2(b, A^*)$.)
- $\mathsf{pw}^* \neq \perp$ models the real-world scenario where the adversary modifies the P-to-P' message and incorporates a password guess pw^* for P'. If $\mathsf{pw}^* = \mathsf{pw}'$, the functionality sets $B = \mathsf{msg}_2(b, A^*)$ and $K' = \mathsf{key}_2(b, A^*)$.
- Otherwise (i.e., $\mathsf{pw}^* = \perp$ and $\mathsf{pw} \neq \mathsf{pw}'$; or $\mathsf{pw}^* \neq \perp$ and $\mathsf{pw}^* \neq \mathsf{pw}'$) the functionality samples B and K' at random.

Parameters:

- POPF functionality $\mathcal{F}_{\text{POPF}}$ (see Figure 8) with domain $\{0,1\}^*$ and range $\mathcal{M}_1$.

On input $(\mathsf{Program}, \mathsf{sid}, P, P', \mathsf{pw}, A)$, party P does the following:

1. Send $(\mathsf{Program}, \mathsf{sid}, \mathsf{pw}, A)$ to $\mathcal{F}_{\text{POPF}}$ and wait for response $(\mathsf{Program}, \mathsf{sid}, \phi)$.
2. Send (sid, ϕ) to P'.

On input $(\mathsf{SampleResp}, \mathsf{sid}, P', P, \mathsf{pw}')$, party P' samples $b \leftarrow \mathcal{R}$ and records $\langle \mathsf{pw}', b \rangle$.

On message (sid, ϕ) from P, if there is a record $\langle \mathsf{pw}', b \rangle$ then party P' does the following:

3. Send $(\mathsf{Eval}, \mathsf{sid}, \phi, \mathsf{pw}')$ to $\mathcal{F}_{\text{POPF}}$ and wait for response $(\mathsf{Eval}, \mathsf{sid}, A')$.
4. Compute $B := \mathsf{msg}_2(b, A')$ and $K' := \mathsf{key}_2(b, A')$.
5. Output (sid, B, K').

Figure 15: (O)EKE first-round protocol, in the $\mathcal{F}_{\text{POPF}}$-hybrid world.

The protocol. See Figure 15 for the first-round protocol. Recall that the first round of all (O)EKE variants we analyze in this work is identical.

Security analysis.

Lemma 5.1. *Suppose that* KA *is a secure and pseudorandom KA protocol. Then the protocol in Figure 15 realizes $\mathcal{F}_{\text{EKE-1r}}$ (Figure 14) in the $\mathcal{F}_{\text{POPF}}$-hybrid world via a simulator $\mathcal{S}$ that emulates* TestOutput *in the same way as $\mathcal{F}_{\text{POPF}}$.*

Proof. The following lemma will be useful while analyzing the security of the first-round protocol (a proof and further discussion is provided in Sect. 3.3):

Lemma 5.2. *If a KA protocol is secure and pseudorandom then the following two distributions are indistinguishable:*

$A \leftarrow \mathcal{M}_1$	$A \leftarrow \mathcal{M}_1$
$b \leftarrow \mathcal{R}$	
$B := \mathsf{msg}_2(b, A)$	$B \leftarrow \mathcal{M}_2$
$K' := \mathsf{key}_2(b, A)$	$K' \leftarrow \mathcal{K}$
output (A, B, K')	output (A, B, K')

We construct the simulator $\mathcal{S}$ in Figure 16. As standard in UC, we assume that the adversary $\mathcal{A}$ is "dummy" that merely passes all messages to and from the environment $\mathcal{Z}$. We use the following conventions: if $\mathcal{S}$ sends a message m to $\mathcal{Z}$ that pretends to be from functionality $\mathcal{F}$ to $\mathcal{A}$, or from party P to P' and intercepted by $\mathcal{A}$, we abbreviate it as "send m from $\mathcal{F}$ to $\mathcal{A}$ (or from P to P')" — although $\mathcal{A}$ does not exist in the ideal world. Similarly, if $\mathcal{S}$ receives a message m from $\mathcal{Z}$ that instructs $\mathcal{A}$ to send m to $\mathcal{F}$ or P, we abbreviate it as "on m from $\mathcal{A}$ to $\mathcal{F}$ (or P)". These conventions are reused in later proofs.

On $(\mathsf{Program}, \mathsf{sid}, P, P')$ from $\mathcal{F}_{\text{EKE-1r}}$:

1. Send $(\mathsf{Program}, \mathsf{sid})$ from $\mathcal{F}_{\text{POPF}}$ to $\mathcal{A}$.
2. On $(\mathsf{Program}, \mathsf{sid}, \phi)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{POPF}}$, send (sid, ϕ) from P to P'.

On $(\mathsf{SampleResp}, \mathsf{sid}, P', P)$ from $\mathcal{F}_{\text{EKE-1r}}$ and (sid, ϕ^*) from $\mathcal{A}$ to P':

3. If $\phi^* = \phi$, send $(\mathsf{Deliver}, \mathsf{sid}, P, P', \perp, \perp)$ to $\mathcal{F}_{\text{EKE-1r}}$.
4. Otherwise do the following steps:
   (1) Send $(\mathsf{Extract}, \mathsf{sid}, \phi^*)$ from $\mathcal{F}_{\text{POPF}}$ to $\mathcal{A}$.
   (2) On $(\mathsf{Extract}, \mathsf{sid}, \mathsf{pw}^*, \alpha^*)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{POPF}}$, send $(\mathsf{Eval}, \mathsf{sid}, \phi^*, \mathsf{pw}^*)$ from $\mathcal{F}_{\text{POPF}}$ to $\mathcal{A}$.
   (3) On $(\mathsf{Eval}, \mathsf{sid}, A^*)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{POPF}}$, send $(\mathsf{Deliver}, \mathsf{sid}, P, P', \mathsf{pw}^*, A^*)$ to $\mathcal{F}_{\text{EKE-1r}}$.

Simulation of $\mathcal{F}_{\text{POPF}}$: run the code of $\mathcal{F}_{\text{POPF}}$ (Figure 8), except that when $\mathcal{A}$ queries $(\mathsf{Eval}, \mathsf{sid}, \phi, x)$ in step 5A, if ϕ has been generated (in step 2), send $(\mathsf{Eval}, \mathsf{sid}, P, P', x)$ to $\mathcal{F}_{\text{EKE-1r}}$ and return $\mathcal{F}_{\text{EKE-1r}}$'s response to $\mathcal{A}$.

Figure 16: Simulator $\mathcal{S}$ for the (O)EKE first-round protocol.

We now argue that $\mathcal{S}$ generates an ideal-world view that is indistinguishable from the real-world view. The proof goes by the following hybrid argument:

Hybrid 0: This is the real world. Recall that the passwords of P and P' are pw and pw', respectively; in the real world the flow of events is (for brevity, we omit sid in parties' messages and outputs below — same for later proofs):

- P sends ϕ to P' (intercepted by the man-in-the-middle adversary $\mathcal{A}$);
- $\mathcal{A}$ sends ϕ^* to P';
- P' outputs (B, K').

Furthermore, $\mathcal{A}$ can evaluate $F_{\phi^*}(x)$ for ϕ^*, x of its choice by querying $(\mathsf{Eval}, \mathsf{sid}, \phi^*, x)$ to $\mathcal{F}_{\text{POPF}}$ and receiving answer A.

Hybrid 1: In the case that $\phi^* = \phi \wedge \mathsf{pw} \neq \mathsf{pw}'$, P' outputs $B \leftarrow \mathcal{M}_2$ and $K' \leftarrow \{0,1\}^\kappa$.

In hybrid 0, $A' = F_{\phi^*}(\mathsf{pw}') = F_\phi(\mathsf{pw}')$ is uniformly random in $\mathcal{M}_1$, and $B = \mathsf{msg}_2(b, A')$ and $K' = \mathsf{key}_2(b, A')$. By Thm. 5.2, hybrids 0 and 1 are indistinguishable.

Hybrid 2: Consider the case that $\phi^* \neq \phi$ (i.e., $\mathcal{A}$ modifies the P-to-P' message), and on $(\mathsf{Extract}, \mathsf{sid}, \phi^*)$ from $\mathcal{F}_{\text{POPF}}$, $\mathcal{A}$ replies with $(\mathsf{Extract}, \mathsf{sid}, \mathsf{pw}^*, \alpha^*)$ where $\mathsf{pw}^* \neq \mathsf{pw}'$. (Intuitively, ϕ^* contains a wrong password guess for P'.) Then sample $B \leftarrow \mathcal{M}_2$ and $K' \leftarrow \{0,1\}^\kappa$.

In hybrid 1 $K' = \mathsf{key}_2(b, A')$, where $A' = F_{\phi^*}(\mathsf{pw}')$ is determined via the following process: $\mathcal{F}_{\text{POPF}}$ sends $(\mathsf{Extract}, \mathsf{sid}, \phi^*)$ and receives $\mathcal{A}$'s response $(\mathsf{Extract}, \mathsf{sid}, \mathsf{pw}^*, \alpha^*)$, and later defines A' as $R(\alpha^*, \mathsf{pw}')$ (i.e., $\mathcal{F}_{\text{POPF}}$ enters step 5B of Figure 8) which is uniformly random in $\mathcal{M}_1$. Thus, a reduction to Thm. 5.2, on input (A', B, K'), can randomly guess an R query and program the answer as A', and simulate other parts of hybrid 1 as usual and copy $\mathcal{Z}$'s output bit. The reduction loses a factor of $1/q$, where q is the number of R queries. By Thm. 5.2, hybrids 1 and 2 are indistinguishable.

Hybrid 3: When $\mathcal{A}$ queries $(\mathsf{Eval}, \mathsf{sid}, \phi, x)$ to $\mathcal{F}_{\text{POPF}}$, sample $a \leftarrow \mathcal{R}$ and answer with $\mathsf{msg}_1(a)$.

The only difference between hybrids 2 and 3 is that $F_\phi(x)$ (where $x \neq \mathsf{pw}$) is set to $A \leftarrow \mathcal{M}_1$ in hybrid 2 and $\mathsf{msg}_1(a)$ in hybrid 3. Note that a is not used anywhere else in the entire experiment, so a straightforward reduction to the first pseudorandomness of KA shows that hybrids 2 and 3 are indistinguishable.

Comparison between hybrid 3 and the ideal world. We now argue that hybrid 3 is identical to the ideal world, where the simulator $\mathcal{S}$ as defined in Figure 16 interacts with $\mathcal{F}_{\text{EKE-1r}}$. There are four cases:

- $\phi^* = \phi \wedge \mathsf{pw} = \mathsf{pw}'$: In hybrid 3 $A' = F_{\phi^*}(\mathsf{pw}') = F_\phi(\mathsf{pw}) = A$, so $B = \mathsf{msg}_2(b, A') = \mathsf{msg}_2(b, A)$ and $K' = \mathsf{key}_2(b, A') = \mathsf{key}_2(b, A)$. In the ideal world $\mathcal{S}$ sends $(\mathsf{Deliver}, \mathsf{sid}, P, P', \bot, \bot)$ to $\mathcal{F}_{\text{EKE-1r}}$, which enters step 1 and overwrites $\mathsf{pw}^* := \mathsf{pw}$ and $A^* := A$; then $\mathcal{F}_{\text{EKE-1r}}$ enters step 2 and $\mathsf{pw}^* = \mathsf{pw} = \mathsf{pw}'$, so $B = \mathsf{msg}_2(b, A^*) = \mathsf{msg}_2(b, A)$ and $K' = \mathsf{key}_2(b, A^*) = \mathsf{key}_2(b, A)$. So hybrid 3 and the ideal world are identical.

- $\phi^* = \phi \wedge \mathsf{pw} \neq \mathsf{pw}'$: In hybrid 3 $B \leftarrow \mathcal{M}_2$ and $K' \leftarrow \{0,1\}^\kappa$ (changed in hybrid 1). In the ideal world, $\mathcal{F}_{\text{EKE-1r}}$ enters step 1 and overwrites pw^* and A^* just as in the previous case, but then it enters step 2 and $\mathsf{pw}^* = \mathsf{pw} \neq \mathsf{pw}'$, so $B \leftarrow \mathcal{M}_2$ and $K' \leftarrow \{0,1\}^\kappa$. So hybrid 3 and the ideal world are identical.

- $\phi^* \neq \phi \wedge \mathsf{pw}^* = \mathsf{pw}'$: In hybrid 3 $A' = F_{\phi^*}(\mathsf{pw}')$, $B = \mathsf{msg}_2(b, A')$ and $K' = \mathsf{key}_2(b, A')$. In the ideal world $\mathcal{S}$ sends $(\mathsf{Deliver}, \mathsf{sid}, P, P', \mathsf{pw}^*, A^*)$ to $\mathcal{F}_{\text{EKE-1r}}$, where $A^* = F_{\phi^*}(\mathsf{pw}^*) = F_{\phi^*}(\mathsf{pw}')$ is the value programmed by $\mathcal{A}$. Then $\mathcal{F}_{\text{EKE-1r}}$ enters step 2 and $\mathsf{pw}^* = \mathsf{pw}'$, so $B = \mathsf{msg}_2(b, A^*) = \mathsf{msg}_2(b, A')$ and $K' = \mathsf{key}_2(b, A^*) = \mathsf{key}_2(b, A')$. So hybrid 3 and the ideal world are identical.

- $\phi^* \neq \phi \wedge \mathsf{pw}^* \neq \mathsf{pw}'$: In hybrid 3 $B \leftarrow \mathcal{M}_2$ and $K' \leftarrow \{0,1\}^\kappa$ (changed in hybrid 2). In the ideal world, $\mathcal{F}_{\text{EKE-1r}}$ enters step 2 just as in the previous case, but $\mathsf{pw}^* \neq \mathsf{pw}'$, so $B \leftarrow \mathcal{M}_2$ and $K' \leftarrow \{0,1\}^\kappa$. So hybrid 3 and the ideal world are identical.

Finally, $\mathcal{A}$ can evaluate $F_{\phi^*}(x)$ by querying $\mathcal{F}_{\text{POPF}}$. Both hybrid 3 and the ideal world simply run the code $\mathcal{F}_{\text{POPF}}$, except that when $\phi^* = \phi \wedge x \neq \mathsf{pw}$, both hybrid 3 and the ideal world answers with $\mathsf{msg}_1(a)$ where $a \leftarrow \mathcal{R}$. So hybrid 3 and the ideal world are identical.

We conclude that hybrid 3 and the ideal world are identical in all cases. This completes the proof. □

Remark 5.3. *Thm. 5.1 essentially covers the hybrids that are reused in the proofs for EKE and OEKE, namely if (1) $\phi^* = \phi \wedge \mathsf{pw} \neq \mathsf{pw}'$ (i.e., the adversary passes the first message without modification, and the two parties' passwords do not match), or (2) $\phi^* \neq \phi \wedge \mathsf{pw}^* \neq \mathsf{pw}'$ (i.e., the adversary modifies the first message, which contains a wrong password guess), it is safe to change everything on the P' side to uniformly random (and independent of the P side); furthermore, $F_\phi(x)$ (where $x \neq \mathsf{pw}$) can be set to $\mathsf{msg}_1(a)$ for $a \leftarrow R$, instead of a random $A \leftarrow \mathcal{M}_1$. Readers who would prefer to get rid of the $\mathcal{F}_{\text{PAKE}-1\text{r}}$ functionality and see a UC-security proof for (O)EKE as a monolith, can easily merge the hybrids above into the main proof, with the indistinguishability argument and the argument that the environment's view in the last hybrid matches the ideal world essentially unchanged.*

5.2 The EKE Protocol

Theorem 5.4. *Suppose that* KA *is a correct, secure, strongly pseudorandom, and pseudorandom non-malleable KA protocol. Then the EKE-PRF protocol (Figure 17) realizes $\mathcal{F}_{\text{PAKE}}$ in the $(\mathcal{F}_{\text{POPF}}, \mathcal{F}_{\text{EKE-1r}})$-hybrid world via a simulator $\mathcal{S}$ that emulates* TestOutput *in the same way as $\mathcal{F}_{\text{POPF}}$.*[17]

[17] Applying a PRF while producing the session key is necessary for the protocol to realize $\mathcal{F}_{\text{PAKE}}$; for an explanation, see Sect. A.4.

Parameters:

- EKE first round functionality $\mathcal{F}_{\text{EKE-1r}}$ (Figure 14) for a KA protocol KA.
- POPF functionality $\mathcal{F}_{\text{POPF}}$ (see Figure 8) with domain $\{0,1\}^*$ and range $\mathcal{M}_2$.
- A pseudorandom function $\mathsf{PRF}_K : \Phi \to \mathcal{K}$ with key space $\mathcal{K}$.

On input $(\mathsf{NewSession}, \mathsf{sid}, P, P', \mathsf{pw}, \text{“initiator”})$, party P does the following:

1. Sample and record $a \leftarrow \mathcal{R}$, and compute $A := \mathsf{msg}_1(a)$.
2. Send $(\mathsf{Program}, sid, P, P', \mathsf{pw}, A)$ to $\mathcal{F}_{\text{EKE-1r}}$.

On input $(\mathsf{NewSession}, \mathsf{sid}, P', P, \mathsf{pw}', \text{“respondent”})$, party P' sends $(\mathsf{SampleResp}, \mathsf{sid}, P, P', \mathsf{pw}')$ to $\mathcal{F}_{\text{EKE-1r}}$.

On message (sid, B, K') from $\mathcal{F}_{\text{EKE-1r}}$, party P' does the following:

3. Send $(\mathsf{Program}, \mathsf{sid}, \mathsf{pw}', B)$ to $\mathcal{F}_{\text{POPF}}$ and wait for response $(\mathsf{Program}, \mathsf{sid}, \phi')$.
4. Send (sid, ϕ') to P.
5. Output $(\mathsf{sid}, \mathsf{PRF}_{K'}(\phi'))$.

On message (sid, ϕ') from P', party P does the following:

6. Send $(\mathsf{Eval}, \mathsf{sid}, \phi', \mathsf{pw})$ to $\mathcal{F}_{\text{POPF}}$ and wait for response $(\mathsf{Eval}, \mathsf{sid}, B)$.
7. Retrieve a and compute $K := \mathsf{key}_1(a, B)$.
8. Output $(\mathsf{sid}, \mathsf{PRF}_K(\phi'))$.

Figure 17: EKE-PRF protocol, in the $(\mathcal{F}_{\text{POPF}}, \mathcal{F}_{\text{EKE-1r}})$-hybrid world.

Proof. The following lemma will be useful while analyzing the security of EKE-PRF (cf. Sect. 3.3):

Lemma 5.5. *If a KA protocol is secure and strongly pseudorandom then the following two distributions are indistinguishable:*

$a \leftarrow \mathcal{R}$	$a \leftarrow \mathcal{R}$
$A := \mathsf{msg}_1(a)$	$A := \mathsf{msg}_1(a)$
$B \leftarrow \mathcal{M}_2$	$B \leftarrow \mathcal{M}_2$
$K := \mathsf{key}_1(a, B)$	$K \leftarrow \mathcal{K}$
output (A, B, K)	output (A, B, K)

The simulator $\mathcal{S}$ is shown in Figure 18.

The proof goes by the following hybrid argument (see Table 6 for a summary):

Hybrid 0: This is the real world. Recall that the passwords of P and P' are pw and pw', respectively; the flow of events is:

- P tells $\mathcal{F}_{\text{EKE-1r}}$ to $\mathsf{Program}$ a function mapping $\mathsf{pw} \mapsto A$ and send it to P';
- It is intercepted by the man-in-the-middle adversary $\mathcal{A}$, who possibly modifies it to program $\mathsf{pw}^* \mapsto A^*$ instead (intuitively pw^* is $\mathcal{A}$'s guess for the password of P', pw');
- $\mathcal{A}$ tells $\mathcal{F}_{\text{EKE-1r}}$ to $\mathsf{Deliver}$ the function to P';
- P' tells $\mathcal{F}_{\text{EKE-1r}}$ to $\mathsf{SampleResp}$, which generates B, K';
- P' tells $\mathcal{F}_{\text{POPF}}$ to program a function $F_{\phi'}$ mapping $\mathsf{pw}' \mapsto B$;
- P' outputs $\mathsf{PRF}_{K'}(\phi')$ and sends ϕ' to P;

Let $T = \{\}$ be the record of honest POPF evaluations as in $\mathcal{F}_{\text{POPF}}$.
Maintain a *local* random function $h : \{0,1\}^* \to \mathcal{R}$. Whenever $h(x)$ is referred to but undefined, sample $a \leftarrow \mathcal{R}$ and define $h(x) := a$. We stress that h is local to $\mathcal{S}$ and is unavailable to other parties.

On $(\mathsf{NewSession}, \mathsf{sid}, P, P', \text{“initiator”})$ from $\mathcal{F}_{\text{PAKE}}$:

1. Send $(\mathsf{Program}, \mathsf{sid}, P, P')$ from $\mathcal{F}_{\text{EKE-1r}}$ to $\mathcal{A}$.

On $(\mathsf{NewSession}, \mathsf{sid}, P', P, \text{“respondent”})$ from $\mathcal{F}_{\text{PAKE}}$:

2. Send $(\mathsf{SampleResp}, \mathsf{sid}, P', P)$ from $\mathcal{F}_{\text{EKE-1r}}$ to $\mathcal{A}$.

On $(\mathsf{Eval}, \mathsf{sid}, P, P', x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{EKE-1r}}$:

3. Set $a := h(x)$, compute $A := \mathsf{msg}_1(a)$, and return A to $\mathcal{A}$.

On $(\mathsf{Deliver}, \mathsf{sid}, P, P', \mathsf{pw}^*, A^*)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{EKE-1r}}$:

4. Send $(\mathsf{Program}, \mathsf{sid})$ from $\mathcal{F}_{\text{POPF}}$ to $\mathcal{A}$ and wait until $\mathcal{A}$ responds with $(\mathsf{Program}, \mathsf{sid}, \phi')$ to $\mathcal{F}_{\text{POPF}}$ such that there is no entry $(\phi', \cdot, \cdot) \in T$.
5. Send (sid, ϕ') from P' to P.
6. If $\mathsf{pw}^* = \perp$, send $(\mathsf{NewKey}, \mathsf{sid}, P', 0^\kappa)$ to $\mathcal{F}_{\text{PAKE}}$.
7. If $\mathsf{pw}^* \neq \perp$ do:
   (1) Send $(\mathsf{TestPwd}, \mathsf{sid}, P', \mathsf{pw}^*)$ to $\mathcal{F}_{\text{PAKE}}$.
   (2) Sample $b \leftarrow \mathcal{R}$ and compute $(K')^* := \mathsf{key}_2(b, A^*)$.
   (3) Send $(\mathsf{NewKey}, \mathsf{sid}, P', \mathsf{PRF}_{(K')^*}(\phi'))$ to $\mathcal{F}_{\text{PAKE}}$.

On $(\mathsf{sid}, (\phi')^*)$ from $\mathcal{A}$ to P:

8. If either (1) $\mathsf{pw}^* = \perp \wedge (\phi')^* = \phi'$ or (2) $\mathsf{pw}^* \neq \perp \wedge (\phi')^* = \phi'$ and the password guess on $\mathsf{Deliver}$ was incorrect, send $(\mathsf{NewKey}, \mathsf{sid}, P, 0^\kappa)$ to $\mathcal{F}_{\text{PAKE}}$.
9. If $\mathsf{pw}^* \neq \perp \wedge (\phi')^* = \phi'$ and the password guess on $\mathsf{Deliver}$ was correct:
   (1) Send $(\mathsf{TestPwd}, \mathsf{sid}, P, \mathsf{pw}^*)$ to $\mathcal{F}_{\text{PAKE}}$.
   (2) Set $a := h(\mathsf{pw}^*)$, compute $K^* = \mathsf{key}_1(a, \mathsf{msg}_2(b, A^*))$ and $SK^* = \mathsf{PRF}_{K^*}(\phi')$, and send $(\mathsf{NewKey}, \mathsf{sid}, P, SK^*)$ to $\mathcal{F}_{\text{PAKE}}$.
10. Otherwise (i.e., if $(\phi')^* \neq \phi'$ or ϕ' is undefined because no $\mathsf{Deliver}$ message has been sent):
   (1) Send $(\mathsf{Extract}, \mathsf{sid}, (\phi')^*)$ from $\mathcal{F}_{\text{POPF}}$ to $\mathcal{A}$.
   (2) On $(\mathsf{Extract}, \mathsf{sid}, (\mathsf{pw}')^*, \alpha^*)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{POPF}}$, send $(\mathsf{Eval}, \mathsf{sid}, (\phi')^*, (\mathsf{pw}')^*)$ from $\mathcal{F}_{\text{POPF}}$ to $\mathcal{A}$.
   (3) On $(\mathsf{Eval}, \mathsf{sid}, (B')^*)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{POPF}}$, set $a := h((\mathsf{pw}')^*)$, send $(\mathsf{TestPwd}, \mathsf{sid}, P, (\mathsf{pw}')^*)$, compute $K^* := \mathsf{key}_1(a, (B')^*)$, and send $(\mathsf{NewKey}, \mathsf{sid}, P, \mathsf{PRF}_{K^*}((\phi')^*))$ to $\mathcal{F}_{\text{PAKE}}$.

Simulation of $\mathcal{F}_{\text{POPF}}$: run the code of $\mathcal{F}_{\text{POPF}}$ as in Figure 8, except that on $(\mathsf{Eval}, \mathsf{sid}, \phi', \mathsf{pw}^*)$, if the password guess on $\mathsf{Deliver}$ was correct return $B = \mathsf{msg}_2(b, A^*)$.

Figure 18: Simulator $\mathcal{S}$ for the EKE-PRF protocol.

#	case addressed	change	property used
1	$\mathsf{pw} = \mathsf{pw}'$ and $\mathcal{A}$ eavesdrops or re-encrypts	$K := K'$	KA correctness
2	$\mathsf{pw} = \mathsf{pw}' \wedge \mathsf{pw}^* = \perp$	B, K' \$	KA pseudorandom non-malleability
3	$(\phi')^*$ contains a wrong password guess or $\mathsf{pw} \neq \mathsf{pw}' \wedge (\phi')^* = \phi'$	K \$	Thm. 5.5 (KA security / strong pseudorandomness)
4	$\mathsf{pw}^* \neq \perp \wedge \mathsf{pw} = \mathsf{pw}' \neq \mathsf{pw}^* \wedge (\phi')^* = \phi'$	K \$	Thm. 5.5 (KA security / strong pseudorandomness)
5	$\mathcal{A}$ eavesdrops	$SK := SK'$	none
6	all cases except $\mathsf{pw}^* = \mathsf{pw}'$	$\mathsf{PRF}_{K'}$ \$	PRF property
7	cases in hybrids 3 and 4	PRF_K \$	PRF property
8	$\mathcal{A}$ re-encrypts	$SK := \mathsf{PRF}_K((\phi')^*)$	PRF property
9		$K := \mathsf{key}_1(a, B)$	Thm. 5.5 (KA security / strong pseudorandomness)

Table 6: Summary of hybrids for EKE-PRF security. "\$" denotes "chosen at random from respective range". SK, SK' denote the outputs of P, P' respectively.

- It is again intercepted by $\mathcal{A}$, who possibly modifies it to $(\phi')^*$ representing a function $F_{(\phi')^*}$ programmed on $(\mathsf{pw}')^* \mapsto (B')^*$ instead (intuitively $(\mathsf{pw}')^*$ is $\mathcal{A}$'s guess for the password of P, pw);
- $\mathcal{A}$ sends $(\phi')^*$ to P;
- P computes the key exchange message $B^* = F_{(\phi')^*}(\mathsf{pw})$ and uses it to output $\mathsf{PRF}_K((\phi')^*)$.

Note that there are three B values here: $B = F_\phi(\mathsf{pw}')$ is computed by P', $(B')^* = F_{(\phi')^*}((\mathsf{pw}')^*)$ is computed by $\mathcal{A}$, and $B^* = F_{(\phi')^*}(\mathsf{pw})$ is computed by P.

Throughout the proof, when $(\phi')^* \neq \phi'$ (i.e., $\mathcal{A}$ modifies the P'-to-P message), and on $(\mathsf{Extract}, \mathsf{sid}, (\phi')^*)$ from $\mathcal{F}_{\text{POPF}}$, $\mathcal{A}$ replies with $(\mathsf{Extract}, \mathsf{sid}, (\mathsf{pw}')^*, \alpha^*)$, we call this case "$(\phi')^*$ contains the correct password guess" if $(\mathsf{pw}')^* = \mathsf{pw}$, and "$(\phi')^*$ contains a wrong password guess" if $(\mathsf{pw}')^* \neq \mathsf{pw}$. (Note that we view $(\phi')^* = \phi'$ as $(\phi')^*$ contains no password guess, neither correct nor wrong.)

Hybrid 1: In the case that $\mathsf{pw} = \mathsf{pw}' \wedge \mathsf{pw}^* = \perp \wedge B^* = B$ and $(\phi')^*$ does not contain a wrong password guess, P sets $K := K'$ instead of $K := \mathsf{key}_1(a, B^*)$. We have two sub-cases here:

- $\mathsf{pw} = \mathsf{pw}' \wedge \mathsf{pw}^* = \perp \wedge (\phi')^* = \phi'$ (which implies $B^* = B$ as $B^* = F_{(\phi')^*}(\mathsf{pw}) = F_{\phi'}(\mathsf{pw}') = B$). Intuitively this means that $\mathcal{A}$ merely eavesdrops.
- $\mathsf{pw} = \mathsf{pw}' \wedge \mathsf{pw}^* = \perp \wedge (\phi')^* \neq \phi' \wedge B^* = B$. Intuitively this means that $\mathcal{A}$ re-encrypts.

By the syntax of $\mathcal{F}_{\text{EKE-1r}}$, $B = \mathsf{msg}_2(b, A)$ and $K' = \mathsf{key}_2(b, A)$, where $A = \mathsf{msg}_1(a)$ according to protocol description (step 2). By the correctness of KA,

$$\mathsf{key}_1(a, B^*) = \mathsf{key}_1(a, B) = \mathsf{key}_1(a, \mathsf{msg}_2(b, \mathsf{msg}_1(a))) = \mathsf{key}_2(b, \mathsf{msg}_1(a)) = K'$$

with overwhelming probability, so hybrid 1 is (statistically) indistinguishable from hybrid 0.

Hybrid 2: In the case that $\mathsf{pw} = \mathsf{pw}' \wedge \mathsf{pw}^* = \perp$, P' samples $B \leftarrow \mathcal{M}_2$ and $K' \leftarrow \mathcal{K}$.

The difference between hybrids 1 and 2 is that in hybrid 1, $B = \mathsf{msg}_2(b, A)$ and $K' = \mathsf{key}_2(b, A)$, whereas in hybrid 2, $B \leftarrow \mathcal{M}_2$ and $K' \leftarrow \mathcal{K}$. We construct a reduction $\mathcal{R}$ to the pseudorandom non-malleability of KA: $\mathcal{R}$, given (A, B, K'), can simulate the experiment up to when P' outputs — which includes $\mathcal{A}$'s access to $\mathcal{F}_{\text{EKE-1r}}$, as well as B and K' — without knowing a or b. (Note that in $\mathcal{F}_{\text{EKE-1r}}$, b is used only when computing B and K'.) $\mathcal{R}$ simulates the second message ϕ' as expected, by programming $F_{\phi'}(\mathsf{pw}') = B$ into the POPF. When $\mathcal{A}$ sends $(\phi')^*$, $\mathcal{R}$ checks whether $B^* = F_{(\phi')^*}(\mathsf{pw})$ equals $B = F_{\phi'}(\mathsf{pw}') = F_{\phi'}(\mathsf{pw})$, and proceeds as follows:

- If $B^* \neq B$, $\mathcal{R}$ outputs B^* to the pseudorandom non-malleability challenger and receives K, outputting $\mathsf{PRF}_K((\phi')^*)$ to $\mathcal{Z}$.
- If $B^* = B$, the pseudorandom non-malleability experiment prohibits $\mathcal{R}$ from outputting B^*. Instead, $\mathcal{R}$ checks whether $(\phi')^*$ contains a wrong password guess by checking that $(\phi')^* \neq \phi'$ and sending the appropriate $\mathsf{Extract}$ query from $\mathcal{F}_{\text{POPF}}$ to $\mathcal{A}$. If $(\phi')^*$ does contain a wrong password guess, $\mathcal{R}$ aborts. If not, $\mathcal{R}$ picks some other element of $\mathcal{M}_2$ to output to the pseudorandom non-malleability challenger (just to finish the experiment) and outputs $\mathsf{PRF}_{K'}((\phi')^*)$ to $\mathcal{Z}$ (note that in hybrid 1 P was changed to use $K = K'$ in this case, so the correct value for P is still output to $\mathcal{Z}$).

Finally, $\mathcal{R}$ copies $\mathcal{Z}$'s output bit.

We can see that if $B = \mathsf{msg}_2(b, A)$ and $K' = \mathsf{key}_2(b, A)$ then $\mathcal{R}$ simulates hybrid 1, and if $B \leftarrow \mathcal{M}_2$ and $K' \leftarrow \mathcal{K}$ then $\mathcal{R}$ simulates hybrid 2, as long as we don't have that $B^* = B$ and $(\phi')^*$ contains a wrong password guess. If that latter case occurs, $F_{(\phi')^*}$ is programmed on some $(\mathsf{pw}')^* \neq \mathsf{pw}$, therefore $B^* = F_{(\phi')^*}(\mathsf{pw}) = R(\alpha^*, \mathsf{pw})$ for some α^* (i.e., $\mathcal{F}_{\text{POPF}}$ enters step 5B of Figure 8), i.e., B^* is a uniformly random element of $\mathcal{M}_2$ that is independent of B. So $B = B^*$ happens with probability $1/|\mathcal{M}_2|$, which is negligible when KA is a correct and secure KA protocol.[18] We conclude that $\mathcal{R}$'s distinguishing advantage (in the pseudorandom non-malleability experiment for KA) is negligibly smaller than $\mathcal{Z}$'s distinguishing advantage between hybrids 1 and 2, so hybrids 1 and 2 are indistinguishable.

Hybrid 3: In the case that $(\phi')^*$ contains a wrong password guess or $\mathsf{pw} \neq \mathsf{pw}' \wedge (\phi')^* = \phi'$, P samples $K \leftarrow \mathcal{K}$.

In hybrid 2,

- If $(\phi')^*$ contains a wrong password guess, then as we have just argued under hybrid 2, $B^* = R(\alpha^*, \mathsf{pw})$.
- If $\mathsf{pw} \neq \mathsf{pw}' \wedge (\phi')^* = \phi'$ we have $B^* = F_{(\phi')^*}(\mathsf{pw}) = F_{\phi'}(\mathsf{pw})$. Since ϕ' was programmed on $\mathsf{pw}' \neq \mathsf{pw}$, B^* is again defined as $R(\alpha^*, \mathsf{pw})$.

Either way, $B^* = R(\alpha^*, \mathsf{pw})$ is a uniformly random element of $\mathcal{M}_2$. This means that in hybrid 2 $K = \mathsf{key}_1(a, B^*)$ for $A = \mathsf{msg}_1(a)$ and $B^* \leftarrow \mathcal{M}_2$, whereas in hybrid 3 $K \leftarrow \mathcal{K}$. Thus, a reduction to Thm. 5.5, on input (A, B^*, K), can randomly guess an R query and program the answer as B^*, and simulate the other parts of hybrid 2 as usual (using A and K for P), copying $\mathcal{Z}$'s output bit.

[18] An adversary $\mathcal{A}$ may break the security of a correct KA with probability negligibly close to $1/|\mathcal{M}_2|$ as follows. Given $A = \mathsf{msg}_1(a), B = \mathsf{msg}_2(b, A)$ with $a, b \leftarrow \mathcal{R}$, $\mathcal{A}$ samples $b' \leftarrow \mathcal{R}$ and computes $B' = \mathsf{msg}_2(b', A)$. If $B = B'$ then with overwhelming probability $\mathsf{key}_1(a, B) = \mathsf{key}_1(a, B') = \mathsf{key}_2(b', A)$ by correctness, therefore $\mathcal{A}$ can distinguish $\mathsf{key}_1(a, B)$ from random. It is well-known that if $\mathbb{H}_\alpha(X)$ is the Rényi entropy of a random variable X one has $\mathbb{H}_\alpha(X)$ non-increasing in α. In particular, $\mathbb{H}_2(X) \leq \mathbb{H}_0(X)$: setting $X = [\mathsf{msg}_2(b, A) : b \leftarrow \mathcal{R}]$ gives the inequality $\Pr[B = B'] \geq 1/|\mathcal{M}_2|$.

The reduction loses a factor of $1/q$, where q is the number of R queries. Since KA is secure and strongly pseudorandom, by Thm. 5.5, hybrids 2 and 3 are indistinguishable.

Hybrid 4: In the case that $\mathsf{pw}^* \neq \bot \wedge \mathsf{pw} = \mathsf{pw}' \neq \mathsf{pw}^* \wedge (\phi')^* = \phi'$, P samples $K \leftarrow \mathcal{K}$.

The change in this hybrid is the same as in hybrid 3 (P samples $K \leftarrow \mathcal{K}$ instead of $K := \mathsf{key}_1(a, B^*)$), but the argument for indistinguishability is different. In this case $B^* = F_{(\phi')^*}(\mathsf{pw}) = F_{\phi'}(\mathsf{pw}') = B$, so $\mathcal{F}_{\text{POPF}}$ does not enter step 5B and use R to compute B^*. However, by the syntax of $\mathcal{F}_{\text{EKE-1r}}$, when $\mathsf{pw}^* \neq \mathsf{pw}'$ we still have $B \leftarrow \mathcal{M}_2$. Therefore the reduction to Thm. 5.5 under hybrid 3 still works, except that here it does not need to guess an R query and lose a factor of q.

Summary of hybrid 4. Let us pause a bit and summarize the changes we have made so far. On the P' side, we have changed B and K' in some cases; on the P side, we have changed K in some cases. See Tables 7 and 8 for a summary.

#	case	B, K' definitions	according to...
1	$\mathsf{pw} = \mathsf{pw}' \wedge \mathsf{pw}^* = \bot$	B, K' \$	hybrid 2
2	$\mathsf{pw} \neq \mathsf{pw}' \wedge \mathsf{pw}^* = \bot$	B, K' \$	$\mathcal{F}_{\text{EKE-1r}}$ syntax
3	$\mathsf{pw}^* = \mathsf{pw}$	$B = \mathsf{msg}_2(b, A^*)$ $K' = \mathsf{key}_2(b, A^*)$	$\mathcal{F}_{\text{EKE-1r}}$ syntax
4	$\mathsf{pw}^* \neq \bot \wedge \mathsf{pw}' \neq \mathsf{pw}$	B, K' \$	$\mathcal{F}_{\text{EKE-1r}}$ syntax

Table 7: Definitions of B and K' in hybrid 4

#	case	K definition	according to...
1	$\mathsf{pw} = \mathsf{pw}' \wedge \mathsf{pw}^* = \bot \wedge (\phi')^* = \phi'$ (eavesdropping case)	$K = K'$	hybrid 1
2	$\mathsf{pw}^* = \mathsf{pw} = \mathsf{pw}' \wedge (\phi')^* = \phi'$	$K = \mathsf{key}_1(a, B^*)$	protocol description
3	$\mathsf{pw}^* \neq \bot \wedge \mathsf{pw} = \mathsf{pw}' \neq \mathsf{pw}^* \wedge (\phi')^* = \phi'$	K \$	hybrid 4
4	$\mathsf{pw} \neq \mathsf{pw}' \wedge (\phi')^* = \phi'$	K \$	hybrid 3
5	$\mathsf{pw} = \mathsf{pw}' \wedge \mathsf{pw}^* = \bot \wedge B^* = B \wedge$ $(\phi')^*$ contains the correct password guess (re-encryption case)	$K = K'$	hybrid 1
6	all other cases where $(\phi')^*$ contains the correct password guess	$K = \mathsf{key}_1(a, B^*)$	protocol description
7	$(\phi')^*$ contains a wrong password guess	K \$	hybrid 3

Table 8: Definition of K in hybrid 4. Both eavesdropping and re-encryption cases assume the two parties' passwords match

Changing session keys. Now we consider the outputs of P and P' under a variety of cases (note that up until now we have changed only KA keys K, K', not the PAKE session keys that P and P' output). Let SK be the session key of P and SK' be the session key of P'.

The following hybrids 5–7 are immediate (hybrid 5 is purely conceptual, while hybrids 6 and 7 uses the fact that PRF is a PRF):

Hybrid 5: In case 1 of Table 8, set $SK := SK'$.

Hybrid 6: In cases 1, 2 and 4 of Table 7, replace $\mathsf{PRF}_{K'}$ with a random function G'.

Hybrid 7: In cases 3, 4 and 7 of Table 8, replace PRF_K with a random function G.

The following hybrids 8 and 9 deal with case 5 (re-encryption).

Hybrid 8: In case 5 of Table 8, set $SK := \mathsf{PRF}_K((\phi')^*)$.

This case is covered by hybrid 1, which sets $K := K'$ (and thus $SK = \mathsf{PRF}_K((\phi')^*) = \mathsf{PRF}_{K'}((\phi')^*)$); then by hybrid 6, which changes (SK, SK') from $(\mathsf{PRF}_{K'}((\phi')^*), \mathsf{PRF}_{K'}(\phi'))$ to $(G'((\phi')^*), G'(\phi'))$. Since $(\phi')^* \neq \phi'$, SK and SK' are independent of each other, so changing SK back to $\mathsf{PRF}_K((\phi')^*)$ generates an indistinguishable view.

At this point, in the cases when $K = K'$ (eavesdropping and re-encryption), P does not use a random function to output (hybrids 5 and 8 define SK specifically in these cases). Therefore P and P' don't ever use the same G or G' to output: P' outputs with G' and P outputs with G, with G, G' independently drawn.

Hybrid 9: In case 5 of Table 8, P sets $K := \mathsf{key}_1(a, B)$.

Note that in hybrid 8 $K = K'$ is only used while computing SK (in particular, the P' side has been changed to all random and does not use K'). In hybrid 8 $K \leftarrow \mathcal{K}$, whereas in hybrid 9 $K = \mathsf{key}_1(a, B)$ for $A = \mathsf{msg}_1(a)$ and $B \leftarrow \mathcal{M}_2$. Thus, a reduction to Thm. 5.5 (that uses its challenge K to compute SK for P) shows that hybrids 8 and 9 are indistinguishable.

Comparison between hybrid 9 and the ideal world. We now argue that hybrid 9 is identical to the ideal world, where the simulator $\mathcal{S}$ as defined in Figure 18 interacts with $\mathcal{F}_{\text{PAKE}}$. We first consider the P' side output SK'. We refer to cases in Table 7:

- In cases 1 and 2, in hybrid 9 SK' is uniformly random. In the ideal world, $\mathcal{S}$ enters step 6, where it sends NewKey without TestPwd, so the P' session is fresh and $\mathcal{F}_{\text{PAKE}}$ samples a uniformly random SK' for P'.
- In case 3, in hybrid 9 $SK' = \mathsf{PRF}_{K'}(\phi')$ where $K' = \mathsf{key}_2(b, A^*)$. In the ideal world, $\mathcal{S}$ enters step 7, where it sends TestPwd resulting in "correct guess" and then NewKey where the session key is $\mathsf{PRF}_{K'}(\phi')$ where $K' = \mathsf{key}_2(b, A^*)$, so the P' session is compromised and $\mathcal{F}_{\text{PAKE}}$ sets $SK' := \mathsf{PRF}_{K'}(\phi')$ for P'.
- In case 4, in hybrid 9 SK' is uniformly random. In the ideal world, $\mathcal{S}$ enters step 7, where it sends TestPwd resulting in "wrong guess" and then NewKey, so the P' session is interrupted and $\mathcal{F}_{\text{PAKE}}$ samples a uniformly random SK' for P'. (Note that in this case the session key chosen by $\mathcal{S}$ in NewKey does not matter.)

We conclude that for SK', hybrid 9 and the ideal world are identical in all cases.

Next, we consider the P side output SK. We refer to cases in Table 8:

- In case 1 (eavesdropping), in hybrid 9 $SK = SK'$. In the ideal world, $\mathcal{S}$ enters step 8(1), where it sends NewKey without TestPwd, so the P session is fresh. Since the P' session was also fresh when P' output and $\mathsf{pw} = \mathsf{pw}'$, $\mathcal{F}_{\text{PAKE}}$ sets $SK := SK'$ for P.
- In case 2, in hybrid 9 $SK = \mathsf{PRF}_K((\phi')^*) = \mathsf{PRF}_K(\phi')$ where $K = \mathsf{key}_1(a, B^*)$. Note that $B^* = F_{(\phi')^*}(\mathsf{pw}) = F_{\phi'}(\mathsf{pw}') = B = \mathsf{msg}_2(b, A^*)$. In the ideal world, $\mathcal{S}$ enters step 9, where it sends TestPwd resulting in "correct guess" and then NewKey where the session key is $\mathsf{PRF}_{K^*}(\phi')$ and $K^* = \mathsf{key}_1(a, \mathsf{msg}_2(b, A^*))$. Then the P session is compromised and $\mathcal{F}_{\text{PAKE}}$ sets $SK := \mathsf{PRF}_{K^*}(\phi')$ for P. We can see that in both hybrid 9 and the ideal world, $SK = \mathsf{PRF}_{\mathsf{key}_1(a', \mathsf{msg}_2(b, A^*))}(\phi')$.

- In case 3, in hybrid 9 SK is uniformly random. In the ideal world, $\mathcal{S}$ enters step 8(2), where it sends NewKey without TestPwd, so the P session is fresh. Since the P' session was interrupted by a wrong password guess when P' output, $\mathcal{F}_{\text{PAKE}}$ samples a uniformly random SK for P.

- In case 4, in hybrid 9 SK is uniformly random. In the ideal world there are several sub-cases: (i) if $\mathsf{pw}^* = \perp$, then $\mathcal{S}$ enters step 8(1). This is identical to case 1, except that the P' session was fresh when P' output but $\mathsf{pw} \neq \mathsf{pw}'$, so $\mathcal{F}_{\text{PAKE}}$ samples a uniformly random SK for P (independent of SK'); (ii) if $\mathsf{pw}^* = \mathsf{pw}' \neq \mathsf{pw}'$, then $\mathcal{S}$ enters step 9, where it sends TestPwd resulting in "wrong guess" and then NewKey, so the P session is interrupted and $\mathcal{F}_{\text{PAKE}}$ samples a uniformly random SK for P; (iii) if $\mathsf{pw}^* \neq \perp \wedge \mathsf{pw}^* \neq \mathsf{pw}'$, then $\mathcal{S}$ enters step 8(2), so SK is uniformly random by the same argument as in case 3.

- Case 5 (re-encryption) is the most complicated one because it is changed five times: in hybrids 1, 2, 6, 8 and 9, where both K and SK are changed back and forth between the "real" value and uniformly random. Eventually in hybrid 9 $SK = \mathsf{PRF}_K((\phi')^*)$ where $K = \mathsf{key}_1(a, B)$. In the ideal world, $\mathcal{S}$ enters step 10, where it sends TestPwd resulting in "correct guess" and then NewKey where the session key is $\mathsf{PRF}_{K^*}((\phi')^*)$ and $K^* = \mathsf{key}_1(a, (B')^*)$. Note that in this case $(B')^* = B^* = B$. Then the P session is compromised and $\mathcal{F}_{\text{PAKE}}$ sets $SK := \mathsf{PRF}_{K^*}((\phi')^*)$ for P. We can see that in both hybrid 9 and the ideal world, $SK = \mathsf{PRF}_{\mathsf{key}_1(a,B)}((\phi')^*)$.

- Case 6 is similar to case 5, except that B^* might not equal B. In hybrid 9 $SK = \mathsf{PRF}_K((\phi')^*)$ where $K = \mathsf{key}_1(a, B^*)$. Since $(\phi')^*$ contains a correct password guess, $(\mathsf{pw}')^* = \mathsf{pw}$, so $B^* = F_{(\phi')}(\mathsf{pw}) = F_{(\phi')}((\mathsf{pw}')^*) = (B')^*$. The ideal world is identical to case 5: $\mathcal{F}_{\text{PAKE}}$ sets $SK := \mathsf{PRF}_{K^*}((\phi')^*)$ for P, where $K^* = \mathsf{key}_1(a, (B')^*)$. We can see that in both hybrid 9 and the ideal world, $SK = \mathsf{PRF}_{\mathsf{key}_1(a,(B')^*)}((\phi')^*)$.

- In case 7, in hybrid 9 SK is uniformly random. In the ideal world, $\mathcal{S}$ enters step 10, where it sends TestPwd resulting in "wrong guess" and then NewKey, so the P' session is interrupted and $\mathcal{F}_{\text{PAKE}}$ samples a uniformly random SK for P.

We conclude that for SK, hybrid 9 and the ideal world are identical in all cases.

We also must argue that the simulation of $\mathcal{F}_{\text{POPF}}$ is indistinguishable between hybrid 9 and the ideal world. The outputs of the POPF are independent of the rest of the simulation, except for the programmed ones. $\mathcal{A}$ already knows any outputs it programs, but P' also programs an output: $F_{\phi'}(\mathsf{pw}') = B$. Therefore if $\mathcal{A}$ ever guesses pw' correctly it will know B, and so we must adjust the simulation to output B in this case — which is exactly what our simulator does. Note that it suffices to only simulate this *after* the password guess on Deliver, as $\mathcal{A}$ must provide a ϕ' that has never been evaluated previously; therefore $\mathcal{A}$ cannot evaluate $F_{\phi'}(\mathsf{pw}')$ before the simulator can program that output into $\mathcal{F}_{\text{POPF}}$.

Finally, the careful reader will note that our argument that hybrid 9 and the ideal world are identical assumes that $\mathcal{A}$ delivers all messages (possibly after modification). If $\mathcal{A}$ drops the message to one party or the other, that does not affect the indistinguishability of the simulation: since that party no longer outputs, we no longer need to simulate them. For example, if $\mathcal{A}$ drops the first message and supplies its own second message to P (as if P' does not exist), the simulator can proceed as if $\mathcal{A}$ had modified the second (P'-to-P) message, since the simulation only depends on what $\mathcal{A}$ sends to P. Even if $\mathcal{A}$ chooses to send messages out of order this still does not affect simulation. For example, suppose when $\mathcal{A}$ receives Deliver, it first sends $(\phi')^*$ to P so P outputs, and then sends the Deliver message to $\mathcal{F}_{\text{EKE-1r}}$. Since $\mathcal{A}$ does not see the output of P after it sends $(\phi')^*$, the Deliver message is independent of any information gathered from sending $(\phi')^*$. Therefore without loss of generality we may assume that these messages are sent in the opposite

Parameters:
- EKE first round functionality $\mathcal{F}_{\text{EKE-1r}}$ (see Figure 14) for a KA protocol KA.

On input $(\mathsf{NewSession}, \mathsf{sid}, P, P', \mathsf{pw}, \text{“initiator”})$, party P does the following:
1. Sample and record $a \leftarrow \mathcal{R}$, and compute $A := \mathsf{msg}_1(a)$.
2. Send $(\mathsf{Program}, sid, P, P', \mathsf{pw}, A)$ to $\mathcal{F}_{\text{EKE-1r}}$.

On input $(\mathsf{NewSession}, \mathsf{sid}, P', P, \mathsf{pw}', \text{“respondent”})$, party P' sends $(\mathsf{SampleResp}, sid, P, P', \mathsf{pw}')$ to $\mathcal{F}_{\text{EKE-1r}}$.

On message $(\mathsf{sid}, B, K' \parallel \tau')$ from $\mathcal{F}_{\text{EKE-1r}}$, party P' does the following:
3. Send (sid, B, τ') to P.
4. Output (sid, K').

On message (sid, B, τ') from P', party P does the following:
5. Retrieve a and compute $K \parallel \tau := \mathsf{key}_1(a, B)$.
6. Check if $\tau = \tau'$. If so, output (sid, K). Otherwise output $(\mathsf{sid}, K_\$)$ where $K_\$ \leftarrow \{0,1\}^\kappa$.

Figure 19: OEKE protocol, in the $\mathcal{F}_{\text{EKE-1r}}$-hybrid world.

order, reducing $\mathcal{A}$ to an adversary that delivers all messages (possibly after modification).

We conclude that hybrid 9 and the ideal world are identical in all cases. This completes the proof. □

The theorem immediately implies the following: let KA be a correct, secure, strongly pseudorandom, and pseudorandom non-malleable KA protocol. Beginning with the protocol in Figure 17, replace $\mathcal{F}_{\text{EKE-1r}}$ with the protocol in Figure 15, and $\mathcal{F}_{\text{POPF}}$ with the protocol in Figure 9. Then by Thms. 4.3, 5.1 and 5.4 the resulting protocol realizes $\mathcal{F}_{\text{PAKE}}$ in the ROM.

Remark 5.6. *Our security analysis of EKE-PRF critically relies on the fact that P is the initiator and P' is the responder, i.e., the message and session key of P' depend on the message of P. If the underlying KA is 1-simultaneous round, and there is no mechanism ensuring that P' always outputs first, then whether ϕ or ϕ' is the first message depends on the adversary who decides which one to deliver first. So in this case both messages need to be included in the PRF, i.e., the session key should be $\mathsf{PRF}_K(\phi, \phi')$.*

5.3 The OEKE Protocol

Theorem 5.7. *Suppose that KA is a correct, pseudorandom non-malleable, and collision resistant KA protocol, whose key space $\mathcal{K} = \{0,1\}^{3\kappa}$. Then the OEKE protocol (Figure 19) realizes $\mathcal{F}_{\text{PAKE}}$ in the $\mathcal{F}_{\text{EKE-1r}}$-hybrid world.*[19]

Proof. The simulator $\mathcal{S}$ is shown in Figure 20. In this proof we use the following convention: for a 3κ-bit string s, $s[1]$ denotes its first κ bits (the K part), and $s[2]$ denotes its last 2κ bits (the τ part).

The proof goes by the following hybrid argument (see Table 9 for a summary):

Hybrid 0: This is the real world. Recall that the passwords of P and P' are pw and pw', respectively; the flow of events is:

- P tells $\mathcal{F}_{\text{EKE-1r}}$ to $\mathsf{Program}$ a function mapping $\mathsf{pw} \mapsto A$ and send it to P';

[19] Our protocol realizing $\mathcal{F}_{\text{EKE-1r}}$ (Figure 15) additionally requires KA to be secure and pseudorandom.

Initialize $T := \{\}$ as the set of $\mathcal{F}_{\text{EKE-1r}}$ evaluations.
On $(\mathsf{NewSession}, \mathsf{sid}, P, P', \text{“initiator”})$ from $\mathcal{F}_{\text{PAKE}}$:

1. Send $(\mathsf{Program}, \mathsf{sid}, P, P')$ from $\mathcal{F}_{\text{EKE-1r}}$ to $\mathcal{A}$.

On $(\mathsf{NewSession}, \mathsf{sid}, P', P, \text{“respondent”})$ from $\mathcal{F}_{\text{PAKE}}$:

2. Send $(\mathsf{SampleResp}, \mathsf{sid}, P', P)$ from $\mathcal{F}_{\text{EKE-1r}}$ to $\mathcal{A}$.

On $(\mathsf{Eval}, \mathsf{sid}, P, P', x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{EKE-1r}}$:

3. If there exists $(x, A, a) \in T$, return A to $\mathcal{A}$.
4. Otherwise, sample $a \leftarrow \mathcal{R}$ and $A := \mathsf{msg}_1(a)$.
5. Add (x, A, a) to T, and return A to $\mathcal{A}$.

On $(\mathsf{Deliver}, \mathsf{sid}, P, P', \mathsf{pw}^*, A^*)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{EKE-1r}}$:

6. If $\mathsf{pw}^* = \bot$, send $(\mathsf{NewKey}, \mathsf{sid}, P', 0^\kappa)$ to $\mathcal{F}_{\text{PAKE}}$. Furthermore, sample $B \leftarrow \mathcal{M}_2$ and $\tau' \leftarrow \{0,1\}^{2\kappa}$, and send (sid, B, τ') from P' to P.
7. Otherwise do the following steps:
   (1) Send $(\mathsf{TestPwd}, \mathsf{sid}, P', \mathsf{pw}^*)$ to $\mathcal{F}_{\text{PAKE}}$, and procede according to its response:

   “correct guess”: Sample $b \leftarrow \mathcal{R}$ and compute $B := \mathsf{msg}_2(b, A^*)$ and $K' \parallel \tau' := \mathsf{key}_2(b, A^*)$.

   “wrong guess”: Sample $(B, K', \tau') \leftarrow \mathcal{M}_2 \times \{0,1\}^\kappa \times \{0,1\}^{2\kappa}$.

   (2) Send $(\mathsf{NewKey}, \mathsf{sid}, P', K')$ to $\mathcal{F}_{\text{PAKE}}$ and (sid, B, τ') from P' to P.

On $(\mathsf{sid}, B^*, \tau^*)$ from $\mathcal{A}$ to P:

8. If $\mathsf{pw}^* = \bot \wedge B^* = B \wedge \tau^* = \tau'$, send $(\mathsf{NewKey}, \mathsf{sid}, P, 0^\kappa)$ to $\mathcal{F}_{\text{PAKE}}$.
9. Otherwise (i.e., if pw^* is undefined because no $\mathsf{Deliver}$ message has been sent, $\mathsf{pw}^* \neq \bot$, $B^* \neq B$, or $\tau^* \neq \tau'$), then for every $(x, A, a) \in T$, compute $K \parallel \tau = \mathsf{key}_1(a, B^*)$ and check whether $\tau^* = \tau$.
   A. If there is more than one such entry, output $\mathsf{Collision}$ and abort.
   B. If there is exactly one such entry, send $(\mathsf{TestPwd}, \mathsf{sid}, P, x)$ and then $(\mathsf{NewKey}, \mathsf{sid}, P, K)$ to $\mathcal{F}_{\text{PAKE}}$.
   C. If there is no such entry, send $(\mathsf{TestPwd}, \mathsf{sid}, P, \bot)$ and then $(\mathsf{NewKey}, \mathsf{sid}, P, 0^\kappa)$ to $\mathcal{F}_{\text{PAKE}}$.

Figure 20: Simulator $\mathcal{S}$ for the OEKE protocol.

#	case addressed	change	property used
1	$\mathsf{pw}^* = \bot \wedge \mathsf{pw} = \mathsf{pw}'$ $\wedge B^* = B$	$K = K'$ if $\tau^* = \tau'$ K \$ otherwise	KA correctness
2	$\mathsf{pw}^* = \bot \wedge \mathsf{pw} = \mathsf{pw}'$	K', τ' \$	KA pseudorandom non-malleability
3	$\mathsf{pw}^* = \bot \wedge \mathsf{pw} \neq \mathsf{pw}'$ $\wedge B^* = B \wedge \tau^* = \tau'$	K \$	none
4	$\mathsf{pw}^* \neq \bot$ $\vee B^* \neq B \vee \tau^* \neq \tau'$	exclude collisions while extracting "good" $(\mathsf{pw}')^*$	KA collision resistance
5		$K = \mathsf{key}_1(a^*, B^*)[1]$ if pw "good" K \$ otherwise	KA collision resistance
6		extract "good" $(\mathsf{pw}')^*$ $K = \mathsf{key}_1(a^*, B^*)[1]$ if $(\mathsf{pw}')^* = \mathsf{pw}$ K \$ if $(\mathsf{pw}')^* \neq \mathsf{pw}$ or no $(\mathsf{pw}')^*$	none

Table 9: Summary of hybrids for OEKE security. "\$" denotes "chosen at random from respective range".

- It is intercepted by the man-in-the-middle adversary $\mathcal{A}$, who possibly modifies it to program $\mathsf{pw}^* \mapsto A^*$ instead;
- $\mathcal{A}$ tells $\mathcal{F}_{\text{EKE-1r}}$ to $\mathsf{Deliver}$ the function to P';
- P' tells $\mathcal{F}_{\text{EKE-1r}}$ to $\mathsf{SampleResp}$, which generates B, K', τ';
- P' outputs K' and sends (B, τ') to P (again intercepted by $\mathcal{A}$);
- $\mathcal{A}$ sends (B^*, τ^*) to P;
- P outputs K if τ^* matches τ, and a random $K_\$$ otherwise.

The case that $\mathsf{pw}^* = \bot$. The following hybrids 1–3 only affect the case where $\mathsf{pw}^* = \bot$, i.e., $\mathcal{A}$ does not modify the P-to-P' message.

Hybrid 1: In the case that $\mathsf{pw}^* = \bot \wedge \mathsf{pw} = \mathsf{pw}' \wedge B^* = B$, do the following:

- If $\tau^* = \tau'$ (i.e., $\mathcal{A}$ is an eavesdropper), then P outputs $K := K'$;
- Otherwise, P outputs $K_\$ \leftarrow \{0,1\}^\kappa$.

In hybrid 0, we have that

- If $\tau^* = \tau = \mathsf{key}_1(a, B^*)[2]$, then P outputs $K := \mathsf{key}_1(a, B^*)[1]$;
- Otherwise, P outputs $K_\$ \leftarrow \{0,1\}^\kappa$.

The difference is that in hybrid 0, P outputs K if $\tau^* = \tau$, whereas in hybrid 1, P outputs K' (and defines K as K') if $\tau^* = \tau'$. Since $B^* = B$, we have $K \,\|\, \tau = \mathsf{key}_1(a, B^*) = \mathsf{key}_1(a, B)$; by the syntax of $\mathcal{F}_{\text{EKE-1r}}$ in the case of $\mathsf{pw}^* = \bot$ and $\mathsf{pw} = \mathsf{pw}'$, $B = \mathsf{msg}_2(b, A)$ and $K' \,\|\, \tau' = \mathsf{key}_2(b, A)$, where $A = \mathsf{msg}_1(a)$ according to protocol description (step 1). Then by correctness of KA, $(K, \tau) = (K', \tau')$ with overwhelming probability, so hybrids 0 and 1 are (statistically) indistinguishable.

Hybrid 2: In the case that $\mathsf{pw}^* = \bot \wedge \mathsf{pw} = \mathsf{pw}'$, sample $B \leftarrow \mathcal{M}_2$ and $K' \,\|\, \tau' \leftarrow \{0,1\}^{3\kappa}$.

The difference between hybrids 1 and 2 is that in hybrid 1, $B = \mathsf{msg}_2(b, A)$ and $K' \| \tau' = \mathsf{key}_2(b, A)$, whereas in hybrid 2, $B \leftarrow \mathcal{M}_2$ and $K' \| \tau' \leftarrow \{0,1\}^{3\kappa}$. We construct a reduction $\mathcal{R}$ to the pseudorandom non-malleability of the underlying key agreement protocol KA: $\mathcal{R}$, given (A, B, K', τ'), can simulate the experiment up to when P' outputs — which includes $\mathcal{A}$'s access to $\mathcal{F}_{\text{EKE-1r}}$, as well as B, K' and τ' — without knowing a or b. (Note that in $\mathcal{F}_{\text{EKE-1r}}$, b is used only when computing B and K'.) When $\mathcal{A}$ sends (B^*, τ^*), $\mathcal{R}$ checks whether $B^* \neq B$. If $B^* \neq B$, $\mathcal{R}$ outputs B^* to the pseudorandom non-malleability challenger and receives $K \| \tau$, then checks if $\tau^* = \tau$ and accordingly outputs either K or a random string to $\mathcal{Z}$. Alternatively, if $B^* = B$, the pseudorandom non-malleability experiment prohibits $\mathcal{R}$ from outputting B^*. Instead, $\mathcal{R}$ picks some other element of $\mathcal{M}_2$ to output (just to finish the experiment), and uses $K' \| \tau'$[20] instead of $K \| \tau$ to choose what to output to $\mathcal{Z}$. $\mathcal{R}$ copies $\mathcal{Z}$'s output bit. We can see that if $B = \mathsf{msg}_2(b, A)$ and $K' \| \tau' = \mathsf{key}_2(b, A')$ then $\mathcal{R}$ simulates hybrid 1, and if $B \leftarrow \mathcal{M}_2$ and $K' \| \tau' \leftarrow \{0,1\}^{3\kappa}$ then $\mathcal{R}$ simulates hybrid 2. Thus, $\mathcal{R}$'s distinguishing advantage is equal to $\mathcal{Z}$'s distinguishing advantage between hybrids 1 and 2, so hybrids 1 and 2 are indistinguishable.

Hybrid 3: In the case that $\mathsf{pw} \neq \mathsf{pw}' \wedge \mathsf{pw}^* = \bot \wedge B^* = B \wedge \tau^* = \tau'$, P outputs $K_\$ \leftarrow \{0,1\}^\kappa$ (i.e., the check of P always fails).

The difference between hybrids 2 and 3 is that in hybrid 2, P computes $K \| \tau = \mathsf{key}_1(a, B^*) = \mathsf{key}_1(a, B)$ and outputs $K_\$ \leftarrow \{0,1\}^\kappa$ only when $\tau \neq \tau^* = \tau'$, whereas in hybrid 3, P always outputs $K_\$ \leftarrow \{0,1\}^\kappa$. Note that $\mathcal{F}_{\text{EKE-1r}}$ outputs a uniformly random $\tau' \leftarrow \{0,1\}^{2\kappa}$, so the probability that $\tau = \tau'$ is negligible. Thus, hybrids 2 and 3 are indistinguishable.

All cases where the adversary does not eavesdrop. Both the case where $\mathsf{pw}^* \neq \bot$ (i.e., $\mathcal{A}$ modifies the P-to-P' message, or chooses its own message first), and the case where $\mathsf{pw}^* = \bot$ and $B^* \neq B \vee \tau^* \neq \tau'$ (i.e., $\mathcal{A}$ modifies the P'-to-P message) are affected by the following hybrids 4–6.

Hybrid 4: In the case that $\mathsf{pw}^* \neq \bot \vee B^* \neq B \vee \tau^* \neq \tau'$, output $\mathsf{Collision}$ and abort if there exists more than one "good" $(\mathsf{pw}')^*$. Here, and in subsequent hybrids, we call $(\mathsf{pw}')^*$ "good" if (1) there was a query $(\mathsf{Eval}, \mathsf{sid}, P, P', (\mathsf{pw}')^*)$ to $\mathcal{F}_{\text{EKE-1r}}$ and (2) $\tau^* = \mathsf{key}_1(a^*, B^*)[2]$. (Intuitively a "good" $(\mathsf{pw}')^*$ is a password guess for P that can be extracted from (B^*, τ^*).)

We upper-bound $\Pr[\mathsf{Collision}]$ via a reduction to the collision-resistance of KA. Suppose that $\mathcal{F}_{\text{EKE-1r}}$ receives at most q distinct Eval queries. The reduction, given $(A_1, \ldots, A_q)$, handles the i-th such query by outputting A_i, without knowing the corresponding "trapdoor" a_i. Then the reduction simulates the P' side to $\mathcal{Z}$, producing some B, K' and τ'. (Note that after hybrid 2, the only case where these are not sampled uniformly at random is that $\mathsf{pw}^* = \mathsf{pw}'$. In this case $B = \mathsf{msg}_2(b, A^*)$ and $K' \| \tau' = \mathsf{key}_2(b, A^*)$, where A^* is chosen by $\mathcal{A}$, so the simulation of this case also does not require knowledge of a.) When $\mathcal{A}$ sends (B^*, τ^*), the reduction outputs B^*. If $\mathsf{Collision}$ happens, then the reduction wins. By the collision-resistance of KA, $\Pr[\mathsf{Collision}]$ is negligible and thus hybrids 3 and 4 are indistinguishable.

Hybrid 5: In the case that $\mathsf{pw}^* \neq \bot \vee B^* \neq B \vee \tau^* \neq \tau'$, P outputs $K_\$ \leftarrow \{0,1\}^\kappa$ if pw is not "good".

The difference between hybrids 4 and 5 occurs when $\tau^* = \tau$ (i.e., the check of P passes), but pw is never sent to $\mathcal{F}_{\text{EKE-1r}}$ via Eval (otherwise pw would be "good"). Then in hybrid 4, P outputs $K = \mathsf{key}_1(a, B^*)[1]$, except for the special case where $\mathsf{pw}^* = \bot \wedge \mathsf{pw} = \mathsf{pw}' \wedge B^* = B$, which already outputs a random $K_\$$ because of hybrid 1. In hybrid 5, P instead outputs $K_\$$. We argue that the probability pw is never sent in an Eval query and $\tau^* = \tau$ is negligible, via a reduction to the collision-resistance of KA.

Let $q \geq 2$ be a parameter to be decided later. The reduction ignores its input $(A_1, \ldots, A_q)$, and

[20] In hybrid 1 we already changed P to use $K' \| \tau' = \mathsf{key}_2(b, A)$ instead of $\mathsf{key}_1(a, B)$ in this case.

simulates the P' side as in the reduction under hybrid 4. When $\mathcal{A}$ sends B^*, the reduction also outputs B^*.

Assuming $\mathsf{Eval}(\ldots, \mathsf{pw})$ is never queried, the correct A $\mathsf{Programed}$ by P is never used and is unknown to $\mathcal{A}$, so for any $i \in [q]$ we could pretend that $A = A_i$ and it would make no difference to the probability that $\tau^* = \tau$. Therefore, for any i, the probability p (given that B^* takes its value) that $\tau^* = \mathsf{key}_1(a, B^*)[2]$ is the same as the probability that $\tau^* = \mathsf{key}_1(a_i, B^*)[2]$. Then the chance of a collision is at least the chance that there exists $i \neq j$ such that $\tau^* = \mathsf{key}_1(a_i, B^*)[2] = \mathsf{key}_1(a_j, B^*)[2]$. These are just q independent events, so this probability is exactly

$$p' = \sum_{k=2}^{q} \binom{q}{k} p^k (1-p)^{q-k} \geq \binom{q}{2} p^2 (1-p)^{q-2},$$

which is obviously increasing in both p and q, so for a lower bound on p' we can reduce p to be at most $\frac{1}{2}$, and then reduce q so that $q \leq 2 + \frac{1}{2p}$. Then $(1-p)^{q-2} \geq (1-p)^{\frac{1}{2p}} = 2^{\frac{\log_2(1-p)}{2p}} \geq \frac{1}{2}$, since $\log_2(1-p) \geq -2p$ for $p \leq \frac{1}{2}$. Substituting $q \mapsto \min(q, 2 + \frac{1}{2p})$ into p' gives

$$p' \geq \frac{1}{4}\left(\min\left(q, 2 + \frac{1}{2p}\right) - 1\right)^2 p^2 = \frac{1}{4}\min\left((q-1)^2 p^2, \left(2 + \frac{1}{2p} - 1\right)^2 p^2\right) \geq \frac{1}{4}\min\left((q-1)^2 p^2, \frac{1}{4}\right),$$

which is valid for $p \leq \frac{1}{2}$. Next, to make it always valid, substitute $p \mapsto \min(p, \frac{1}{2})$ to get

$$p' \geq \frac{1}{4}\min\left((q-1)^2 \min\left(p, \frac{1}{2}\right)^2, \frac{1}{4}\right) \geq \frac{1}{4}\min\left((q-1)^2 p^2, \frac{1}{4}\right),$$

so this inequality is actually always valid.

To summarize, we have related p (the probability of the "bad event" that allows $\mathcal{Z}$ to distinguish between hybrids 4 and 5) and p' (a lower bound of the reduction's advantage against the collision-resistance of KA). Now for any polynomial q, if the reduction has advantage $\mathsf{Adv}_q = p' < \frac{1}{16}$, then by the inequality above, $\mathsf{Adv}_q \geq \frac{1}{4}(q-1)^2 p^2$. Therefore, hybrids 4 and 5 can be distinguished with probability at most

$$p < \frac{2}{q-1}\sqrt{\mathsf{Adv}_q}.$$

Hybrid 6: In the case that $\mathsf{pw}^* \neq \perp \vee B^* \neq B \vee \tau^* \neq \tau'$, do:

- If there is exactly one "good" $(\mathsf{pw}')^*$, and $(\mathsf{pw}')^* = \mathsf{pw}$, then P outputs $K := \mathsf{key}_1(a, B^*)[1]$;
- If there is exactly one "good" $(\mathsf{pw}')^*$, and $(\mathsf{pw}')^* \neq \mathsf{pw}$, then P outputs $K_\$ \leftarrow \{0,1\}^\kappa$;
- If there is no "good" $(\mathsf{pw}')^*$, then P also outputs $K_\$ \leftarrow \{0,1\}^\kappa$.

The difference between hybrids 5 and 6 is that

- In hybrid 5, P outputs K if pw is "good", and $K_\$$ otherwise;
- In hybrid 6, P outputs K if there exists exactly one "good" $(\mathsf{pw}')^*$ and $(\mathsf{pw}')^* = \mathsf{pw}$, and $K_\$$ otherwise.

(If there is more than one "good" $(\mathsf{pw}')^*$, then both hybrids 5 and 6 output $\mathsf{Collision}$ and abort.)

Assuming $\mathsf{Collision}$ does not happen, there is *at most* one "good" pw^*. If pw is "good", this means that there is exactly one "good" pw^* and $\mathsf{pw}^* = \mathsf{pw}$. Vice versa, if there is exactly one "good" pw^* and $\mathsf{pw}^* = \mathsf{pw}$, then of course pw is "good". Thus, the conditions in hybrids 5 and 6 are equivalent, so hybrids 5 and 6 are identical.

Comparison between hybrid 6 and the ideal world. We now argue that hybrid 6 is identical to the ideal world, where the simulator $\mathcal{S}$ as defined in Figure 20 interacts with $\mathcal{F}_{\text{PAKE}}$. We first consider the P' side message B, τ' and session key K'. This is very similar to the corresponding argument in the proof of Thm. 5.4: in hybrid 6 (in fact, after hybrid 2) the only case where B, τ' and K' are not uniformly random is that $\mathsf{pw}^* = \mathsf{pw}'$; in this case $B = \mathsf{msg}_2(b, A^*)$ and $K' \parallel \tau' = \mathsf{key}_2(b, A^*)$ (cf. Table 7). In the ideal world,

- If $\mathsf{pw}^* = \bot$, $\mathcal{S}$ enters step 6, where it sends NewKey without TestPwd, so the P' session is fresh and $\mathcal{F}_{\text{PAKE}}$ samples a uniformly random K' for P'. $\mathcal{S}$ also samples uniformly random B and τ' as the message.
- If $\mathsf{pw}^* = \mathsf{pw}'$, $\mathcal{S}$ enters step 7, where it sends TestPwd resulting in "correct guess" and then NewKey where the session key is $\mathsf{key}_2(b, A^*)[1]$ where $b \leftarrow \mathcal{R}$, so the P' session is compromised and $\mathcal{F}_{\text{PAKE}}$ sets $K' := \mathsf{key}_2(b, A^*)[1]$ for P'. $\mathcal{S}$ also computes $B = \mathsf{msg}_2(b, A^*)$ and $\tau' = \mathsf{key}_2(b, A^*)[2]$.
- If $\mathsf{pw}^* \neq \bot \wedge \mathsf{pw}^* \neq \mathsf{pw}'$, $\mathcal{S}$ enters step 7, where it sends TestPwd resulting in "wrong guess" and then NewKey, so the P' session is interrupted and $\mathcal{F}_{\text{PAKE}}$ samples a uniformly random K' for P'. (Note that in this case the session key chosen by $\mathcal{S}$ in NewKey does not matter.) $\mathcal{S}$ also samples uniformly random B and τ'.

We conclude that for B, τ' and K', hybrid 6 and the ideal world are identical in all cases.

Next, we consider the P side output, which is either K or a uniformly random $K_{\$}$. Since there are fewer hybrids than in the proof in Thm. 5.4, here we do not make a table of all cases and state them in the text instead:

- If $\mathsf{pw} = \mathsf{pw}' \wedge \mathsf{pw}^* = \bot \wedge B^* = B \wedge \tau^* = \tau$ ($\mathcal{A}$ eavesdrops and the two parties' passwords match), in hybrid 6 P outputs $K = K'$ (changed in hybrid 1). In the ideal world, $\mathcal{S}$ enters step 8, where it sends NewKey without TestPwd, so the P session is fresh. Since the P' session was also fresh when P' output and $\mathsf{pw} = \mathsf{pw}'$, $\mathcal{F}_{\text{PAKE}}$ sets $K := K'$ for P.
- If $\mathsf{pw} \neq \mathsf{pw}' \wedge \mathsf{pw}^* = \bot \wedge B^* = B \wedge \tau^* = \tau$ ($\mathcal{A}$ eavesdrops and the two parties' passwords do not match), in hybrid 6 P outputs $K_{\$}$ (changed in hybrid 3). In the ideal world, this is identical to the previous case, except that the P' session was fresh when P' output but $\mathsf{pw} \neq \mathsf{pw}'$, so $\mathcal{F}_{\text{PAKE}}$ samples a uniformly random session key for P (independent of K').
- If $\mathcal{A}$ does not eavesdrop and there is more than one "good" $(\mathsf{pw}')^*$, hybrid 6 outputs Collision and aborts the experiment (changed in hybrid 4). In the ideal world, $\mathcal{S}$ enters step 9A, where it also outputs Collision and aborts.
- If $\mathcal{A}$ does not eavesdrop, there is exactly one "good" $(\mathsf{pw}')^*$, and $(\mathsf{pw}')^* = \mathsf{pw}$, in hybrid 6 P outputs P outputs $K = \mathsf{key}_1(a, B^*)[1]$ (changed in hybrid 6). In the ideal world, $\mathcal{S}$ enters step 9B, where it sends TestPwd resulting in "correct guess" and then NewKey where the session key is $\mathsf{key}_1(a, B^*)[1]$ and a is the value sampled by $(\mathsf{Eval}, \ldots, \mathsf{pw})$. Then the P session is compromised and $\mathcal{F}_{\text{PAKE}}$ sets $K := \mathsf{key}_1(a, B^*)[1]$ for P. We can see that in both hybrid 9 and the ideal world, $K = \mathsf{key}_1(a, B^*)[1]$.
- If $\mathcal{A}$ does not eavesdrop, there is exactly one "good" $(\mathsf{pw}')^*$, and $(\mathsf{pw}')^* \neq \mathsf{pw}$, in hybrid 6 P outputs $K_{\$}$ (changed in hybrid 6). In the ideal world, $\mathcal{S}$ enters step 9B, where it sends TestPwd resulting in "wrong guess" and then NewKey, so the P session is interrupted and $\mathcal{F}_{\text{PAKE}}$ samples a uniformly random session key for P.

- If $\mathcal{A}$ does not eavesdrop and there is no "good" $(\mathsf{pw}')^*$, in hybrid 6 P outputs $K_{\$}$ (changed in hybrid 5). In the ideal world, $\mathcal{S}$ enters step 9C, where it sends TestPwd resulting in "wrong guess" (note that $\mathcal{S}$'s password guess is $\perp$) and then NewKey, so the P session is interrupted and $\mathcal{F}_{\text{PAKE}}$ samples a uniformly random session key for P.

We conclude that for K, hybrid 6 and the ideal world are identical in all cases. In summary, hybrid 6 and the ideal world are identical. This completes the proof. □

The theorem immediately implies the following: let KA be a correct, secure, pseudorandom, pseudorandom non-malleable, and collision-resistant KA protocol. Beginning with the protocol in Figure 19, replace $\mathcal{F}_{\text{EKE-1r}}$ with the protocol in Figure 15, and $\mathcal{F}_{\text{POPF}}$ with the protocol in Figure 9. Then by Thms. 4.3, 5.1 and 5.7 the resulting protocol realizes $\mathcal{F}_{\text{PAKE}}$ in the ROM.

OEKE-PRF and OEKE-RO. Since our Thm. 5.7 considers a general OEKE protocol with *any* KA protocol whose key length is 3κ, this immediately covers OEKE-PRF as a special case, where the KA protocol is obtained via taking another KA protocol whose key length is κ and applying a PRF to the key (on inputs such as 0, 1 and 2). OEKE-RO is in turn a special case of OEKE-PRF, where the PRF is an RO.

6 Conclusion and Future Work

Why are there so many recent works on the UC-security of (O)EKE, the very first PAKE protocol that has been around for over 30 years? Why are there so many subtleties in the analysis of this simple and seemingly innocuous protocol? In this final section, we offer some insights and personal perspectives.

6.1 Subtleties in the Security Model

The first source of complication lies in the security model. To begin with, any security notion of PAKE *must consider a man-in-the-middle adversary*; in other words, the parties cannot have authenticated channels between each other. This setting is different from the vast majority of works on multi-party computation, and as we have seen in Sects. 3 and 5, the most complicated and subtle case is that the man-in-the-middle adversary passes the first message without modification but then modifies the second — which does not have a correspondence in "normal" 2PC where one party is honest and the other is corrupt. Failure to consider such cases (e.g., [MRR20]) renders the security proof incomplete and might even result in incorrect security statements.

Next, the UC PAKE functionality is also non-trivial and somewhat difficult to understand. PAKE is a quintessential example of a cryptographic primitive whose security notion is easy to see intuitively but hard to define formally. At first glance, the security requirement is simply "the only feasible attack is online guessing". But a complete description of the two parties' output behaviors must consider a large number of cases, as each party has three possible states: no attack, successfully attacked, and unsuccessfully attacked (corresponding to the sessions being fresh, compromised, and interrupted, respectively); furthermore, the output of one party depends on not only the state of itself, but also (if the party is unattacked) the state of its counterparty and whether the two parties' passwords match. This is why the UC PAKE functionality (Figure 3) contains some complicated sentences such as

> *If the record* $[\langle P, P', \mathsf{pw}\rangle]$ *is* fresh*, a key* (sid, K') *has been output to* P'*, at which time there was a record* $\langle P', P, \mathsf{pw}\rangle$ *marked* fresh*, then set* $K := K'$*.*

which essentially just says that if there is no attack on either session and the two parties' passwords match, then they should output the same (uniformly random) key. The complication here is that to formally describe this, the party that outputs first needs to output a random key, and the party that outputs next needs to output a key which is the same as the first party's. (It is interesting that this simplest case needs the most complicated language to describe.)

What complicates things even further is that *in UC, the order of events matters*. For example, consider a 2-round PAKE and assume the two parties' passwords match; if one direction is successfully attacked and the other direction is not, one might expect that the unattacked party should output an independent random key. However, this is not necessarily true. Consider two cases: (1) the adversary passes the first (P-to-P') message without modification, but then modifies the second (P'-to-P) and successfully attacks the P session, and (2) the adversary modifies the first message and successfully attacks the P' session, but then passes the second message. As we have seen in Sect. 3, in case (1) the unattacked P' should indeed output an independent random key, as its session is fresh and *the simulator does not know the key of* P'. However, in case (2) when P' outputs the simulator already knows *both the password and the session key of* P', so even if the adversary does not modify the P'-to-P message, the simulator can still use the password to compromise the P session and set the session key of P to be correlated to that of P'. In other words, the simulator in case (2) has more options and is "stronger". This explains why almost all flaws in prior works are failures to consider case (1), rather than case (2).

We also caution that *there is a significant gap between game-based security and UC-security of PAKE*. As we explained at the end of Sect. 3, UC-security is much more subtle, and it is dangerous to give a game-based security proof and then believe that it "naturally extends" to UC-security.[21]

6.2 Subtleties in the Protocol Description

The second type of complexity comes from the fact that both EKE and OEKE have a large number of variants. These variants have two dimensions: whether IC, HIC or POPF is used in protocol messages, and how exactly the parties' session keys are derived. Since we have already poured a lot of ink on POPF, here we mainly focus on output derivation.

The case of EKE. In EKE there is, of course, the option to output the "raw" KA key K. However, it appears generally understood that the KA key needs to be hashed. Still, there are a lot of confusions on what exactly should be included in the hash: Should the transcript ϕ, ϕ' be included? What about the password pw? And is it possible to avoid explicitly working in the ROM by using a PRF instead?

Indeed, it is highly non-trivial to see what needs to be hashed and what does not, and what exactly will go wrong if we don't hash those items that are necessary.[22] Now equipped with our (in)security results, we can give a summary here:

- Outputting the "raw" K requires pseudorandom non-malleability of the underlying KA protocol, which covers an adversary that passes the first message but modifies the second (Sects. A.3 and 3.2).

[21] Our opinion is that the game-based security definition is outdated and should not be used anyway. But the bottom line is that if a PAKE protocol is proven game-based secure, then this only provides a minimal security guarantee, and UC-security needs to be proven separately.

[22] Of course, the "safest" option is to simply hash everything. But this comes with the risk of writing a flawed security proof, as one would then be sure that this version "works". To put it in another way, given a complete security proof, it should be immediately clear what exact items have to be hashed.

- Outputting an RO hash of K — which can be viewed as using a modified KA protocol whose key is $H(K)$ instead of K, then outputting the "raw" key of this modified KA protocol — essentially creates a new KA protocol that has pseudorandom non-malleability *in the specific case of KA protocols with perfect pseudorandomness (such as Diffie-Hellman)*; however, there is a $\Theta(q)$ loss while reducing to KA security (Sect. A.1, the "reducing to DDH" case). Furthermore, if we take a secure and (strong) pseudorandom KA protocol and hash the key at the end, this does not yield a pseudorandom non-malleable KA protocol in general, so pseudorandom non-malleability is still needed (Sect. A.3).

- Including pw in the hash does not help, as all attacks on the "raw" protocol require the adversary to know pw (and query the IC accordingly), so the simulator can already extract pw from protocol messages and additionally allowing extraction from the final hash does not provide any advantage.

- Including the first message ϕ in the hash does not help either, as all attacks on the "raw" protocol require the adversary to pass ϕ without modification — in other words, the first message is consistent among (and known to) all parties including the adversary, so there is no need to hash it.

- Including the second message ϕ' in the hash helps only when HIC or POPF is used, and the problematic case is again the adversary passes the first message but modifies the second. This time the adversary can replace ϕ' with another $(\phi')^*$ which corresponds to the same underlying KA message, and (standard) UC PAKE security dictates that in this case the two parties must output independent keys (Sect. A.4) — which is achieved exactly by letting P' output $H(\phi', K)$ and P output $H((\phi')^*, K)$.[23]

- Finally, a trivial observation is that $H(\phi', K)$ in the bullet above can be replaced by $\mathsf{PRF}_K(\phi')$, avoiding explicit mentioning of an RO. This is what our Thm. 5.4 analyzes.

Independently of the above, *strong* pseudorandomness of the underlying KA protocol is required no matter what is included in the hash (Sect. 3.3).

See Table 10 for a summary of the discussion above.

[23] In the case of using a 1-simultaneous round KA protocol, it might be unclear which message is the second, so both messages need to be hashed. See Thm. 5.6.

output function	insecure with plain Diffie-Hellman?	$\Theta(q^2)$ loss under CDH / $\Theta(q)$ loss under DDH with plain Diffie-Hellman?	only realizes $\mathcal{F}_{\text{PAKE-sp}}$ if HIC/POPF used?	analyzed in...
K	✓		✓	[MRR20, Theorem 10] [SGJ23, Theorem 2] (both theorems overlook both issues) our Thms. B.1 and B.2
$H(K)$		✓	✓	
$H(\phi, \phi', K)$		✓		[DHP+18, Theorem 6] [BCP+23, Theorem 1] (both proofs overlook the security loss)
$H(\mathsf{pw}, \phi, \phi', K)$		✓		
$H(\phi', K)$		✓		
$\mathsf{PRF}_K(\phi')$ (EKE-PRF)	✓			our Thm. 5.4
In all cases, strong pseudorandomness and pseudorandom non-malleability needed in general (overlooked in [MRR20, Theorem 10], [SGJ23, Theorem 2], [BCP+23, Theorem 1])				

Table 10: Summary of various versions of EKE. [DHP+18, Theorem 6] uses Diffie-Hellman KA and IC; [MRR20, Theorem 10] uses general KA and POPF; [SGJ23, Theorem 2] uses general KA and HIC; [BCP+23, Theorem 1] uses general KA and IC; our Thm. 5.4 uses general KA and POPF

The case of OEKE. The "raw" version of OEKE needs a KA protocol whose key is longer than the PAKE session key (κ bits), as there are two things that need to appear independent of each other: the session key SK and the authenticator τ. Our result assumes such a KA protocol, but existing works assume a KA protocol whose key is κ-bit long and then use specific methods to compile it into a KA protocol with long key:

- In OEKE-PRF, $SK = \mathsf{PRF}_K(0)$ and $\tau = \mathsf{PRF}_K(1)$. This version requires the underlying KA to be pseudorandom non-malleable, as an attack similar to that of EKE with "raw" key — where the adversary passes the first message and modifies the second — can cause the two parties' KA keys to be correlated, and the PRF offers no security guarantee in this case (Sect. A.2). (A difference with the EKE attack is that here the adversary does not even need to know the password, as the second message is not encrypted.) A second attack involves the adversary (that also doesn't need to know the password) unilaterally biasing the KA key of P and predicting the session key of P, revealing the necessity of collision resistance in the underlying KA protocol (Sect. 3.1).
- In OEKE-RO, $SK = H(K, 0)$ and $\tau = H(K, 1)$. Now the RO guarantees independent outputs even if the KA keys are correlated. However, a more sophisticated attack shows that pseudorandom non-malleability is still needed (Sect. A.3). Furthermore, this does not alleviate the second attack above, so collision resistance is also needed.

- Including the password pw in the hash *for* τ eliminates the second attack above, as coming up with a valid τ requires knowledge of pw, and if the adversary knows pw (and uses it to attack the P session), then we have to allow it to predict the session key of P anyway. Therefore, collision resistance is not needed anymore (Thm. 3.3). Note that including pw in the hash for the session key SK does not provide any additional advantage.
- The transcript $\phi, (B, \tau)$ is not used in any of the attacks, so there is no need to including it in the hash.

See Table 11 for a summary of the discussion above.

output function (session key, authenticator)	collision resistance needed?	analyzed in...
$(\mathsf{PRF}_K(0), \mathsf{PRF}_K(1))$ (OEKE-PRF)	✓	[SGJ23, Theorem 3] proof (overlooks this property)
$(H(K,0), H(K,1))$ (OEKE-RO)	✓	
$(H(K), H(A,K))$		[SGJ23, Theorem 3] statement
$(H(K), H(\mathsf{pw},K))$		
$(H(\phi,B,\tau,K),$ $H(\mathsf{pw},\phi,B,K))$		[BCP+23, Theorem 2]
$(K[1], K[2])$ (OEKE)	✓	our Thm. 5.7
In all cases, pseudorandom non-malleability needed (overlooked in [SGJ23, Theorem 3], [BCP+23, Theorem 2])		

Table 11: Summary of various versions of OEKE. [SGJ23, Theorem 3] uses general KA and HIC; [BCP+23, Theorem 2] uses general KA and IC; our Thm. 5.7 uses general KA and POPF

Remark 6.1. *For an example of whether hashing the password or not* while deriving the session key *actually matters, see [AP05] which proposes two PAKE protocols, SPAKE1 and SPAKE2, whose only difference is that SPAKE2 includes the password in the final hash while SPAKE1 does not. Careful analysis shows that the (game-based) security of SPAKE1 is in the non-concurrent setting and non-tight under CDH, while SPAKE2 has concurrent security and a tight reduction to CDH.*

6.3 Subtleties in the Security Analysis

Regarding the security analysis, the general lesson is that in the context of UC, a reduction needs to act as the simulator (plus the functionality) while communicating with the environment/adversary; however, while performing the simulator's task, *the reduction loses some information compared with the actual simulator because it needs to embed some challenges in the experiment.* (Of course, "losing some information" is the case for reductions in general. But the complexity of the UC framework leads to this principle being overlooked more frequently.) As we see in Sect. A.1, the flaws in the proofs of [BCP+23, Theorem 1] and [DHP+18, Theorem 6] are that in both proofs the reduction to CDH/DDH does not know a (because it needs to embed the CDH/DDH challenge g^a in the experiment), so g^{ar} (for adversarially chosen r such that g^r is known to the reduction) looks random and the reduction cannot tell which H query is $H(g^{ar})$ — which is overlooked in the proofs. This issue is particularly subtle since it emerges *after* P' *outputs*, so one who thinks the reduction is done

once P' outputs would fail to detect it (this is where [BCP+23] fails — see Sect. A.1 for a detailed discussion). In addition, the issue is non-existent if the adversary modifies the first message but passes the second — in other words, the two messages are not "symmetric". Thus, a tight reduction to CDH in the latter case does not imply a tight reduction in the former case.

Since most hybrid proofs for UC PAKE are complicated and involve a large number of hybrids, it might be helpful to provide a table similar to Tables 6 and 9 at the beginning — in addition to, or in lieu of, a prose summary of the chain of hybrids. We believe this helps the reader understand the essence of the proof without digging into the details of too many hybrids. Furthermore, (assuming the hybrids begin in the real world) it would be useful to include an argument on why the last hybrid is identical to the ideal world, with the challenger split into the simulator and the functionality; this is far from obvious in many cases.

Sampling from $\mathbb{Z}_p$ and $\mathbb{Z}_p^*$ might make a difference. Last but not least, we wish to bring up a point which shows that even seemingly minor issues might become significant in some contexts and thus should not be ignored. Let us start from a topic that appears unrelated. Suppose we have a group of prime order p. Recall that the Square Diffie-Hellman (SDH) assumption says that given g^a where $a \leftarrow \mathbb{Z}_p$, it is hard to compute g^{a^2}. A standard reduction to CDH (see, e.g., the proof of [FKL18, Theorem 3.1]) works as follows: the reduction, on $A = g^a$, samples $r \leftarrow \mathbb{Z}_p$ and feeds (A, A^r) to the CDH solver; upon receiving X from the CDH solver, the reduction outputs $X^{\frac{1}{r}}$.

However, this reduction fails in the case of $r = 0$, which happens with probability $1/p$. A complete description of the reduction should say that it aborts if $r = 0$ and thus loses an additive term $1/p$, which is generally missing in existing works. Alternatively, the reduction could sample $r \leftarrow \mathbb{Z}_p^* = \mathbb{Z}_p \setminus \{0\}$ — which is a reduction between variants of CDH and SDH where a, b are also sampled from $\mathbb{Z}_p^*$.[24]

A similar issue appears in the 2HashDH Oblivious PRF (OPRF) [JKK14, JKKX16, JKKX17, JKX18, HJKW23, DFG+23], which (in its simplest form) involves two parties jointly evaluating the function $f_k(x) = H_2(x, H_1(x)^k)$ on some input x: a user, that holds x, samples $r \leftarrow \mathbb{Z}_p$ and sends $A := H_1(x)^r$ to a server; the server, that holds k, sends $B := A^k$ to the user; finally, the user outputs $H_2(x, B^{\frac{1}{r}})$. Obviously the user fails if $r = 0$, and it should instead sample $r \leftarrow \mathbb{Z}_p^*$. All of the cited works on the 2HashDH OPRF have the user sample $r \leftarrow \mathbb{Z}_p$, and since this protocol is used as a building block in various protocols (including the OPAQUE strong asymmetric PAKE protocol that has been recommended for standardization by IETF [JKX18]), this issue has dragged on for years.

While it might appear pedantic to insist on the edge case of $r = 0$, it turns out that sampling from $\mathbb{Z}_p$ and $\mathbb{Z}_p^*$ might actually make an essential difference, as we have seen in the attack on OEKE in Sect. 3.1. Recall again that the attack works as follows: the adversary sends $B = e$ and $\tau = F_e(1)$ to P, causing P to output $F_e(0)$ — which the adversary can predict without knowing the password. This issue is somewhat hidden as sampling from $\mathbb{Z}_p^*$ is indeed unnecessary for the standard security notion of Diffie-Hellman; however, the key point is that in OEKE *we are using Diffie-Hellman in a somewhat non-standard manner, namely in the context where there is a man-in-the-middle adversary in the higher-level protocol that can in particular control the message* g^b. This means that missing the seemingly $1/p$ probability of sampling $b = 0$ actually "translates to" missing the collision resistance property of the KA protocol, which is necessary for the security of OEKE.

In general, it seems "safer" to always sample from $\mathbb{Z}_p^*$, as this excludes the $r = 0$ case that might cause us trouble. However, we believe it is warranted to develop a thorough understanding of whether

[24] A better reduction would sample $r \leftarrow \mathbb{Z}_p$ and feed (A, Ag^r) to the CDH solver, and upon receiving X output X/A^r. This reduction works even if $r = 0$. Another advantage is that this reduction works nicely even in a composite-order cyclic group.

sampling from $\mathbb{Z}_p$ and sampling from $\mathbb{Z}_p^*$ — or other seemingly minor issues — make an essential difference in a certain context, and what the reason exactly is (see, e.g., [PX23, Definition 2.1] and [MX23, Footnote 12]). This might help reveal some general patterns that are hidden otherwise, such as collision resistance in our case (cf. Footnote 22).

Remark 6.2. *The case of the 2HashDH OPRF in [HJKW23,DFG+23] has a slightly more significant (but still minor) issue: they only require the GDH or the one-more GDH assumption in a cyclic group, without any specific requirements on the order. (Earlier works require the order to be prime.) This means that there might be more than one r-th root of $B = H_1(x)^{kr}$, and the user might use a wrong one (i.e., other than $H_1(x)^k$) in the OPRF output — even assuming that the r-th root(s) exist and can be efficiently computed. As a concrete example, say the group order is $2p$ where p is prime (e.g., $\mathbb{Z}_q^*$ where q is a strong prime). Assume the user's exponent $r \leftarrow \mathbb{Z}_{2p}$ is even, which happens with probability 1/2. If $r = 0$ then the r-th root of B does not exist, so the user fails. Otherwise there are two r-th roots of B, $H_1(x)^k$ and $H_1(x)^{k+p}$, and the user has a 1/2 chance of using the wrong one in the OPRF output (note that if $k \leftarrow \mathbb{Z}_{2p}$ then which one is the right value is independent of the user's view). Overall this OPRF protocol has an over 1/4 correctness failure probability.*[25] *However, this is unrelated to our main point.*

6.4 Future Work

The first future direction that comes to mind is that in recent years there have been a number of *asymmetric* PAKE (aPAKE) protocols using IC or HIC [GJK21, SGJK22, SGJ23], and it would be interesting to see whether the (H)IC there can be replaced by the more efficient POPF as well. We expect the security analyses to be significantly more complicated than the ones in this work, as those aPAKE protocols are built from Authenticated Key Exchange (AKE) rather than KA, and defining the security of AKE (let alone any additional properties necessary in the context of aPAKE) is a much more complicated and delicate task.

Another direction is to explore the quantum security of (O)EKE. While Kyber is a post-quantum KA protocol, the POPF in (O)EKE uses an RO, to which our analysis only considers classical queries. As such, our analysis serves as a first step towards the post-quantum security of (O)EKE, but a complete analysis would at least require working in the Quantum-accessible ROM (QROM) [BDF+11], rather than the ROM. One difficulty is that the study of QROM and the study of UC have been developed mostly in parallel; while there have been works proving the universal composability theorem with a quantum environment [Unr10], to the best of our knowledge there have been no works on formalizing the QROM in the quantum UC framework. It seems that some theoretical foundations need to be laid out before we can pursue this direction.

References

[ABB+20] Michel Abdalla, Manuel Barbosa, Tatiana Bradley, Stanislaw Jarecki, Jonathan Katz, and Jiayu Xu. Universally composable relaxed password authenticated key exchange. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part I*, volume 12170 of *LNCS*, pages 278–307. Springer, Cham, August 2020.

[ABJS24] Afonso Arriaga, Manuel Barbosa, Stanislaw Jarecki, and Marjan Skrobot. C'est Très CHIC: A compact password-authenticated key exchange from lattice-based KEM. In

[25] Interestingly, another section of [DFG+23] correctly requires the group order to be prime (see Theorem 3 in Appendix D). But their analysis of the 2HashDH OPRF (Theorem 2 in Appendix C.2) does not make this requirement.

Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part V*, volume 15488 of *LNCS*, pages 3–33. Springer, Singapore, December 2024.

[AHH21] Michel Abdalla, Björn Haase, and Julia Hesse. Security analysis of CPace. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part IV*, volume 13093 of *LNCS*, pages 711–741. Springer, Cham, December 2021.

[AP05] Michel Abdalla and David Pointcheval. Simple password-based encrypted key exchange protocols. In Alfred Menezes, editor, *CT-RSA 2005*, volume 3376 of *LNCS*, pages 191–208. Springer, Berlin, Heidelberg, February 2005.

[AWZ23] Damiano Abram, Brent Waters, and Mark Zhandry. Security-preserving distributed samplers: How to generate any CRS in one round without random oracles. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 489–514. Springer, Cham, August 2023.

[BCP03] Emmanuel Bresson, Olivier Chevassut, and David Pointcheval. Security proofs for an efficient password-based key exchange. In Sushil Jajodia, Vijayalakshmi Atluri, and Trent Jaeger, editors, *ACM CCS 2003*, pages 241–250. ACM Press, October 2003.

[BCP+23] Hugo Beguinet, Céline Chevalier, David Pointcheval, Thomas Ricosset, and Mélissa Rossi. GeT a CAKE: Generic transformations from key encaspulation mechanisms to password authenticated key exchanges. In Mehdi Tibouchi and Xiaofeng Wang, editors, *ACNS 23International Conference on Applied Cryptography and Network Security, Part II*, volume 13906 of *LNCS*, pages 516–538. Springer, Cham, June 2023.

[BDF+11] Dan Boneh, Özgür Dagdelen, Marc Fischlin, Anja Lehmann, Christian Schaffner, and Mark Zhandry. Random oracles in a quantum world. In Dong Hoon Lee and Xiaoyun Wang, editors, *ASIACRYPT 2011*, volume 7073 of *LNCS*, pages 41–69. Springer, Berlin, Heidelberg, December 2011.

[BFGJ17] Jacqueline Brendel, Marc Fischlin, Felix Günther, and Christian Janson. PRF-ODH: Relations, instantiations, and impossibility results. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part III*, volume 10403 of *LNCS*, pages 651–681. Springer, Cham, August 2017.

[BGHJ24] Manuel Barbosa, Kai Gellert, Julia Hesse, and Stanislaw Jarecki. Bare PAKE: Universally composable key exchange from just passwords. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part II*, volume 14921 of *LNCS*, pages 183–217. Springer, Cham, August 2024.

[BM92] Steven M. Bellovin and Michael Merritt. Encrypted key exchange: Password-based protocols secure against dictionary attacks. In *1992 IEEE Symposium on Security and Privacy*, pages 72–84. IEEE Computer Society Press, May 1992.

[BPR00] Mihir Bellare, David Pointcheval, and Phillip Rogaway. Authenticated key exchange secure against dictionary attacks. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 139–155. Springer, Berlin, Heidelberg, May 2000.

[Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001.

[CDG+18] Jan Camenisch, Manu Drijvers, Tommaso Gagliardoni, Anja Lehmann, and Gregory Neven. The wonderful world of global random oracles. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part I*, volume 10820 of *LNCS*, pages 280–312. Springer, Cham, April / May 2018.

[CHK+05] Ran Canetti, Shai Halevi, Jonathan Katz, Yehuda Lindell, and Philip D. MacKenzie. Universally composable password-based key exchange. In Ronald Cramer, editor, *EUROCRYPT 2005*, volume 3494 of *LNCS*, pages 404–421. Springer, Berlin, Heidelberg, May 2005.

[Cry20] Crypto Forum Research Group. PAKE selection, 2020. https://github.com/cfrg/pake-selection.

[DFG+23] Gareth T. Davies, Sebastian H. Faller, Kai Gellert, Tobias Handirk, Julia Hesse, Máté Horváth, and Tibor Jager. Security analysis of the WhatsApp end-to-end encrypted backup protocol. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part IV*, volume 14084 of *LNCS*, pages 330–361. Springer, Cham, August 2023.

[DHP+18] Pierre-Alain Dupont, Julia Hesse, David Pointcheval, Leonid Reyzin, and Sophia Yakoubov. Fuzzy password-authenticated key exchange. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 393–424. Springer, Cham, April / May 2018.

[DS16] Yuanxi Dai and John P. Steinberger. Indifferentiability of 8-round Feistel networks. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO 2016, Part I*, volume 9814 of *LNCS*, pages 95–120. Springer, Berlin, Heidelberg, August 2016.

[FKL18] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 33–62. Springer, Cham, August 2018.

[FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *CRYPTO'99*, volume 1666 of *LNCS*, pages 537–554. Springer, Berlin, Heidelberg, August 1999.

[GJK21] Yanqi Gu, Stanislaw Jarecki, and Hugo Krawczyk. KHAPE: Asymmetric PAKE from key-hiding key exchange. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 701–730, Virtual Event, August 2021. Springer, Cham.

[GK10] Adam Groce and Jonathan Katz. A new framework for efficient password-based authenticated key exchange. In Ehab Al-Shaer, Angelos D. Keromytis, and Vitaly Shmatikov, editors, *ACM CCS 2010*, pages 516–525. ACM Press, October 2010.

[GL03] Rosario Gennaro and Yehuda Lindell. A framework for password-based authenticated key exchange. In Eli Biham, editor, *EUROCRYPT 2003*, volume 2656 of *LNCS*, pages 524–543. Springer, Berlin, Heidelberg, May 2003.

[HJKW23] Julia Hesse, Stanislaw Jarecki, Hugo Krawczyk, and Christopher Wood. Password-authenticated TLS via OPAQUE and post-handshake authentication. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 98–127. Springer, Cham, April 2023.

[HL19] Björn Haase and Benoît Labrique. AuCPace: Efficient verifier-based PAKE protocol tailored for the IIoT. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pages 1–48, 2019.

[Jar23] Stanislaw Jarecki. Randomized half-ideal cipher on groups with applications to UC (a)PAKE. https://www.youtube.com/watch?v=GL4m7StDsPg, 2023. Talk at EUROCRYPT 2023.

[JKK14] Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. Round-optimal password-protected secret sharing and T-PAKE in the password-only model. In Palash Sarkar and Tetsu Iwata, editors, *ASIACRYPT 2014, Part II*, volume 8874 of *LNCS*, pages 233–253. Springer, Berlin, Heidelberg, December 2014.

[JKKX16] Stanislaw Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. Highly-efficient and composable password-protected secret sharing (or: How to protect your bitcoin wallet online). In *2016 IEEE European Symposium on Security and Privacy*, pages 276–291. IEEE Computer Society Press, March 2016.

[JKKX17] Stanislaw Jarecki, Aggelos Kiayias, Hugo Krawczyk, and Jiayu Xu. TOPPSS: Cost-minimal password-protected secret sharing based on threshold OPRF. In Dieter Gollmann, Atsuko Miyaji, and Hiroaki Kikuchi, editors, *ACNS 17International Conference on Applied Cryptography and Network Security*, volume 10355 of *LNCS*, pages 39–58. Springer, Cham, July 2017.

[JKSS12] Tibor Jager, Florian Kohlar, Sven Schäge, and Jörg Schwenk. On the security of TLS-DHE in the standard model. In Reihaneh Safavi-Naini and Ran Canetti, editors, *CRYPTO 2012*, volume 7417 of *LNCS*, pages 273–293. Springer, Berlin, Heidelberg, August 2012.

[JKX18] Stanislaw Jarecki, Hugo Krawczyk, and Jiayu Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 456–486. Springer, Cham, April / May 2018.

[KV11] Jonathan Katz and Vinod Vaikuntanathan. Round-optimal password-based authenticated key exchange. In Yuval Ishai, editor, *TCC 2011*, volume 6597 of *LNCS*, pages 293–310. Springer, Berlin, Heidelberg, March 2011.

[LLHG23] Xiangyu Liu, Shengli Liu, Shuai Han, and Dawu Gu. EKE meets tight security in the Universally Composable framework. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 685–713. Springer, Cham, May 2023.

[MRR20] Ian McQuoid, Mike Rosulek, and Lawrence Roy. Minimal symmetric PAKE and 1-out-of-N OT from programmable-once public functions. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 425–442. ACM Press, November 2020.

[MX23] Ian McQuoid and Jiayu Xu. An efficient strong asymmetric PAKE compiler instantiable from group actions. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part VIII*, volume 14445 of *LNCS*, pages 176–207. Springer, Singapore, December 2023.

[Nat23] National Institute of Standards and Technology. Module-lattice-based key-encapsulation mechanism standard. Technical Report Federal Information Processing Standards Publications (FIPS PUBS) 203, Initial Public Draft, U.S. Department of Commerce, Washington, D.C., 2023.

[Ode20] Oded Goldreich. On folklore in TOC, 2020. https://www.wisdom.weizmann.ac.il/~oded/etc.html#op25.

[PX23] Chris Peikert and Jiayu Xu. Classical and quantum security of elliptic curve VRF, via relative indifferentiability. In Mike Rosulek, editor, *CT-RSA 2023*, volume 13871 of *LNCS*, pages 84–112. Springer, Cham, April 2023.

[RX23] Lawrence Roy and Jiayu Xu. A universally composable PAKE with zero communication cost - (and why it shouldn't be considered UC-secure). In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 714–743. Springer, Cham, May 2023.

[SAB+22] Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022.

[SGJ23] Bruno Freitas Dos Santos, Yanqi Gu, and Stanislaw Jarecki. Randomized half-ideal cipher on groups with applications to UC (a)PAKE. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 128–156. Springer, Cham, April 2023.

[SGJK22] Bruno Freitas Dos Santos, Yanqi Gu, Stanislaw Jarecki, and Hugo Krawczyk. Asymmetric PAKE with low computation and communication. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 127–156. Springer, Cham, May / June 2022.

[Sho20] Victor Shoup. Security analysis of SPAKE2+. https://www.youtube.com/watch?v=IAUhBRr8Rgc&t=1379s, 2020. Talk at TCC 2020.

[Unr10] Dominique Unruh. Universally composable quantum multi-party computation. In Henri Gilbert, editor, *EUROCRYPT 2010*, volume 6110 of *LNCS*, pages 486–505. Springer, Berlin, Heidelberg, May / June 2010.

[Xag22] Keita Xagawa. Anonymity of NIST PQC round 3 KEMs. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 551–581. Springer, Cham, May / June 2022.

A Additional Attacks and Subtleties

In Sect. 3 we presented three attacks on (O)EKE, showing that the underlying KA protocol must satisfy the notions of strong pseudorandomness (for EKE), pseudorandom non-malleability (for EKE), and collision resistance (for OEKE). In this section we present four more attacks revealing further subtleties in the security of (O)EKE; in particular, Sect. A.3 shows that pseudorandom non-malleability is also needed for OEKE, and Sect. A.4 shows that EKE using HIC or POPF only realizes a weaker UC PAKE functionality (no matter what the underlying KA protocol is).

A.1 Further Subtleties in EKE with Hashed Diffie-Hellman under CDH and DDH

Recall that in the attack on EKE in Sect. 3.2, the adversary passes the P-to-P' message but modifies the P'-to-P message, and causes the key of P to be the square of the key of P'. This issue appears to go away if we replace the output key g^{ab} with $H(g^{ab})$, where H is a hash function (potentially an RO). Indeed, since hashed Diffie-Hellman is secure under CDH/DDH in the ROM, one might conjecture that CDH/DDH suffices for the UC-security of EKE with this KA protocol. While this is true, close scrutiny reveals that there are further subtleties involving the *tightness* of the security analysis.

Consider a generalization of the attack in Sect. 3.2: the adversary (that correctly guesses pw) passes $\phi = \mathcal{E}(\mathsf{pw}, g^a)$ from P to P' (so the session key of P' is $H(g^{ab})$), and replaces the P'-to-P message $\phi' = \mathcal{E}(\mathsf{pw}, g^b)$ with $(\phi')^* = \mathcal{E}(\mathsf{pw}, X)$, where $X \neq g^b$ is a group element of the adversary's choice. Then the session key of P is $H(X^a)$. The UC-security of the protocol requires $H(g^{ab})$ to be pseudorandom even given $H(X^a)$ (because in the ideal world the simulator cannot compromise the P' session, so the session key of P', $H(g^{ab})$, is independent of everything else; cf. Sect. 3.2). In other words, UC-security implies the hardness of following problem: given g^a, g^b (where $a, b \leftarrow \mathbb{Z}_p$), the adversary outputs a group element $X \neq g^b$ and receives $H(X^a)$, and needs to distinguish $H(g^{ab})$ from a random string.

This is the Oracle Diffie-Hellman (ODH) assumption, except that the oracle $H_a(\cdot)$ can be queried only once. (ODH says that given g^a, g^b and access to $H_a(\cdot)$ which on input $X \neq g^b$ outputs $H(X^a)$, it is hard to distinguish $H(g^{ab})$ from random.) Henceforth we call this assumption *1-query ODH*. While 1-query ODH is equivalent to CDH in the ROM (i.e., assuming that H is an RO), if DDH is hard the reduction to CDH loses a factor of $\Theta(q^2)$ where q is the number of the adversary's H queries. Indeed, an adversary on g^a, g^b can sample a random integer $r \leftarrow \mathbb{Z}_p$, make q queries to H including $h_1 := H(g^{ar})$, output $X = g^r$ and receive h_2, check if $h_1 = h_2$, and abort if not. To simulate this, the reduction to CDH must make a guess on which H query is g^{ar} and lose a factor of $\Theta(q)$, since it only sees g^a and $X = g^r$. Next, assuming the guess is correct, this essentially becomes reducing the security of hashed Diffie-Hellman to CDH — where the reduction needs to make a second guess on which H query is g^{ab}, losing another factor of $\Theta(q)$.

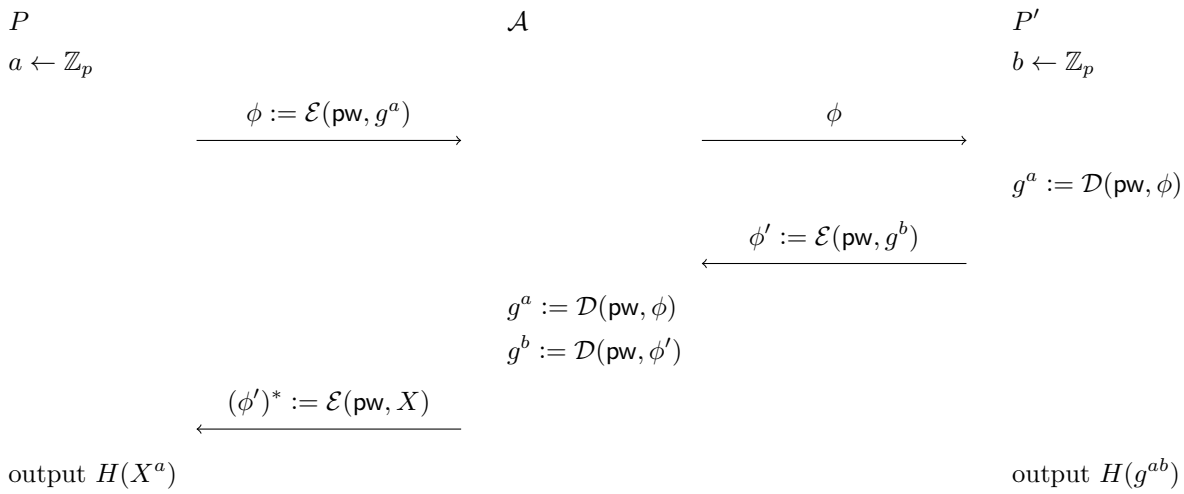

Figure 21: Attack on EKE with hashed Diffie-Hellman. $\mathcal{A}$ only guesses pw in the second round. UC-security requires $H(g^{ab})$ to be pseudorandom even given $H(X^a)$, which is 1-query ODH. Assuming $X = g^r$, $\mathcal{A}$ can make q queries to H including $H((g^a)^r)$, and $\mathcal{Z}$ can check for consistency with the output of P; the reduction to CDH/DDH needs to make a guess on which H query is $H(g^{ar})$

The above shows that if we instantiate EKE with hashed Diffie-Hellman, there is a quadratic security loss while reducing to the CDH assumption. This was first briefly observed in [LLHG23] (and in fact is the starting point of that paper); however, [LLHG23] presents the reduction having to guess the $H(g^{ab})$ query and having to guess the $H(g^{ar})$ query as two *separate* issues, while in fact both of them can appear in the *same* session, causing a quadratic loss.

We note that if the reduction has access to a DH oracle, then both guesses can be replaced by going over all H queries and checking which one is the "right" query; in other words, tight security can be achieved if we reduce to the GDH assumption. Furthermore, since hashed Diffie-Hellman is tightly secure under the DDH assumption, a reduction to DDH only needs to make the first guess, which incurs a $\Theta(q)$ security loss.

reduce to...	first guess	second guess	overall loss
CDH	✓	✓	$\Theta(q^2)$
DDH	✓		$\Theta(q)$
GDH			none

Table 12: Security loss of reducing 1-query ODH to various assumptions

The issue above persists even if the hashed Diffie-Hellman KA includes the entire transcript in the final hash, i.e., the key is $H(g^a, g^b, g^{ab})$ rather than $H(g^{ab})$. The adversary would make $\Theta(q)$ $H(g^a, g^r, \star)$ queries including $h_1 := H(g^a, g^r, g^{ar})$, and after it outputs $X = g^r$ and receives h_2, it can still check if $h_1 = h_2$. To simulate the experiment, the reduction again has to make a guess on which H query contains g^{ar}, even though it knows both g^a and g^r. This attack also naturally extends to the case where the PAKE session key is $H(\phi, \phi', g^{ab})$ where $\phi = \mathcal{E}(\mathsf{pw}, g^a)$ and $\phi' = \mathcal{E}(\mathsf{pw}, g^b)$.

Remark A.1. *The concrete security loss under CDH is slightly lower if the entire transcript is hashed, since the adversary's best strategy is to make $q/2$ $H(g^a, g^b, \star)$ queries and $q/2$ $H(g^a, g^r, \star)$*

queries, and both of the reduction's guesses choose one of $q/2$ queries at random — incurring a $q^2/4$ loss.[26] *By contrast, if the key is $H(g^{ab})$, then the reduction needs to guess over q queries on which one would be $H(g^{ab})$ (say it is the i-th query) and then guess over i possibilities: which of the first $i-1$ queries would be $H(g^{ar})$, or none of them would be $H(g^{ar})$ — so the loss is $\sum_{i=1}^{q} i = q(q+1)/2$. (Note that at the i-th query the reduction can output the query input and stop simulating the experiment, so if the $H(g^{ar})$ query happens after that, it does not matter which exact query is $H(g^{ar})$. However, the reduction does need to guess if the $H(g^{ar})$ query would happen before or after the $H(g^{ab})$ query.) Still, the loss is $\Theta(q^2)$ in both cases.*

Remark A.2. *[BFGJ17] presents a comprehensive study of a large number of variants of the ODH assumption, including 1-query ODH (called sn-PRF-ODH therein*[27]*). However, their main result about 1-query ODH is in the standard model, where the RO $H(K)$ is replaced by a PRF $\mathsf{PRF}_K(x)$ (for some adversarially chosen x).*

Flaw in [BCP+23]. We now show the flaw in the proof of [BCP+23, Theorem 1]. This is subtle so let us proceed slowly. [BCP+23] presents a *general* statement using any 2-round KA protocol that is secure and pseudorandom (called a KEM that is indistinguishable, fuzzy, and anonymous therein), and the PAKE session key is an RO H of the PAKE transcript and the KA key (see [BCP+23, Fig.5]). When instantiated with Diffie-Hellman, it becomes the "reduce to DDH" case in Table 12, so the reduction to DDH needs to make a guess over all H queries. [BCP+23] uses q_H to denote the number of H queries[28] and $\mathsf{Adv}^{\mathrm{ind}}_{\mathrm{KEM}}(t)$ to denote the adversary's advantage against KA security; using these notations, there should be a

$$q_H \cdot \mathsf{Adv}^{\mathrm{ind}}_{\mathrm{KEM}}(t)$$

additive term in the overall distinguishing advantage of the environment. However, such a term does not appear in [BCP+23, Theorem 1]. What exactly goes wrong in the proof of this theorem?

Let us first recall the attacking scenario that causes the security loss above. The adversary passes $\phi = \mathcal{E}(\mathsf{pw}, g^a)$ from P to P'; after that, P' sends $\phi' = \mathcal{E}(\mathsf{pw}, g^b)$ aimed at P and outputs its session key $H(\phi, \phi', g^{ab})$. Upon receiving ϕ', the adversary chooses $r \leftarrow \mathbb{Z}_p$ and computes $(\phi')^* = \mathcal{E}(\mathsf{pw}, g^r)$ (here the adversary needs to know pw), gets g^a by decrypting ϕ and makes $\Theta(q)$ $H(\phi, (\phi')^*, \star)$ queries including $H(\phi, (\phi')^*, (g^a)^r)$, and sends $(\phi')^*$ to P. After that, P outputs $H(\phi, (\phi')^*, g^{ar})$ and the environment can check if it matches the H query, and aborts if not. (We stress that the primary goal of this attack is to cause the reduction to fail, rather than to actually distinguish between the real world and the ideal world.)

The relevant hybrid in the proof is game $\mathbf{G}_{6.1}$ on pp.29–30, which says

> *On Bob's side: Upon receiving* **Epk** *from an honest Alice, instead of setting $SK \leftarrow H(\mathsf{ssid}, P_i, P_j, \mathbf{Epk}, \mathbf{Ec}, K)$, if $\mathtt{SamePwd}(\mathsf{ssid}, P_i, P_j) = \mathsf{true}$, one sets $K' \leftarrow H^*_K(\mathsf{ssid}, \mathsf{success})$ [...] and updates the definition $SK \leftarrow H(\mathsf{ssid}, P_i, P_j, \mathbf{Epk}, \mathbf{Ec}, K')$.*

In our terms (using the specific Diffie-Hellman KA), this means

> *On the side of P', upon receiving ϕ from P (passed by the adversary without modification), instead of setting the key of P' as $H(\phi, \phi', g^{ab})$, if the passwords of P and P' are equal, one sets the key of P' as $H(\phi, \phi', K')$ where $K' \leftarrow \mathbb{G}$.*

[26] Here we assume q is even; if q is odd, the loss is $[(q+1)/2] \cdot [(q-1)/2] = (q^2-1)/4$.

[27] [BFGJ17] considers all cases where the adversary may or may not have (a limited or unlimited number of) access to the "left oracle" $H_a(\cdot)$ that on $X \neq g^b$ computes $H(X^a)$ and the "right oracle" $H_b(\cdot)$ that on $X \neq g^a$ computes $H(X^b)$. In 1-query ODH there is a single query to $H_a(\cdot)$ and no query to $H_b(\cdot)$, so it is called "sn".

[28] In fact [BCP+23] uses the notation q_H without defining it, but from the context it is clear that q_H is the number of H queries.

(Note that up to game $\mathbf{G}_{6.1}$, there is no change on the side of P in our attacking scenario; in particular, the paragraph "On Alice's side" below specifies that "one keeps" the key of P.) The subsequent analysis says "we can simply successively replace $[(g^a, g^b, g^{ab})]$ with $[(g^a, g^b, K')]$, using the indistinguishability of the KEM [the DDH assumption]: the gap is bounded by $q'_{D_1} \cdot \mathsf{Adv}^{\mathrm{ind}}_{\mathrm{KEM}}(t)$." (The text in brackets is a translation to our terms.)

While this appears to be a straightforward reduction to DDH, the problem is that the reduction, given (g^a, g^b, g^{ab}) or (g^a, g^b, K'), can embed g^{ab} or K' while computing the session key of P', but it *must simulate the rest of the experiment — that is, after P' outputs — which involves g^a*. The rest of the experiment includes the following: the adversary chooses $r \leftarrow \mathbb{Z}_p$ and computes $(\phi')^* = \mathcal{E}(\mathsf{pw}, g^r)$, makes $\Theta(q)$ $H(\phi, (\phi')^*, \star)$ queries including $H(\phi, (\phi')^*, g^{ar})$, and sends $(\phi')^*$ to P. After that, the environment can check if the output of P is $H(\phi, (\phi')^*, g^{ar})$. *While the simulator knows a, the reduction does not (since g^a is part of its DDH challenge), so g^{ar} looks random to the reduction even though it knows both g^a and g^r. Hence, in order for the output of P to be $H(\phi, \phi^*, g^{ar})$, the reduction must guess over all H queries and lose a factor of $\Theta(q_H)$.* This subtle point is overlooked in [BCP+23], which misses this $\Theta(q_H)$ factor.

Flaw in [DHP+18]. A very similar (yet more hidden) issue appears in the proof of [DHP+18, Theorem 6], which shows the UC-security of EKE using IC and hashed Diffie-Hellman (like [BCP+23], the entire PAKE transcript is hashed while deriving the session keys) under CDH. The problematic hybrid is game $\mathbf{G}_9$ on p.51, which says

> *$\mathcal{F}$ now generates a random session key upon a first* NewKey *query for an honest party P_i with fresh record (P_i, pw_i) where the other party is also honest, if (at least) one of the following events happens: [...] No output was sent to the other party yet.*

In our terms, this means

> *In the case that the adversary passes the P-to-P' message without modification, P' now outputs a random session key.*

(Note that up to game $\mathbf{G}_9$, there is no change on the side of P in our attacking scenario; in particular, game $\mathbf{G}_5$ deals with the case that the adversary makes an *incorrect* password guess while sending a message to P, and game $\mathbf{G}_6$ deals with the case that the adversary modifies the P-to-P' message.) [DHP+18] then claims the indistinguishability between game $\mathbf{G}_9$ and the previous game in Lemma 13, whose proof is only sketched and says "it is similar to the proof of Lemma 12 [under game $\mathbf{G}_5$]" and that the reduction should simply embed g^a, g^b as its CDH challenge.

However, the proofs of Lemma 13 and Lemma 12 are actually quite different. Game $\mathbf{G}_5$ says that if the adversary modifies the P'-to-P message $\phi' = \mathcal{E}(\mathsf{pw}, g^b)$ to another $(\phi')^* = \mathcal{E}(\mathsf{pw}^*, \star)$, then P outputs a random session key if $\mathsf{pw}^* \neq \mathsf{pw}$. The reduction to CDH here is relatively simple: since $\mathcal{D}(\mathsf{pw}, (\phi')^*)$ is some g^{b^*} where b^* is unknown to the adversary, the reduction can use it to embed a CDH challenge. (Of course, since the session key is a hash of the KA key, the reduction needs to guess over all of the adversary's H queries and lose a factor of q_H — which the proof correctly identifies.) However, just as what we have seen about the flaw in [BCP+23], in game $\mathbf{G}_9$ the reduction needs to simulate the rest of the experiment even after P' outputs its session key, which forces the reduction to make another guess over all H queries on which one is $H(\phi, (\phi')^*, g^{ar})$ (where the adversary sends $(\phi')^* = \mathcal{E}(\mathsf{pw}, g^r)$ to P). This part of the reduction is missing in [DHP+18].

How serious are these flaws? The flaws in the security proofs in [BCP+23, DHP+18] do not affect the theorem statements much: the concrete security bound in [BCP+23, Theorem 1] misses an

additive factor, and technically speaking the statement of [DHP+18, Theorem 6] is correct (it only claims the UC-security of EKE without presenting any concrete bound). While our other attacks reveal more serious issues in (O)EKE security analyses, the attack in this section shows that there are significant issues in the *security proofs* of EKE: not only are there flaws in the reductions, but they actually miss *an entire class of adversaries*. In the specific case of hashed Diffie-Hellman, a similar attack on TLS has been noticed in prior works [JKSS12] (in fact this is one of the main motivations of studying the 1-query ODH assumption), yet this attack has been overlooked in the context of PAKE, and we believe we are the first to point out the necessity of 1-query ODH in the security of EKE.

A.2 OEKE-PRF with Plain Diffie-Hellman Is Not Necessarily Secure

We now show that the OEKE-PRF protocol is also not necessarily UC-secure if we use plain Diffie-Hellman as the underlying KA protocol, due to an attack similar to that in Sect. 3.2. The adversary passes the P-to-P' message $\mathcal{E}(\mathsf{pw}, g^a)$ without modification, causing P' to output $\mathsf{PRF}_{g^{ab}}(0)$. If PRF is such that $\mathsf{PRF}_{k^2}(x)$ is predictable from $\mathsf{PRF}_k(x)$ for a random $k \leftarrow \mathbb{G}$,[29] then the adversary, upon seeing $B = g^b$ and $\tau' = \mathsf{PRF}_{g^{ab}}(1)$ from P', can send $B^* = B^2$ and $\tau^* = \mathsf{PRF}_{g^{2ab}}(1)$ to P; the check of P will pass and P will output $\mathsf{PRF}_{g^{2ab}}(0)$ — again, the session keys of P and P' are correlated. Note that unlike the attack on EKE in Sect. 3.2, here the adversary does not even need to know the password.

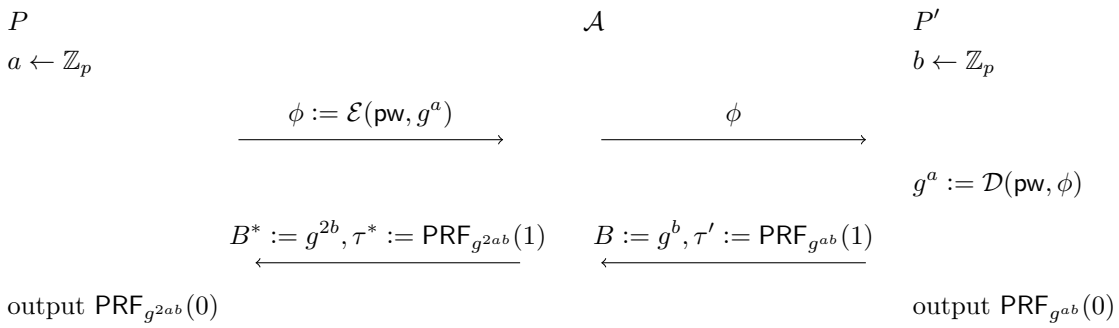

Figure 22: Attack on OEKE-PRF with plain Diffie-Hellman, assuming $\mathsf{PRF}_{k^2}(x)$ is predictable from $\mathsf{PRF}_k(x)$. $\mathcal{A}$ does not need to know pw

The above shows that OEKE-PRF with a general PRF is insecure when instantiated with plain Diffie-Hellman; rather, the PRF has to satisfy the requirement that $\mathsf{PRF}_{k^2}(x)$ is pseudorandom even given $\mathsf{PRF}_k(x)$. This condition is trivially met if $\mathsf{PRF}_k(x) = H(k, x)$ (where H is an RO), i.e., OEKE-RO is not subject to this attack. (This attack does not invalidate any existing results on the security of OEKE.)

A.3 EKE and OEKE Are Insecure If the Underlying KA Is Not Pseudorandom Non-Malleable

The attack in Sect. 3.2 shows that some form of non-malleability is necessary for the security of EKE, but it does not really show that *pseudorandom* non-malleability (Thm. 3.6) is needed in EKE, or any form of non-malleability is needed in OEKE.

[29] Note that the security definition of PRF requires that the key be chosen at random, and says nothing about the PRF's behavior under two correlated keys.

Here by non-malleability, we mean that it satisfies a variant of Thm. 3.6 where B is sampled as a real KA message in both experiments, i.e., the following two distributions are indistinguishable:

$a \leftarrow \mathcal{R}$	$a \leftarrow \mathcal{R}$
$b \leftarrow \mathcal{R}$	$b \leftarrow \mathcal{R}$
$A := \mathsf{msg}_1(a)$	$A := \mathsf{msg}_1(a)$
$B := \mathsf{msg}_2(b, A)$	$B := \mathsf{msg}_2(b, A)$
$K' := \mathsf{key}_2(b, A)$	$K' \leftarrow \mathcal{K}$
$B^* \leftarrow \mathcal{A}(A, B, K')$	$B^* \leftarrow \mathcal{A}(A, B, K')$
abort if $B^* = B$	abort if $B^* = B$
$K := \mathsf{key}_1(a, B^*)$	$K := \mathsf{key}_1(a, B^*)$
output K to $\mathcal{A}$	output K to $\mathcal{A}$

In contrast, in the security experiment of Thm. 3.6, the experiment on the right also changes B to random — which is why we call this property *pseudorandom* non-malleability. Pseudorandom non-malleability is needed for the following reason. Suppose the adversary passes the P-to-P' message ϕ without modification, and on ϕ' from P' queries $B^* := \mathcal{D}(\mathsf{pw}^*, \phi')$ on some password guess pw^*. In the real world B^* is the "real" $\mathsf{msg}_2(b, A)$ if $\mathsf{pw}^* = \mathsf{pw}$, and uniformly random otherwise. However, in the ideal world *the simulator does not know whether* pw^* *is the correct password at this point*, so its simulation must be indistinguishable from both cases. By transitivity, the adversary must not distinguish $B := \mathsf{msg}_2(b, A)$ from $B \leftarrow \mathcal{M}_2$. In other words, the *joint distribution* of B and K' must be indistinguishable from random, even to an adversary that gets to modify ϕ' and see what session key P then generates.

Of course, in the Diffie-Hellman example there is no difference between computing B as $B := \mathsf{msg}_2(b, A)$ and sampling $B \leftarrow \mathcal{M}_2$, as the messages in Diffie-Hellman KA are uniform. But in the general case there is a difference (even assuming the KA protocol satisfies strong pseudorandomness); indeed, in this section we present a counterexample showing that KA security, strong pseudorandomness, and non-malleability combined do not imply pseudorandom non-malleability.

We then show that both EKE and OEKE with our KA protocol are insecure, demonstrating why pseudorandom non-malleability is necessary for the security of (O)EKE.

Counterexample. The counterexample is similar to the one in Sect. 3.3, using an additional field of the message to detect improperly generated or modified messages.

$$\mathsf{msg}_1(a) = g^a$$
$$\mathsf{msg}_2(b, A) = (g^b, H_0(A^b))$$
$$\mathsf{key}_1(a, (B_0, B_1)) = \begin{cases} H_1(B_0^a) & \text{if } B_1 = H_0(B_0^a) \\ H_2(g^a) & \text{if } B_1 = H_0(B_0^a) + 1 \\ H_3(a, B) & \text{otherwise} \end{cases}$$
$$\mathsf{key}_2(b, A) = H_1(A^b)$$

(where $H_0, H_1, H_2 \colon \mathbb{G} \to \{0,1\}^\kappa, H_3 \colon \mathbb{Z}_p \times \mathbb{G} \to \{0,1\}^\kappa$ are ROs).

Correctness, security, and pseudorandomness all hold assuming CDH, for the same reasons as with the previous counterexample. Additionally, strong pseudorandomness holds because a uniformly random B triggers the $H_3(a, B)$ case with overwhelming probability, which is indistinguishable from random because a cannot be guessed by the adversary (without solving discrete log); and a real B

triggers the $H_1(B_0^a)$ case, which is also indistinguishable from random by CDH.[30]

Now let's see why pseudorandom non-malleability fails. The adversary $\mathcal{A}$ receives (A, B, K') (where $B = (B_0, B_1)$) from the challenger, and outputs $B^* = (B_0, B_1 + 1)$. The challenger then sends $K = \mathsf{key}_1(a, B^*)$. In the real distribution this will trigger the second case and so $K = H_2(A)$, while in the random distribution it will trigger the third case and output $H_3(a, B)$ — because in the random distribution B_1 is uniform, so $B_1 + 1$ is as well, so both of them have negligible chance of equaling $H_0(B_0^a)$. To distinguish, $\mathcal{A}$ simply checks whether $K = H_2(A)$.

Attacks on EKE and OEKE. This counterexample also works on the whole EKE protocol, not just the pseudorandom non-malleability definition. Below we illustrate the attack, where the adversary behaves similarly to the pseudorandom non-malleability experiment: it passes the P-to-P' message without modification, and on the P'-to-P message ϕ', the adversary decrypts using pw (the password of P) and obtains (B_0, B_1), then encrypts $(B_0, B_1 + 1)$. If pw' (the password of P') is equal to pw, then $(B_0, B_1) = (g^b, H_0(g^{ab}))$, so key_1 will enter the second case and P will output the predictable value $H_2(g^a)$; otherwise (B_0, B_1) is uniformly random, so with overwhelming probability key_1 will enter the third case and P will output $H_3(a, (B_0, B_1 + 1))$. In other words, the enviroment/adversary can check if the passwords of P and P' match by modifying the P'-to-P message and observing the session key of P only — which is not allowed by the UC-security of PAKE (throughout the entire experiment the simulator never knows pw', so it cannot make any decision based on whether $\mathsf{pw} = \mathsf{pw}'$ or not). Note that this attack requires the adversary to know pw but not pw'; and unlike other attacks where the adversary passes the first message and changes the second (Sects. A.2, A.4 and 3.2), here the environment does not need to observe the session key of P'.

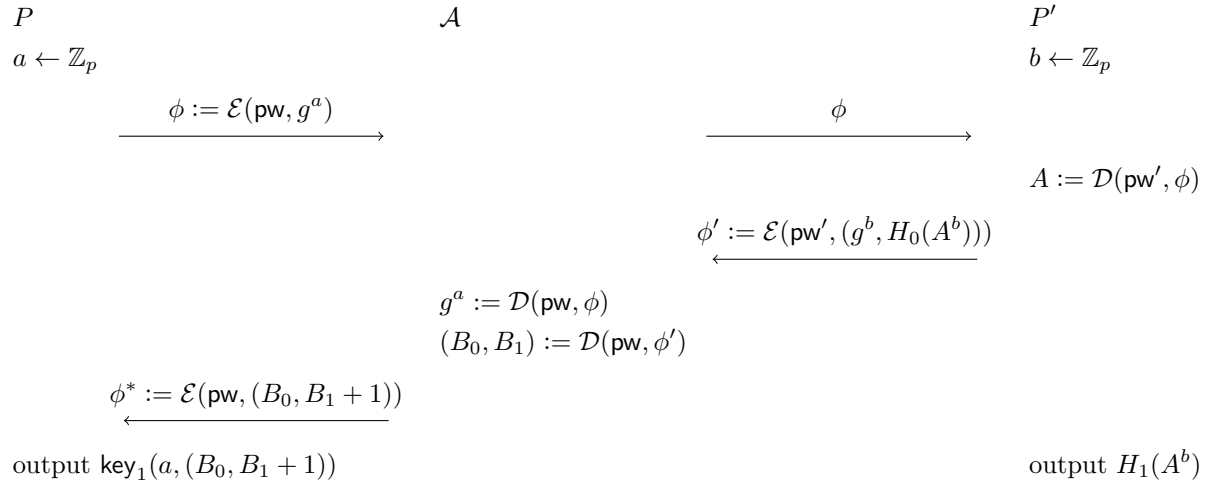

Figure 23: Attack on EKE with the KA protocol in this section (which is not pseudorandom non-malleable). $\mathcal{A}$ causes P to output the predictable $H_2(g^a)$ if $\mathsf{pw} = \mathsf{pw}'$ and the unpredictable $H_3(a, (B_0, B_1 + 1))$ otherwise, without attacking the P' session. This is not allowed by UC-security

A similar attack works against OEKE, with the adversary again modifying only the second (P'-to-P) message. For OEKE, the adversary receives (B_0, B_1, τ') from P', and changes them to

[30]The KA protocol can be additionally made collision-resistant by changing the ROs H_1, H_2, H_3 to have 3κ-bit outputs and requiring B_0 to not be the identity element, since all three cases of key_1 hash something that uniquely identifies A.

$(B_0, B_1 + 1, \tau^*)$, where $K^* \| \tau^* := H_2(g^a)$ (this requires the adversary to know pw and decrypt the P-to-P' message). If $\mathsf{pw} = \mathsf{pw}'$, we have $\mathsf{key}_1(a, (B_0, B_1 + 1)) = H_2(g^a)$, so $\tau^* = \tau$ and P will output K^*. If $\mathsf{pw} \neq \mathsf{pw}'$, with overwhelming probability $\mathsf{key}_1(a, (B_0, B_1 + 1)) = H_3(a, (B_0, B_1))$ which has negligible probability of matching τ^*, so the session key of P is uniformly random with overwhelming probability. Again, the enviroment/adversary can check if the passwords of P and P' match by modifying the P'-to-P message and observing the session key of P only.

Non-malleability. Given how badly EKE and OEKE break when using this KA protocol, it might seem that non-malleability is the problem. However, our KA protocol does satisfy non-malleability under CDH, as we will prove now. It is just pseudorandom non-malleability that is the problem.

The reduction to CDH, on challenge $A = g^a$ and $B = g^b$, samples $K' \leftarrow \{0,1\}^\kappa$ and sends (A, B, K') to the non-malleability adversary $\mathcal{A}$. Next, $\mathcal{A}$ outputs some $B^* \neq B$, and the reduction responds with some K according to the following rules:

- If $B_0^* = B_0$ and $B_1^* = B_1 + 1$, respond with $H_2(A)$.
- If $B_1^* = H_0(X)$ for some past $H_0(X)$ query, sample a random bit $d \leftarrow \{0,1\}$ to decide whether to treat it as a correct guess. If $d = 1$, output $H_1(X)$.
- If $B_1^* = H_0(X) + 1$ for some past $H_0(X)$ query, sample a random bit $d \leftarrow \{0,1\}$ to decide whether to treat it as a correct guess. If $d = 1$, output $H_2(A)$.
- If none of these cases matches, or if $d = 0$, output $K \leftarrow \{0,1\}^\kappa$.

So far, we have a nearly perfect simulation of the random K' distribution. The only exceptions are that $\mathcal{A}$ could guess a in a H_3-query, but we could then use a to solve CDH; and the (negligible) chance that there is a collision in H_0 or that $\mathcal{A}$ will find a new H_0 pre-image to B_1^* after it was already sent to the reduction.

In order for $\mathcal{A}$ to distinguish this from the real K' distribution, it must make a query to the preimage $H_1(g^{ab})$. Therefore, the reduction guesses a random $H_1(X)$ query, and outputs X in the CDH experiment. If $\mathcal{A}$ has advantage $\mathsf{Adv}_{\mathrm{NM}}$ and makes q queries to H_1, then the reduction has advantage

$$\mathsf{Adv}_{\mathrm{CDH}} \geq \frac{1}{2q}\mathsf{Adv}_{\mathrm{NM}} - \mathrm{negl.},$$

since the reduction wins when both guesses (d and the H_1 query) are correct, which have probabilities $1/2$ and $1/q$, respectively.

The above shows a crucial difference between non-malleability and pseudorandom non-malleability: non-malleability can be obtained by taking any secure and pseudorandom KA protocol and applying an RO hash to the key, although the reduction is loose (an extension of non-malleability of hashed Diffie-Hellman under DDH); whereas pseudorandom non-malleability is not implied by any existing properties and must be presented as a property on its own. This in particular means that [BCP+23, Theorem 1], [BCP+23, Theorem 2], and [SGJ23, Theorem 3] are false: while the (O)EKE protocols there hashes the KA key at the end, this only guarantees non-malleability, and the theorems do not mention pseudorandom non-malleability which is actually needed.

A.4 EKE Using HIC/POPF Only Realizes a Weaker UC Functionality

All attacks we have discussed work for (O)EKE no matter whether the underlying encryption scheme is IC, HIC or POPF; in this section we present an attack that only applies to EKE using HIC and POPF. Here we are satisfied with giving an informal argument, and do not prove the UC-insecurity

(like what we did in Thm. 3.4). Like the attack in Sect. 3.2, the attack in this section was first mentioned in Jarecki's EUROCRYPT talk [Jar23] and is not our original work. The modification of the protocol (item (3) below) was also suggested in the talk. The modification of the UC PAKE functionality (item (2) below) is ours.

A key difference between HIC/POPF and IC is that the encryption algorithm in the latter is deterministic, whereas the former is *randomized*; that is, in HIC/POPF the output of $\mathcal{E}(\mathsf{pw}, m; r)$ depends on the randomness r used in the algorithm. This yields the following attack: the adversary passes the P-to-P' message without modification, causing P' to output session key K'. Then on the P'-to-P message $\phi' = \mathcal{E}(\mathsf{pw}, B; r')$, the adversary sets $\mathsf{pw}^* = \mathsf{pw}$ with probability 1/2 and $\mathsf{pw}^* \neq \mathsf{pw}$ with probability 1/2, and decrypts and re-encrypts using pw^*. That is, the adversary computes $B^* = \mathcal{D}(\mathsf{pw}^*, \phi')$ and sends

$$(\phi')^* := \mathcal{E}(\mathsf{pw}^*, B^*; r^*)$$

to P, where r^* is a fresh randomness sampled from the corresponding space. Let K be the session key of P; K is equal to K' with probability 1/2 (if $\mathsf{pw}^* = \mathsf{pw}$) and independent of K' with probability 1/2 (if $\mathsf{pw}^* \neq \mathsf{pw}$).

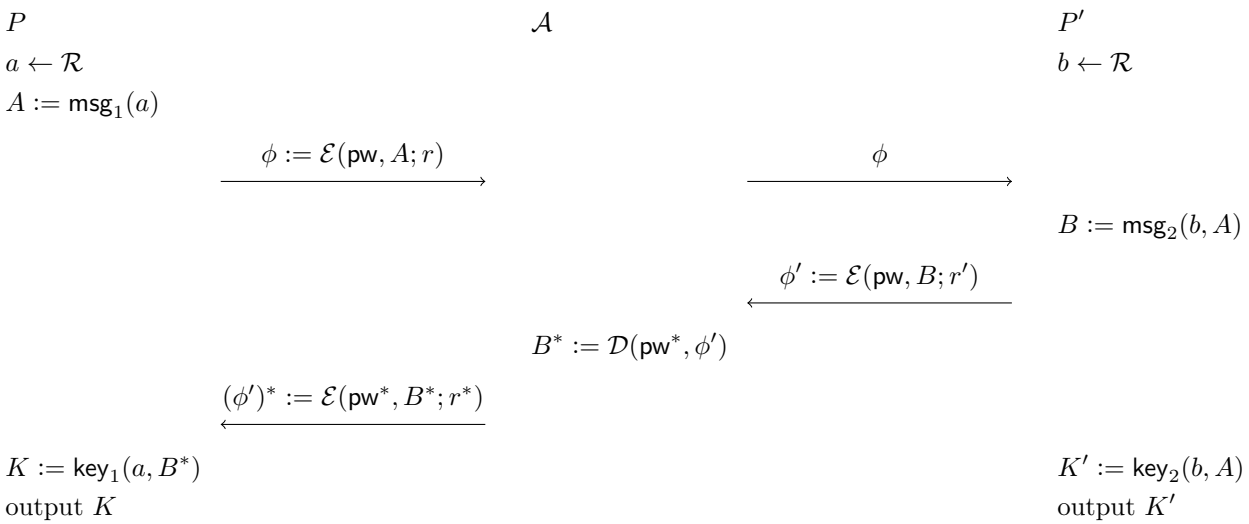

Figure 24: Attack on EKE using HIC/POPF (with any KA protocol). $\mathcal{A}$ sets $\mathsf{pw}^* = \mathsf{pw}$ with probability 1/2 and $\mathsf{pw}^* \neq \mathsf{pw}$ with probability 1/2. Either way $(\phi')^* \neq \phi'$ with overwhelming probability due to the fresh r^*, but in the former case $K = K'$ and in the latter case K and K' are independent. This "second round conditional password test" is not allowed by standard UC-security

The simulator can extract pw^* in the second round (which happens after P' outputs K'), but it does not know whether $\mathsf{pw}^* = \mathsf{pw}$ unless and until it sends $(\mathsf{TestPwd}, \mathsf{sid}, P, \mathsf{pw}^*)$ to $\mathcal{F}_{\mathrm{PAKE}}$. The simulator has two options here. If it sends the TestPwd command, then the simulation fails in the case of $\mathsf{pw}^* = \mathsf{pw}$, since (as we argued in Sect. 3.2) K' is independent of the simulator's view, so after compromising the P session, it cannot set K to be equal to K'. If the simulator does not send the TestPwd command, then the simulation fails in the case of $\mathsf{pw}^* \neq \mathsf{pw}$, as the P session is fresh and $\mathcal{F}_{\mathrm{PAKE}}$ will let P output the same session key as P', while the two session keys are independent in the real world. Either way, the simulator fails with probability roughly 1/2. (Note that this is not an issue if we use IC, since the simulator can detect whether $\mathsf{pw}^* = \mathsf{pw}$ by observing whether $(\phi')^* = \phi'$, and send TestPwd only if $\mathsf{pw}^* \neq \mathsf{pw}$. Here the randomization of HIC/POPF ensures that

even if $\mathsf{pw}^* = \mathsf{pw}$, $(\phi')^*$ and ϕ are still independent.)

The above shows that [MRR20, Theorem 10] and [SGJ23, Theorem 2] are false in a manner different from Sects. 3.2 and 3.3, since these two theorems imply that EKE with plain Diffie-Hellman is secure using POPF and HIC, respectively. (In fact the same attack works even if we use hashed Diffie-Hellman, if only the key is hashed.) Closer scrutiny shows that EKE using HIC/POPF realizes a weaker UC functionality, with an additional command TestSamePwd that works in the same way as TestPwd except that no change is made if $\mathsf{pw}^* = \mathsf{pw} = \mathsf{pw}'$ (where pw' is the key of P'). (In the attacking scenario above, this allows the simulation to go through in the case of $\mathsf{pw}^* = \mathsf{pw}$, as the TestSamePwd command does not have any effect and the P session is still fresh, so $\mathcal{F}_{\text{PAKE}}$ will set K to be equal to K'.) We call this modified PAKE functionality *PAKE with same password test*, or $\mathcal{F}_{\text{PAKE-sp}}$.

Of course, another way to get around this issue is to modify the protocol so that the P'-to-P message is included in the final hash, or rather (if we do not want to explicitly use the ROM here) to define the session key as $\mathsf{PRF}_K(\phi')$. We call this modified protocol EKE-PRF. Furthermore, as we noticed above, the attack does not work if IC is used. In this work we present three results:

1. EKE (with the underlying KA satisfying appropriate properties, same below) using IC realizes $\mathcal{F}_{\text{PAKE}}$;
2. EKE using POPF realizes $\mathcal{F}_{\text{PAKE-sp}}$;
3. EKE-PRF using POPF realizes $\mathcal{F}_{\text{PAKE}}$.

Our main result is (3) (Thm. 5.4), and we briefly argue (1) and (2) in Sect. B. For (2), we believe our $\mathcal{F}_{\text{PAKE-sp}}$ functionality might be of independent interest; for (1), we believe it would be helpful to present a (correct) proof for EKE using IC, as this is the most studied version of EKE so far.

B Additional Results on the Security of EKE

In this section we give proof sketches for the security of ("raw") EKE. Recall that in Thm. 5.4 we only proved the UC-security of EKE-PRF (where the session key is $F_K(\phi')$ rather than the KA key K), and that is because EKE only realizes a weaker functionality, $\mathcal{F}_{\text{PAKE-sp}}$, if instantiated with POPF — as demonstrated by the attack in Sect. A.4. Below we argue for two results:

1. EKE using IC realizes $\mathcal{F}_{\text{PAKE}}$; and
2. EKE using POPF realizes $\mathcal{F}_{\text{PAKE-sp}}$,

which correspond to items (1) and (2) in Sect. A.4. In both results, the vast majority of the proof is identical to that of Thm. 5.4, so we only highlight the differences. We begin with (2) since the proof of (1) is almost immediate given the proof of (2).

B.1 EKE Using POPF Realizes $\mathcal{F}_{\text{PAKE-sp}}$

We first present the $\mathcal{F}_{\text{PAKE-sp}}$ functionality in Figure 25. The only difference with $\mathcal{F}_{\text{PAKE}}$ is that the ideal adversary can additionally send a TestSamePwd command, which works exactly as TestPwd except that if $\mathsf{pw}^* = \mathsf{pw} = \mathsf{pw}'$ (i.e., the password guess is correct and the two parties' passwords match) then the functionality does not do anything.

Consider the EKE protocol, which is the EKE-PRF protocol in Figure 17 except that P outputs K instead of $\mathsf{PRF}_K(\phi')$, and P' outputs K' instead of $\mathsf{PRF}_{K'}(\phi')$. We have:

Theorem B.1. *The EKE protocol realizes $\mathcal{F}_{\text{PAKE-sp}}$ in the $(\mathcal{F}_{\text{POPF}}, \mathcal{F}_{\text{EKE-1r}})$-hybrid world.*

- On input $(\mathsf{NewSession}, \mathsf{sid}, P, P', \mathsf{pw}, \mathsf{role})$ from P, send $(\mathsf{NewSession}, \mathsf{sid}, P, P', \mathsf{role})$ to $\mathcal{S}$. Furthermore, if this is the first NewSession message for sid, or this is the second NewSession message for sid and there is a record $\langle P', P, \cdot \rangle$, then record $\langle P, P', \mathsf{pw} \rangle$ and mark it fresh.
- On $(\mathsf{TestPwd}, \mathsf{sid}, P, \mathsf{pw}^*)$ from $\mathcal{S}$, if there is a record $\langle P, P', \mathsf{pw} \rangle$ marked fresh, then do:
  - If $\mathsf{pw}^* = \mathsf{pw}$, then mark the record compromised and send "correct guess" to $\mathcal{S}$.
  - If $\mathsf{pw}^* \neq \mathsf{pw}$, then mark the record interrupted and send "wrong guess" to $\mathcal{S}$.
- On $(\mathsf{TestSamePwd}, \mathsf{sid}, P, \mathsf{pw}^*)$ from $\mathcal{S}$, if there is a record $\langle P, P', \mathsf{pw} \rangle$ marked fresh, then do:
  - If $\mathsf{pw}^* = \mathsf{pw} = \mathsf{pw}'$, then ignore the command (and the record remains fresh).
  - If $\mathsf{pw}^* = \mathsf{pw} \neq \mathsf{pw}'$, then mark the record compromised and send "correct guess" to $\mathcal{S}$.
  - If $\mathsf{pw}^* \neq \mathsf{pw}$, then mark the record interrupted and send "wrong guess" to $\mathcal{S}$.
- On $(\mathsf{NewKey}, \mathsf{sid}, P, K^* \in \{0,1\}^\kappa)$ from $\mathcal{S}$, if there is a record $\langle P, P', \mathsf{pw} \rangle$, and this is the first NewKey message for sid and P, then output (sid, K) to P, where K is defined as follows:
  - If the record is compromised, or either P or P' is corrupted, then set $K := K^*$.
  - If the record is fresh, a key (sid, K') has been output to P', at which time there was a record $\langle P', P, \mathsf{pw} \rangle$ marked fresh, then set $K := K'$.
  - Otherwise sample $K \leftarrow \{0,1\}^\kappa$.

  Finally, mark the record completed.

Figure 25: UC PAKE with same password test functionality $\mathcal{F}_{\text{PAKE-sp}}$

Proof (sketch). The simulator $\mathcal{S}$ is shown in Figure 26.

The only modifications from the EKE-PRF simulator in Figure 18 are:

- In a NewKey command, we replace $\mathsf{PRF}_{K^*}(\phi')$ or $\mathsf{PRF}_{K^*}((\phi')^*)$ with K^*;
- We add case 12(3)(i), where a TestSamePwd command instead of TestPwd is sent.

The first modification is consistent with what changes in the real world, namely parties output K (resp. K') rather than $\mathsf{PRF}_K((\phi')^*)$ (resp. $\mathsf{PRF}_{K'}(\phi')$). For the second modification, recall that TestSamePwd is exactly the same as TestPwd, except that in the case of $(\mathsf{pw}')^* = \mathsf{pw} = \mathsf{pw}'$, $\mathcal{F}_{\text{PAKE-sp}}$ on TestSamePwd does nothing. This means that the only essential difference between the ideal world here and the ideal world in the proof of Thm. 5.4, is that if

$$\mathsf{pw} = \mathsf{pw}' \wedge \mathsf{pw}^* = \perp \wedge (\phi')^* \neq \phi' \wedge B^* = B \wedge (\phi')^* \text{ contains the correct password guess,}$$

we now let P and P' output the same session key. (This is the re-encryption case in Sect. A.4 and the proof of Thm. 5.4.) This again matches what changes in the real world, where in this case the session keys of P and P' are indeed the same.

Given the intuition above, we can easily come up with a series of hybrids that are very similar to those in the proof of Thm. 5.4, with only two modifications:

- Wherever $\mathsf{PRF}_K(\phi')$ (resp. $\mathsf{PRF}_{K'}(\phi')$ or $\mathsf{PRF}_{K^*}(\phi')$) is mentioned, it is replaced by K (resp. K' or K^*). Furthermore, the following hybrids that trivially reduce to the fact that PRF is a PRF are now removed:

Initialize $U := \{\}$ as the set of H-evaluations.
Let $T = \{\}$ be the record of honest POPF evaluations as in $\mathcal{F}_{\text{POPF}}$.

To compute $H(x)$:

1. If there is an entry $(x, y) \in U$ return y.
2. Otherwise sample $y \leftarrow \mathcal{R}$, add (x, y) to U and return y.

On $(\mathsf{NewSession}, \mathsf{sid}, P, P', \text{“initiator”})$ from $\mathcal{F}_{\text{PAKE}}$:

3. Send $(\mathsf{Program}, \mathsf{sid}, P, P')$ from $\mathcal{F}_{\text{EKE-1r}}$ to $\mathcal{A}$.

On $(\mathsf{NewSession}, \mathsf{sid}, P', P, \text{“respondent”})$ from $\mathcal{F}_{\text{PAKE}}$:

4. Send $(\mathsf{SampleResp}, \mathsf{sid}, P', P)$ from $\mathcal{F}_{\text{EKE-1r}}$ to $\mathcal{A}$.

On $(\mathsf{Eval}, \mathsf{sid}, P, P', x)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{EKE-1r}}$:

5. Compute $A = \mathsf{msg}_1(H(x))$ and return A to $\mathcal{A}$.

On $(\mathsf{Deliver}, \mathsf{sid}, P, P', \mathsf{pw}^*, A^*)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{EKE-1r}}$:

6. Send $(\mathsf{Program}, \mathsf{sid})$ from $\mathcal{F}_{\text{POPF}}$ to $\mathcal{A}$ and wait until $\mathcal{A}$ responds with $(\mathsf{Program}, \mathsf{sid}, \phi')$ to $\mathcal{F}_{\text{POPF}}$ such that there is no entry $(\phi', \cdot, \cdot) \in T$.
7. Send (sid, ϕ') from P' to P.
8. If $\mathsf{pw}^* = \bot$, send $(\mathsf{NewKey}, \mathsf{sid}, P', 0^\kappa)$ to $\mathcal{F}_{\text{PAKE}}$.
9. If $\mathsf{pw}^* \neq \bot$ do:
   (1) Send $(\mathsf{TestPwd}, \mathsf{sid}, P', \mathsf{pw}^*)$ to $\mathcal{F}_{\text{PAKE}}$.
   (2) Sample $b \leftarrow \mathcal{R}$ and compute $(K')^* := \mathsf{key}_2(b, A^*)$.
   (3) Send $(\mathsf{NewKey}, \mathsf{sid}, P', (K')^*)$ to $\mathcal{F}_{\text{PAKE}}$.

On (sid, ϕ'^*) from $\mathcal{A}$ to P:

10. If either (1) $\mathsf{pw}^* = \bot \wedge \phi'^* = \phi'$ or (2) $\mathsf{pw}^* \neq \bot \wedge \phi'^* = \phi'$ and the password guess on $\mathsf{Deliver}$ was incorrect, send $(\mathsf{NewKey}, \mathsf{sid}, P, 0^\kappa)$ to $\mathcal{F}_{\text{PAKE}}$.
11. If $\mathsf{pw}^* \neq \bot \wedge \phi'^* = \phi'$ and the password guess on $\mathsf{Deliver}$ was correct:
    (1) Send $(\mathsf{TestPwd}, \mathsf{sid}, P, \mathsf{pw}^*)$ to $\mathcal{F}_{\text{PAKE}}$.
    (2) Compute $K^* = \mathsf{key}_1(H(\mathsf{pw}^*), \mathsf{msg}_2(b, A^*))$ and send $(\mathsf{NewKey}, \mathsf{sid}, P, K^*)$ to $\mathcal{F}_{\text{PAKE}}$.
12. Otherwise (i.e., if $\phi'^* \neq \phi'$ or ϕ' is undefined because no $\mathsf{Deliver}$ message has been sent):
    (1) Send $(\mathsf{Extract}, \mathsf{sid}, \phi'^*)$ from $\mathcal{F}_{\text{POPF}}$ to $\mathcal{A}$.
    (2) On $(\mathsf{Extract}, \mathsf{sid}, \mathsf{pw}'^*, \alpha^*)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{POPF}}$, send $(\mathsf{Eval}, \mathsf{sid}, \phi'^*, \mathsf{pw}'^*)$ from $\mathcal{F}_{\text{POPF}}$ to $\mathcal{A}$.
    (3) On $(\mathsf{Eval}, \mathsf{sid}, B^*)$ from $\mathcal{A}$ to $\mathcal{F}_{\text{POPF}}$,
        (i) If $\mathsf{pw}^* = \bot \wedge (\phi')^* \neq \phi'$, and there has been an $(\mathsf{Eval}, \mathsf{sid}, \phi', (\mathsf{pw}')^*)$ command to $\mathcal{F}_{\text{POPF}}$ whose answer is $(\mathsf{Eval}, \mathsf{sid}, B^*)$, then send $(\mathsf{TestSamePwd}, \mathsf{sid}, P, (\mathsf{pw}')^*)$ to $\mathcal{F}_{\text{PAKE}}$, compute $K^* := \mathsf{key}_1(H((\mathsf{pw}')^*), B^*)$, and send $(\mathsf{NewKey}, \mathsf{sid}, P, K^*)$ to $\mathcal{F}_{\text{PAKE}}$.
        (ii) Otherwise send $(\mathsf{TestPwd}, \mathsf{sid}, P, (\mathsf{pw}')^*)$ to $\mathcal{F}_{\text{PAKE}}$, compute $K^* := \mathsf{key}_1(H((\mathsf{pw}')^*), B^*)$, and send $(\mathsf{NewKey}, \mathsf{sid}, P, K^*)$ to $\mathcal{F}_{\text{PAKE}}$.

Simulation of $\mathcal{F}_{\text{POPF}}$: run the code of $\mathcal{F}_{\text{POPF}}$ as in (Figure 8) except that on $(\mathsf{Eval}, \mathsf{sid}, \phi', \mathsf{pw}^*)$, if the password guess on $\mathsf{Deliver}$ was correct return $B = \mathsf{msg}_2(b, A^*)$.

Figure 26: Simulator $\mathcal{S}$ for the EKE protocol realizing $\mathcal{F}_{\text{PAKE-sp}}$.

- Hybrid 5 which sets the two session keys equal when $K = K'$;
- Hybrid 6 which replaces $\mathsf{PRF}_{K'}$ (where K' is a random string independent of the rest of the experiment) with a random function G'; and
- Hybrid 7 which replaces PRF_K (where K is a random string independent of the rest of the experiment) with a random function G.

- Hybrid 8 in the previous proof — which exactly deals with the re-encryption case above, and the change is that P and P' output session keys $(\mathsf{PRF}_K((\phi')^*), G'(\phi'))$ rather than $(G'((\phi')^*), G'(\phi'))$ — is also removed. The reason is slightly different than the above, though; the key point is that in our setting *both before the hybrid and after the hybrid, the session keys of P and P' are the same*, so this hybrid is not needed anymore.

- Hybrid 9 in the previous proof — which changes K from K' (which is uniformly random since hybrid 2) back to the "real" $\mathsf{key}_1(a, B)$ in the re-encryption case — should change *both* K and K' back to $\mathsf{key}_1(a, B)$, because here K and K' need to be the same. This hybrid goes through due to KA security. □

B.2 EKE Using IC Realizes $\mathcal{F}_{\text{PAKE}}$

We now turn to protocol EKE using IC, which is the EKE protocol in Sect. B.1 except that POPF Program is replaced by IC encryption and POPF Eval is replaced by IC decryption. we have:

Theorem B.2. *The EKE using IC protocol realizes $\mathcal{F}_{\text{PAKE}}$ in the $(\mathcal{F}_{\text{IC}}, \mathcal{F}_{\text{EKE-1r}})$-hybrid world.*

Proof (sketch). This is simpler than Thm. B.1 so we only provide a more high-level sketch. The only modifications from the EKE-PRF simulator in Figure 18 are:

- In a NewKey command, we replace $\mathsf{PRF}_{K^*}(\phi')$ with K^*;
- We simulate the interface of $\mathcal{F}_{\text{IC}}$ rather than $\mathcal{F}_{\text{POPF}}$.

The key point is that now *the re-encryption case becomes non-existent*, since it is impossible to have $\phi' = \mathcal{E}(\mathsf{pw}, B)$, $(\phi')^* = \mathcal{E}(\mathsf{pw}, B)$, and $(\phi')^* \neq \phi'$. So we simply remove hybrids 8 and 9, and (as in the proof of Thm. B.1) also remove hybrids 5, 6, and 7. □

Remark B.3. *Our $\mathcal{F}_{\text{PAKE-sp}}$ functionality accurately captures the real adversary's ability while attacking EKE instantiated with POPF. The only additional attack compared to EKE using IC is re-encryption, where the adversary only attacks the P session but can simultaneously check the password of P'* if the password guess for P is correct *(in which case P and P' output the same key if and only if the passwords of P and P' match). This "conditional same password check" is exactly what the* TestSamePwd *command does.*

C Properties of Kyber

In this section we argue that Kyber (Sect. 2.1) satisfies correctness, security, strong pseudorandomness, pseudorandom non-malleability, and collision resistance — properties that are needed for the security of (O)EKE.

For correctness, the idea is that $\mathsf{Kyber.Dec}(a, B) = b$, so key_1 will see B as being a valid message (i.e., $B = \mathsf{msg}_2(b, A)$) and will output $H'(b, A, B)$, the same as key_2. Security and pseudorandomness follow easily from the corresponding properties of the underlying public-key encryption scheme:

CPA-security, pseudorandom public keys, and pseudorandom ciphertexts.

By applying the FO transform we have added the properties of strong pseudorandomness, pseudorandom non-malleability, and collision resistance. Collision resistance is the easiest to see: key_1 always outputs either $H'(b, A, B)$ or $H'(a, B)$, and in both cases something that uniquely identifies A is included in the RO. Therefore, collision resistance holds as long as H' has sufficient output length for τ to be at least 2κ-bit long.

For strong pseudorandomness, notice that a uniformly random ciphertext B has negligible probability of equaling $\mathsf{msg}_2(\mathsf{Kyber.Dec}(a, B), A)$, since msg_2 is defined using $H(b, A)$, and this will be a fresh RO query. (Technically, this requires $\mathsf{Kyber.Enc}$ to preserve sufficient entropy from $H(b, A)$; this is true for Kyber.) Therefore, $\mathsf{key}_1(a, B) = H'(a, B)$ on uniformly random B, so the key is indistinguishable from uniformly random. This shows that (real A, random B, real K) is indistinguishable from (real A, random B, random K), which (by Sect. 3.3) is sufficient to show strong pseudorandomness.

Finally, we need to show pseudorandom non-malleability. Normal non-malleability is implied by the CCA-security of the FO transform, which suggests that we should look for a similar argument to show pseudorandom non-malleability. The main idea of the FO transform is that on any adversarially generated B, one can look through all $H(b, A)$ queries made by the adversary and see whether it matches any of them. If it does, then we already know b, and can simulate the output of key_1 without knowing a by just computing $H'(b, A, B)$. If there is no such query, then there is negligible probability that $B = \mathsf{msg}_2(\mathsf{Kyber.Dec}(a, B), A)$, as either $H(\mathsf{Kyber.Dec}(a, B), A)$ is a fresh query and will add enough entropy to msg_2 to stop it from matching B, or it is one of the previous $H(b, A)$ queries that are already known to not produce the correct B. In that case, key_1 can be simulated by just outputting a uniformly random value, since $H'(a, B)$ is indistinguishable from random. Therefore, the FO transform allows any key_1 to be simulated without knowledge of a, and without key_1 pseudorandom non-malleability is just a combination of security and pseudorandomness.